

Helix Constitution — Universal CLAUDE.md

Field	Value
Revision	34
Created	2026-05-14
Last modified	2026-06-22T00:00:00Z
Status	active
Status summary	§11.4.166 REPEALED (operator decision 2026-06-22): Universal Semgrep static-analysis mandate repealed — Semgrep no longer mandatory; scaffolding, submodules/semgrep, MCP/pre-commit/PATH wiring, docs_chain semgrep_status context, and CM-COVENANT-114-166-PROPAGATION/CM-SEMGREP-WIRED gates removed. Added §11.4.163 mirror — Universal Media Validation & Verification Mandate: every recorded artifact MUST pass MEDIA VALIDATION pipeline before acceptance (OCR/transcription text parsing vs SPECIFY-phase patterns, self-validated analyzer §11.4.107(10), structured verdict with pinpoint on FAIL, post-recording+real-time triggers, paired §1.1 mutation). Added §11.4.164 mirror — Universal Constitution Auto-Propagation & Hook System: post_update_hook.sh detects/registers/installs changed skills/MCP/hooks/scripts; consumer invokes after every constitutional pull. Added §11.4.165 mirror — Universal Independent Verification Agent Mandate: every output passes INDEPENDENT verifier (§11.4.70/.20), iterates to zero-finding GO (§11.4.134). All Classification: universal (§11.4.17). Prior round: Added §11.4.162 mirror — OpenDesign UI design system mandate (User mandate 2026-06-21): every project producing user-facing interfaces MUST use OpenDesign as the mandatory UI design-and-refinement system — NOT ad-hoc CSS or one-off design tools; install as a project dependency and use its design tokens/themes for color palette (light+dark), typography, spacing, and component-level tokens; extend upstream per §11.4.74 for missing patterns; every UI component ships light+dark variants with project brand colors from canonical assets; elements MUST NOT overlap or overlay labels; all UI changes covered by standard test types including visual regression. Classification: universal (§11.4.17). Prior round: Added §11.4.161 mirror — Rootless container runtime mandate (User mandate 2026-06-21): every project MUST use Podman in rootless mode (or equivalent rootless container

Field	Value
	runtime) for ALL containerized workloads — Docker in rootful mode, sudo, or any escalation to root is FORBIDDEN unless the target platform has no rootless option AND that constraint is documented per §11.4.112; the <code>vasic-digital/containers</code> submodule (§11.4.76) MUST be used as the sole container orchestration layer — no ad hoc docker/podman commands outside <code>pkg/boot / pkg/compose / pkg/health</code> ; missing capabilities extend the container Submodule upstream per §11.4.74; integration tests boot infra on-demand via the Submodule. Classification: universal (§11.4.17). Prior round: Added §11.4.160 mirror — Vision-verified recording + HelixQA bridge mandate (User mandate 2026-06-21): every video recording for feature/QA evidence MUST be processed through a vision/OCR pipeline that reads on-screen content and confirms expected results BEFORE acceptance; the recording system MUST provide a bridge feeding captured frames to HelixQA's test infrastructure (or equivalent) for content verification — automated read-the-screen against specified expected patterns; the bridge MUST capture frames at ≤5s intervals, run OCR/vision analysis with self-validated analyzer (§11.4.107(10)), compare against SPECIFY-phase patterns, produce per-frame PASS/FAIL with evidence path, surface failures immediately for re-record per §11.4.159(L). Honest boundary: vision verification confirms on-screen content — does NOT replace §11.4.5 quality analysis nor §11.4.108 runtime-signature; FLAG_SECURE surfaces use the §11.4.117 proxy oracle. Classification: universal (§11.4.17). Prior round: Added §11.4.153 mirror — Comprehensive per-feature Status + Status_Summary document set with mandatory video-recording confirmation (User mandate 2026-06-15): every project MUST maintain under docs/features/ a feature Status set (Status.md + §11.4.56 Status_Summary.md) enumerating EVERY component/client-app/binary/surface + EVERY feature (incl. ported from cli_agents §11.4.74), NO feature left out, reconciled vs codebase (§11.4.6/.118); per-feature fields Component/Feature/Category/Implementation/Wiring(§11.4.108)/Real-use/Tests-coverage(§11.4.4(b))/Validation(PASS/FAIL/SKIP/PENDING_FORENSICS/OPERATOR-BLOCKED §11.4.45) Video-confirmation; every user-visible confirmed claim backed by a recorded REAL-USE video (real prompts → real LLM/service responses → real results, no frozen frame §11.4.107, no faked/bluff response §11.4.2/.5), autonomous-infeasible⇒honest §11.4.3/.52 SKIP; video-analysis remediation loop §11.4.102/.146/.134; always-in-sync §11.4.45/.106 docs_chain + §11.4.86 fingerprint; four-format export HTML+PDF+DOCX (adds DOCX to §11.4.65 for this class). Propagation gate <code>CM-COVENANT-114-153-PROPAGATION</code> + recommended gates <code>CM-FEATURE-STATUS-COMPLETE</code> + <code>CM-FEATURE-STATUS-VIDEO-CONFIRMED</code> . Classification: universal (§11.4.17). Prior round: Added §11.4.152 mirror — Crashlytics-recorded-data continuous monitoring + systematic-debug + regression-test-coverage mandate (User mandate 2026-06-13): every project with Firebase Crashlytics enabled/wired MUST continuously monitor ALL four recorded surfaces (fatal crashes, ANRs, performance traces, AND non-fatals), systematic-debug

Field	Value
	each (reproduce-before-fix §11.4.102/.115), fix/improve, and cover every closure with a permanent §11.4.135 regression guard (RED-on-broken → GREEN-on-fixed) + a closure log citing the console issue id/URL + the validation/verification test paths; regular cadence (the §11.4.47 five-trigger set), no false results, no bluff — a console-resolved Crashlytics issue with no falsifiable regression test is FORBIDDEN (the silent-recurrence vector). STRENGTHENS+COMPLETES §11.4.47 (review/dedup finds it; §11.4.152 fixes it + proves it stays fixed). Project-level reference instantiation = a consuming project's §6.O + §6.AC. Composes §11.4/.1/.5/.6/.9/.34/.40/.43/.47/.69/.90/.102/.107/.108/.115/.118/.123/.130/.135/.138/.150 §1.1. Propagation gate CM-COVENANT-114-152-PROPAGATION + recommended gate CM-CRASHLYTICS-ISSUE-FULLY-COVERED . Classification: universal (§11.4.17). Prior round: Added §11.4.151 mirror — Project-prefixed release-tag/version-naming mandate (User mandate 2026-06-12): every release tag AND version name on the main repo AND every owned submodule MUST be prefixed <PREFIX>-<version> (e.g. myproject-1.0.0-dev-0.0.1); prefix resolution order (1) HELIX_RELEASE_PREFIX from .env (authoritative, git-ignored §11.4.30, documented in tracked .env.example §11.4.77) else (2) lowercased snake_case project-root dir name §11.4.29; SAME prefix across main + all owned submodules in one release = greppable across every repo; version codes increment monotonically. Composes §2.1/ §11.4.28/.29/.30/.40/.77/.113/.126. Propagation gate CM-COVENANT-114-151-PROPAGATION + recommended gate CM-RELEASE-PREFIX-NAMING . Classification: universal (§11.4.17). Prior round: Added §11.4.148 + §11.4.149 mirrors — workable-item integrity + testing-diary family (User mandate 2026-06-10). §11.4.148 BINDS+STRENGTHENS §11.4.15/.16/.21/.54/.91/.93/.95/.106 into one DB↔docs↔external-tracker integrity contract: (D1) no item without a valid status+type+stable id on ALL three surfaces; (D2) comprehensive structured description (what/manifest/repro/acceptance); (D3) BLOCKED items carry WHY + enumerated unblock CHOICES (tightens §11.4.21; Blocked / BLOCKED = documented alias of canonical operator-blocked); (D4) regular never-missed bidirectional DB↔docs↔tracker sync, §11.4.86-fingerprinted + §11.4.106 docs_chain-bound; (D5) generic idempotent external-tracker push (statuses/types/assignee-from-env/sub-tasks, sink-side proof §11.4.69, machinery project-agnostic §11.4.28). §11.4.149 per-workable-item TESTING DIARY (date/time + tested-by + result + observations + action+why), distinct from item_history/Reopens: append-only test_diary table with PASS-requires-evidence schema constraint + in-depth diary + derived summary VIEW + four-format per-item exports §11.4.86-fingerprinted §11.4.106-bound + external-tracker SUB-TASK {TODO In-progress Completed} lifecycle + minimal-LLM deterministic tooling 100% test-covered §11.4.27. Both Classification: universal (§11.4.17); project-specific dual per-display brightness/auto-dim/font-size landed ONLY in the consumer layer

Field	Value
	(RK3588/Presenter), NOT here. Propagation gates CM-COVENANT-114-148-PROPAGATION + CM-COVENANT-114-149-PROPAGATION + paired §1.1 mutations. Prior round: Added §11.4.147 mirror — Crashed-agent respawn-until-complete + no-work-loss registry mandate (User mandate 2026-06-10): every dispatched agent/subagent MUST be tracked through its full lifecycle so a crash NEVER loses/forgets/corrupts its work (crash ≠ done, §11.4.6) — (a) durable append-only agent REGISTRY (status CLOSED SET {dispatched in-flight crashed respawned complete}) on the §11.4.116 syn substrate, SINGLE SOURCE OF TRUTH for “is this work owed?”; (b) mechanical CRASH-DETECTION (transient rate-limit/socket-close) → flip to crashed + keep OPEN → RESPAWN until complete autonomously per §11.4.101 with ALREADY-DEFINED backoff; (c) PARTIAL-STATE preserve → §11.4.84-check → resume-or-clean restart (§9.2 backup) in a per-agent worktree; (d) the §11.4.87/.94/.97/.126 loop done-condition MUST NOT read satisfied while any entry is non- complete . Forensic FACT this session: 5 subagents killed by rate-limit + 2 by socket-close + one PARTIAL dual-sliders edit, re-dispatched by pure conductor vigilance with NO mechanical guarantee; agent-side analogue of §11.4.144. Composes §11.4.6/.58/.84/.87/.94/.97/.101/.116/.102/.108/.125/.142/.40/.128/.144/§9.2/§1.1. Classification: universal (§11.4.17). Prior round: Added §11.4.144 mirror — Tracked/recorded-device availability-following mandate (User mandate 2026-06-10): every device the project is tracking / following / recording (test / debug / manual-testing, across every reachable transport — USB / wireless ADB / SSH / serial / network introspection API) MUST be availability-FOLLOWED — connection state continuously monitored, any drop handled, never silently abandoned and never presented as a continuous recording; the §11.4.128 always-on recorder KNOWS a tracked device is absent (its loop guards on the reachable state) but, lacking a following discipline, merely spins idle (no data, no offline event, no resume, no escalation) → silent corpus hole = §11.4 PASS-bluff at the recording-integrity layer. On a drop the system MUST automatically (a) DETECT + log an honest offline event (§11.4.6 fabricated-continuity bluff; §11.4.128 always-on gap), (b) WAIT using the project’s ALREADY-DEFINED reconnection timings — never invented (§11.4.6), the SAME grace/reconnect/poll budgets the recovery path already uses, (c) RE-ATTACH + log an honest online/resume event the moment the device returns, (d) ESCALATE to the sanctioned device-recovery path (per §11.4.69 feature class device_recovery) if the device does not return within the defined timeout — through the sanctioned, authorization-gated recovery entry point ONLY, never bypassing its gate never performing a destructive recovery (power-cycle) autonomously without that authorization (§11.4.21/.101 — high-blast-radius recovery is gated; while blocked keep following + log the blocked-escalation honestly). Composes §11.4.128/.69/.6/.14/.21/.101. Classification: universal (§11.4.17). Prior round: Added §11.4.143 mirror — Real-user-journey mandate for video-streaming-app full-automation

Field	Value
	tests (User mandate 2026-06-10, verbatim “All video streaming apps ... require to choose some title and to press proper UI button to start or resume playing! Proper UI interaction to play exact show with proper content and subtitles is MANDATORY! Without it we just eventually play on 2nd display sample, and that’s it mostly!”): any full automation test asserting a video player / streaming app plays content MUST drive the REAL end-user journey through the app’s OWN UI — launch → BROWSE the actual catalog → choose a SPECIFIC title → press the real Play/Resume button → confirm THAT chosen content plays with its correct subtitles on the intended routing target — NEVER a sample / demo / loop-clip / deep-link / <code>am start -a VIEW</code> / synthetic shortcut (those validate ROUTING only, not real chosen-content playback → a §11.4 PASS-bluff at the user-journey layer); non-introspectable streaming UIs use the §11.4.117 CV/OCR pixel oracle; login via the §11.4.10 credential single-source; chosen-content confirmation via §11.4.107 liveness + §11.4.136 real-content + §11.4.137 subtitle correctness; honest <code>operator_attended</code> SKIP-with-reason per §11.4.52/.3 ONLY where autonomous is genuinely infeasible (hard login / geo-block / secure-surface blanking) — NEVER a faked routing-only PASS. Composes §11.4.48/.107/.117/.136/.137/.52/.3/§107/§1.1. Classification: universal (§11.4.17). Prior round: Added §11.4.142 mirror — Universal code-review mandate — every change reviewed, always, no exception (User mandate 2026-06-09, verbatim “ALL changes we do MUST pass through the code review step!!! ALWAYS!!!”): EVERY change to ANY governed repository — no exception (source/fixes/tests/gates/mutations/docs/doc-tooling/build-CI/config/governance/conductor-edits/sub-agent-output/refactors/one-liners) — MUST pass an INDEPENDENT code-review step BEFORE acceptance/commit/build; the ABSOLUTE form of §11.4.125 (no “just a doc edit”/“just a one-liner”/“self-review §11.4.92 suffices”/“trivial change” carve-out); independence load-bearing (reviewer structurally separated from author per §11.4.70/.20, §11.4.92 self-review PRECEDES never satisfies); anti-bluff (§11.4/.1, conclusions captured-evidence §11.4.5/.69) + iterates to clean GO per §11.4.134; honest boundary §11.4.6 — does NOT replace §11.4.108/.40, it is the FIRST of MULTIPLE STRONG LAYERS. Composes §11.4.1/.4/.5/.6/.20/.40/.69/.70/.92/.108/.110/.125/.134/§107/§1.1. Classification: universal (§11.4.17). Prior round: Added §11.4.132 mirror — Risk-ordered validation priority mandate (User mandate 2026-06-07): tests/validations/verifications MUST run in RISK-DESCENDING order — the highest-risk set FIRST (closed factors: (a) most-recently-worked, (b) historically most-problematic, (c) highest crash/break/regress likelihood, (d) most-reopened per §11.4.55), each validated GREEN with real (physical) captured evidence (§11.4.5/.69/.107, no bluff §11.4.6), and ONLY AFTER that set is GREEN does the rest of the suite run; running the suite in arbitrary order or lower-risk-before-highest-risk-GREEN is a §11.4.132 violation; REFINES/STRENGTHENS §11.4.130 (generalises “validate-just-fixed-first” to the full risk-ordered set) + §11.4.46 (adds explicit risk-ordering) + §11.4.42 (applies implementation-priority

Field	Value
	discipline to VALIDATION ordering). Composes §11.4.4/.5/.6/.7/.40/.42/.46/.50/.55/.69/.107/.130. Classification: universal (§11.4.17). Prior round: Added §11.4.131 mirror — Standing session-resumption file mandate (User mandate 2026-06-07): every project MUST maintain a SINGLE canonical always-current session-resumption FILE at a fixed project-declared standard path (declared once per §11.4.35, never moved without a §11.4.66 operator decision) — the OUT-OF-THE-BOX entry point for any fresh session (new session = ONLY point the agent at this one file); §11.4.131 promotes §11.4.127 (PREPARE-on-demand) into a STANDING version-controlled ARTIFACT, ALWAYS present + ALWAYS in sync: (A) exists at the declared path at all times; (B) (re)written whenever a fresh session is needed OR live state materially changes (new HEAD/build-artifact id/phase/device state/in-flight job/blocking decision) — §12.10 trigger set, a stale file = §11.4.131 violation (same class as §12.10 stale-CONTINUATION); (C) carries SHORT+FULL §11.4.127 variants + points to handoff docs read-FIRST + git fetch + exact live-state anchors + PHASE/NEXT/terminal-goal + binding constraints, MOMENT-VALID never generic (§11.4.6); (D) §11.4.65 export + §11.4.44 revision header; (E) given ONLY this file's path a fresh session fully resumes with zero additional context. Composes §12.10/.127/.65/.44/.6/.66/.126. Classification: universal (§11.4.17). Prior round: Added §11.4.127 mirror — Session-handoff resumption-prompt mandate (User mandate 2026-06-06): when a fresh session is needed (context limits / degradation) OR the operator asks whether a new session is needed, the agent MUST ALWAYS prepare + proactively provide a ready-to-paste resumption prompt valid for that EXACT moment + project phase (a SHORT first-sentence variant + a FULL detailed block on demand) that points to the live handoff doc (.remember/remember.md if present + docs/CONTINUATION.md per §12.10, read FIRST + git fetch –all), states current PHASE + immediate NEXT action + terminal goal, embeds exact live-state anchors (build IDs/artifact MD5, device serials, commit HEAD, in-flight PIDs + log paths, captured-evidence paths), and restates binding constraints (anti-bluff §11.4, no-force-push §11.4.113, exact version/naming, hardware gotchas); MUST be moment-valid never a generic template; handoff docs MUST be current BEFORE the prompt is given; a missing/stale/generic prompt is a §11.4.127 violation. Composes §12.10/.6/.66/.87/.103/.126. Classification: universal (§11.4.17). Prior round: Added §11.4.126 mirror — Default autonomous-loop working mode from first prompt (User mandate 2026-06-04): the endless fully-autonomous loop is the DEFAULT working mode, engaged automatically the moment the operator sends the FIRST request/prompt of a session — no per-session activation handshake; the loop continues until EITHER (A) a new fully-validated-and-verified version (tag) is created AND published across all owned submodules + main repo to all remotes (per §2.1/§11.4.40/§11.4.113), OR (B) for non-release main-stream work that work is fully completed AND all side work streams done AND nothing is left in the working queue for the current scope; STOPS ONLY on

Field	Value
	explicit operator STOP, empty current-scope queue, or §12 host-safety; goal ABSOLUTE EFFICIENCY; mimicking/imitation/false-results/bluff of ANY kind ABSOLUTELY FORBIDDEN (composes the §11.4 anti-bluff covenant — actually perform the work, never narrate/pretend it); §11.4.126 is the CAPSTONE promoting §11.4.87 from opt-in to always-on. Composes §11.4.87/.94/.97/.101/.103/.66/.6/.40/.42/.72/.113 + §2.1 + §12. Classification: universal (§11.4.17). Prior round: Added §11.4.125 mirror — Code-review agent gate before pre-build + main build (mandatory multi-layer review) (User mandate 2026-06-04): after all batch fixes/changes/implementations are done and BEFORE the pre-build test sweep + main (artifact) build, the agent MUST dispatch dedicated code-review agent(s) (subagent-driven per §11.4.70/.20) that analyze all batch work + all existing data/facts/captured-evidence + the existing codebase (blast radius) + current git history, determine quality + safety + will-it-REALLY-work, and validate+verify every covering test genuinely exercises the work-under-test and catches its negation with ZERO false-result/bluff possibility; any finding (defect / error-prone change / safety risk / will-not-work / bluff-capable test / coverage gap) MUST be fixed+polished+improved+test-covered (four-layer §11.4.4(b), TDD-RED-first §11.4.43/.115) BEFORE the build proceeds; review iterates until no blocking findings; one of MULTIPLE STRONG LAYERS complementing (never replacing) the pre-build sweep + §11.4.92 multi-pass (author-side) + §11.4.108 + §11.4.110. Composes §11.4/.1/.4/.6/.40/.43/.50/.70/.20/.92/.102/.107/.108/.110. Classification: universal (§11.4.17). Prior round: Added §11.4.124 mirror — Dead/unwired-code investigate-before-remove mandate (User mandate 2026-06-04): “zero importers ⇒ dead ⇒ delete” is a guess (§11.4.6), never a finding — before removing ANY seemingly-dead element (zero-importer / never-called / unwired function/type/file/module/package/asset/config/build-target) the agent MUST FIRST investigate via git history (<code>git log --follow , -s / -G</code> pickaxe, blame on the deleted call-site) and capture as FACT where/how it was wired in + when/how it became dead + whether “no references” is real or a hidden reference (reflection/DI/build-tags/codegen/plugin/FFI/config) the static tool cannot see; removal is permitted ONLY with captured PROOF it is genuinely unneeded AND MUST be its own separate descriptive commit citing the git-history evidence (+ §11.4.122 operator-confirmation for end-user capabilities; tracked via §11.4.90 Obsolete); with NO such proof the element MUST NOT be removed — instead wire it in properly (restore a mistakenly-deleted call-site per §11.4.114; finish never-completed wiring) and add any missing/unwired tests (§11.4.27/.43/.115); ALWAYS be extra careful — when uncertain, default to NOT removing per §11.4.6/.101/.122. Classification: universal (§11.4.17). Prior round: Added §11.4.123 mirror — Rock-solid-proof-or-deep-research mandate (User mandate 2026-06-03): every reported issue / fix / claimed completion MUST be 100% validated with rock-solid CAPTURED proof (§11.4.5/.69/.107) before fixed/implemented/ completed; metadata-only / config-only / absence-of-error / grep-without-runtime PASS

Field	Value
	forbidden (§11.4/.1); when UNSURE how to validate, the agent MUST ALWAYS first perform deep web research (§11.4.8+§11.4.99) to discover/build an evidence-producing method — declaring “untestable” or accepting a metadata-only PASS without exhausting that research is itself a §11.4.123 violation; forensic case study (FACT) 1.1.8-dev FLAG_SECURE + non-introspectable-UI validation unclear → deep research yielded the CV/OCR/liveness/sink-probe oracle stack (§11.4.107/.112/.117). Classification: universal (§11.4.17). Prior round: Added §11.4.122 mirror — No-silent-removal-of-existing-components-without-operator-confirmation mandate (User mandate 2026-06-03): no application/component/service/package/feature/driver/module/prebuilt — any already-existing end-user capability — may be removed from the existing codebase/System without FIRST interactively asking the operator (§11.4.66, never silent/autonomous) + a EXPLICIT keep-or-remove decision; silent removal is a release blocker; forensic case study F2 Apple-TV-class app + F4 Huawei HMS component removed without asking, operator reversed both; tracked DROP path ask → approve → <code>obsolete</code> reason <code>feature-removed</code> +citation (§11.4.90) → remove. Classification: universal (§11.4.17). Prior round: Added §11.4.114–§11.4.121 mirrors (eight new universal anchors from the 1.1.8-dev live-defect remediation, 2026-06-03): §11.4.114 last-known-good-tag regression isolation (diff the broken state against the known-good tag FIRST — the tagged-good version is the regression oracle), §11.4.115 RED-baseline-on-the-broken-artifact + polarity-switch (one test reproduces the defect on the pre-fix artifact then flips RED_MODE=0 to GREEN-guard — refines §11.4.43), §11.4.116 real-time conductor↔autonomous-test-framework sync channel (append-only JSONL event stream + atomically-rewritten status snapshot the orchestrator tails live; every verdict event carries its evidence path), §11.4.117 CV/OCR pixel-oracle fallback for non-introspectable UIs (drive + assert via pixels when the accessibility tree is blank — refine §11.4.48/.52/.107), §11.4.118 discovery-pressure to confirm known-issue-set completeness (fixing the reported set is necessary-not-sufficient; provable enumerated discovery coverage), §11.4.119 single-resource-owner partitioning for parallel hardware testing (exactly ONE stream owns each device’s exclusive resource — refines §11.4.58/.103), §11.4.120 fix-breaks-its-own-gate reconciliation (rewrite the gate to assert the NEW mechanism; never fake-pass, never revert the fix), §11.4.121 no-commit-while-build-writes-tracked-artifacts (defer commit to build completion so partial/stale artifacts never land — build-output analogue of §11.4.84). §11.4.108–§11.4.113 + §11.4.78–§11.4.107 mirrors continue from earlier Revisions.
Issues	none
Issues summary	—

Field	Value
Fixed	§11.4.108 mirror
Fixed summary	§11.4.107 lands in lockstep with the Constitution.md §11.4.107 addition.
Continuation	—

Table of contents

- [How inheritance works](#)
- [MANDATORY DEVELOPMENT PRINCIPLES](#)
- [MANDATORY ANTI-BLUFF COVENANT — END-USER QUALITY GUARANTEE](#)
 - [§11.4.1 — FAIL-bluffs are equally forbidden](#)
 - [§11.4.2 — Recorded-evidence requirement](#)
 - [§11.4.3 — Per-environment-topology test dispatch](#)
 - [§11.4.4 — Test-interrupt-on-discovery + retest-from-clean-baseline](#)
 - [§11.4.5 — Captured-evidence quality analysis](#)
 - [§11.4.6 — No-guessing mandate](#)
 - [§11.4.7 — Demotion-evidence rule](#)
 - [§11.4.8 — Deep-web-research-before-implementation](#)
 - [§11.4.9 — Batch-source-fixes-before-rebuild](#)
 - [§11.4.10 — Credentials-handling mandate](#)
 - [§11.4.10.A — Pre-store credential leak audit \(User mandate, 2026-05-17\)](#)
 - [§11.4.11 — File-layout discipline](#)
 - [§11.4.12 — Auto-generated docs sync](#)
 - [§11.4.13 — Out-of-band sink-side captured-evidence](#)
 - [§11.4.14 — Test playback cleanup](#)
 - [§11.4.15 — Item-status tracking](#)
 - [§11.4.16 — Item-type tracking](#)
 - [§11.4.17 — Universal-vs-project classification of new rules \(User mandate, 2026-05-14\)](#)

- §11.4.20 — Subagent-driven-by-default mandate (User mandate, 2026-05-14)
- §11.4.18 — Script documentation mandate (User mandate, 2026-05-14)
- §11.4.19 — Fixed-document column-alignment mandate (User mandate, 2026-05-14)
- §11.4.21 — Operator-blocked status + self-resolution exhaustion (User mandate, 2026-05-14)
- §11.4.22 — Document-sync commit discipline (User mandate, 2026-05-14)
- §11.4.24 — Build-resource stats tracking mandate (User mandate, 2026-05-14)
- §11.4.25 — Full-Automation-Coverage Mandate (User mandate, 2026-05-15)
- §11.4.26 — Constitution-Submodule Update Workflow Mandate (User mandate, 2026-05-15)
- §11.4.31 — Submodule-Dependency-Manifest Mandate (User mandate, 2026-05-15)
- §11.4.32 — Post-Constitution-Pull Validation Mandate (User mandate, 2026-05-15)
- §11.4.30 — .gitignore + No-Versioned-Build-Artifacts Mandate (User mandate, 2026-05-15)
- §11.4.29 — Lowercase-Snake_Case-Naming Mandate (User mandate, 2026-05-15)
- §11.4.28 — Submodules-As-Equal-Codebase + Decoupling + Dependency-Layout Mandate (User mandate, 2026-05-15)
- §11.4.27 — No-Fakes-Beyond-Unit-Tests + 100%-Test-Type-Coverage Mandate (User mandate, 2026-05-15)
- §11.4.33 — Type-aware closure-status vocabulary (User mandate, 2026-05-15)
- §11.4.34 — Reopened-source attribution mandate (User mandate, 2026-05-15)
- §11.4.35 — Canonical-root inheritance clarity (User mandate, 2026-05-15)
- §11.4.36 — Mandatory install_upstreams on clone/add Mandate (User mandate, 2026-05-15)
- §11.4.37 — Fetch-before-edit mandate (User mandate, 2026-05-15)

- §11.4.38 — Installable-Asset Evidence Mandate (User mandate, 2026-05-17)
- §11.4.40 — Full-suite retest before release tag mandate (User mandate, 2026-05-17)
- §11.4.41 — Pre-Force-Push Merge-First Mandate (User mandate, 2026-05-17)
- §11.4.42 — Iteration-discipline mandate (User mandate, 2026-05-18)
- §11.4.43 — TDD-Fix-Discipline mandate (User mandate, 2026-05-18)
- §11.4.44 — Document revision header mandate (User mandate, 2026-05-18)
- §11.4.45 — Integration-status-doc maintenance mandate (User mandate, 2026-05-18)
- §11.4.46 — Validate-recent-work-before-post-flash-tests mandate (User mandate, 2026-05-18)
- §11.4.47 — Firebase data review mandate (User mandate, 2026-05-18)
- MANDATORY HOST-SESSION SAFETY (Constitution §12)
 - Forbidden — directly OR indirectly
 - Required safeguards for heavy scripts
 - §12.6 Memory-Budget Ceiling — 60% MAXIMUM
 - §12.10 Continuation document maintenance
- MANDATORY ABSOLUTE DATA SAFETY — ZERO RISK (Constitution §9)
- MANDATORY COMMIT & PUSH CONSTRAINTS
- MANDATORY TESTING CONSTRAINTS
- Code conventions
- When in doubt

This is the **base CLAUDE.md** imported by every project that includes the Helix Constitution submodule. Project-level `CLAUDE.md` may extend or tighten any rule by adding an explicit `override: <section>` block — but MUST NOT weaken them.

Last revision: 2026-05-14

How inheritance works

A consuming project's root `CLAUDE.md` MUST start with a clearly-marked inheritance pointer:

```
## INHERITED FROM constitution/CLAUDE.md

All rules in `constitution/CLAUDE.md` (and the `constitution/
Constitution.md`
it references) apply unconditionally. The project-specific rules below
extend them.
```

Claude Code supports the `@path/to/file` import syntax natively, so a consuming project can also write `@constitution/CLAUDE.md` at the top of its own `CLAUDE.md` and Claude Code will resolve it recursively. For agents that do not support `@imports`, the pointer-block pattern above ensures the inheritance is at least readable.

MANDATORY DEVELOPMENT PRINCIPLES

NO BLUFF. Every change ships with positive-evidence validation on the real target environment. A test that passes without exercising the user-visible behaviour is a critical defect.

(Constitution §7.1 + §11.4 — these references point to `constitution/Constitution.md`.)

CRITICAL: All code changes MUST follow these principles WITHOUT EXCEPTION:

1. **Solutions MUST NOT be error-prone.** Every fix must be robust, not introduce new failure modes. If a fix solves problem A but creates problem B, it is NOT acceptable. Test the fix against ALL existing functionality before committing.
2. **No blocking operations inside synchronized / shared-lock regions.** Long computations, network calls, or sleeps inside `synchronized` / `Lock`-held regions cause deadlocks or timeouts in other threads.
3. **Always consider concurrent callers.** Any method called from multiple threads must be safe for rapid consecutive calls.

4. **Test the fix, not just the symptom.** Verify the fix works AND doesn't break anything else.
5. **Anti-bluff is mandatory** (Constitution §7.1) — every runtime test MUST source the project anti-bluff helper, call at least one explicit user-visible action before any PASS, capture state delta, and assert positive evidence.
6. **Real captured evidence for audio/video** (Constitution §7.1 + §11.4.5) — features producing audio output validated via captured audio; features producing video output validated via captured frames + OCR or pixel-diff.

MANDATORY ANTI-BLUFF COVENANT — END-USER QUALITY GUARANTEE

Forensic anchor — verbatim user mandate (2026-04-28):

“We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!”

This is the historical origin of the project's anti-bluff covenant. Every test, every Challenge, every gate, every mutation pair exists to make the failure mode (PASS on broken-for-end-user feature) mechanically impossible.

Operative rule: the bar for shipping is **not** “tests pass” but **“users can use the feature.”** Every PASS MUST carry positive evidence captured during execution that the feature works for the end user. Metadata-only PASS, configuration-only PASS, “absence-of-error” PASS, and grep-based PASS without runtime evidence are all critical defects regardless of how green the summary line looks.

Tests AND Challenges (HelixQA, integration suites, smoke tests, acceptance suites) are bound equally — a Challenge that scores PASS on a non-functional feature is the same class of defect as a unit test that does.

Canonical authority: `constitution/Constitution.md` §11.4 and its sub-sections §11.4.1 through §11.4.16.

Non-compliance is a release blocker regardless of context.

§11.4.1 — FAIL-bluffs are equally forbidden

A test that crashes for a script-internal reason (undefined variable under `set -u`, regex error, malformed assertion, missing argument) and produces a FAIL exit code is just as misleading as a PASS-bluff. Both let real defects ship undetected. Every test MUST fail ONLY for genuine product defects — script-bug failures must be fixed at the source layer (helper library, shared lib, test source), not patched in individual call sites.

§11.4.2 — Recorded-evidence requirement

A test that emits PASS without captured visual or audio evidence of the user-visible feature actually working on the screen the user would see is a §11.4 PASS-bluff. Every PASS for a user-visible feature MUST be cross-checked against captured recording + action timeline.

§11.4.3 — Per-environment-topology test dispatch

Tests that depend on environment topology MUST detect topology at test entry and dispatch the topology-appropriate variant. SKIP-with-reason is the correct fallback when the required topology is absent; PASS-by-default is forbidden.

§11.4.4 — Test-interrupt-on-discovery + retest-from-clean-baseline

The moment any defect is re-discovered, re-produced, or newly identified during a test cycle, the cycle MUST stop. Then: systematic debugging → fix at root cause → four-layer test coverage (pre-build / post-build / runtime / meta-test paired mutation) → full rebuild → re-deploy on every target → full retest from beginning.

§11.4.5 — Captured-evidence quality analysis

Audio: presence (RMS amplitude), channel count, sample rate + bit depth, glitch census, coexistence-artifact census. Video: presence (non-zero frame count),

routing target, frame health (drops / jitter / freeze), obstruction census (OCR for hostile overlays), resolution + codec. Every check is required for every PASS.

§11.4.6 — No-guessing mandate

Forensic anchor — verbatim user mandate (2026-05-08):

“‘LIKELY’ is guessing, we MUST NOT have guessing, since it can be or may not be! No bluffing and uncertainty is allowed at any cost! We MUST always know exactly precisely what is happening exactly, in any context, under any conditions, everywhere!”

Forbidden vocabulary in tests / gates / status reports / closure narratives / commit messages when describing causes: likely , probably , maybe , might , possibly , presumably , seems , appears to , guess , seemingly , apparently , perhaps , supposedly , conjectured , and synonyms.

Either prove the cause with captured forensic evidence and state it as fact, OR explicitly mark UNCONFIRMED: / UNKNOWN: / PENDING_FORENSICS: with a tracked-task ID for follow-up.

§11.4.7 — Demotion-evidence rule

Demotion from a FAIL classification to a lower-severity classification requires positive evidence captured under the SAME CONDITIONS (same target, same build, same cycle position, same load profile) that originally exposed the defect. “I cannot reproduce in isolation” is a hypothesis, not a finding.

§11.4.8 — Deep-web-research-before-implementation

Before designing a non-trivial fix, perform deep web research: official docs, vendor technical guides, open-source codebases, coding tutorials, issue trackers. Every non-trivial fix’s commit message (or accompanying entry) MUST cite at least one external source OR the literal “NO external solution found — original work”.

§11.4.9 — Batch-source-fixes-before-rebuild

All source-side fixes that DO NOT require runtime validation to design MUST be landed BEFORE the next artifact rebuild. The anti-pattern of “fix A → rebuild → flash → cycle → fix B → rebuild → ...” serializes operator time onto rebuild latency.

§11.4.10 — Credentials-handling mandate

Forensic anchor — verbatim user mandate (2026-05-12):

“Credentials or any secret and sensitive data MUST NOT leak!”

Credentials MUST NEVER be tracked in git. `.env` / `.env.*` / `*.env` patterns + `scripts/testing/secrets/*` (with `.example` + `README.md` exception) git-ignored project-wide. Test scripts MUST NEVER print or log credentials. Per-service file separation limits blast radius. `chmod 600` on credential files, `chmod 700` on parent directory. Rotation on suspected leak.

§11.4.10.A — Pre-store credential leak audit (User mandate, 2026-05-17)

Forensic anchor — verbatim user mandate (2026-05-17):

“Us these for all future testing (full automation testing) and make sure they are not leaking anywhere or get git versioned!”

When an operator provides credentials, API tokens, signing keys, or any other secret material to be stored in the project's gitignored configuration, the storing agent MUST FIRST execute a repo-wide audit for prior leaks of THOSE specific values BEFORE storing: (1) `git ls-files | xargs grep -l <value>` for tree-leaks, (2) `git log -S<value> --all --source --remotes` for history-leaks, (3) surface findings to operator BEFORE storing (operator may rotate, accept-as-compromise, or abort), (4) on finding open a §6/§7 sixth-law-incidents record + redact tracked files in-place to `<redacted-per-§11.4.10>` + record OPERATOR ACTION REQUIRED for rotation per §11.4.10 sub-clause 7, (5) extend pre-push

hook credential-pattern grep to catch the escaped class in the same commit. See Constitution §11.4.10.A for the full mandate.

§11.4.11 — File-layout discipline

Project files organised by purpose, not historical accident. Source under canonical project roots. Tests under canonical test directories. Logs and forensic artifacts under operator-controlled directories — never scattered at repo root, never tracked unless they are reference assets.

§11.4.12 — Auto-generated docs sync

Every auto-generated document MUST be regenerated in the same commit as any edit to its source. All output formats (.md + .html + .pdf) MUST stay in sync at all times.

§11.4.13 — Out-of-band sink-side captured-evidence

Whenever a downstream consumer (HDMI sink, cloud monitor, downstream service) provides a network-accessible introspection API that reports what was actually received, the test suite MUST consume that report as captured-evidence. The on-source-side view alone is insufficient.

§11.4.14 — Test playback cleanup

Every test MUST leave the target in a quiescent state. Cleanup mandatory on every exit path (`trap '<cleanup>' EXIT`). The orchestrator MUST run a post-test sanity check and FAIL the just-completed test if it left orphan state.

§11.4.15 — Item-status tracking

Every active item in the project Issues file carries a `**Status:**` line within five lines of its heading. Six-state vocabulary: `Queued` , `In progress` , `Ready for testing` , `In testing` , `Reopened` , `Fixed` (→ `Fixed.md`) . Status updated as the item progresses. All three Issues / Issues_Summary / Fixed file types kept in sync (Markdown + HTML + PDF).

§11.4.16 — Item-type tracking

Every active item in the project Issues file carries a `**Type:**` line within eight non-blank lines of its heading. Three-value CLOSED vocabulary: `Bug` (product defect / regression / user-visible broken behaviour), `Feature` (new capability not previously offered to end users), `Task` (internal workstream — refactor, doc, infra, gate, audit; the lowest-stakes default when ambiguous). The `Issues_Summary` file carries the Type column for every active item. All three `Issues` / `Issues_Summary` / `Fixed` file types kept in sync (Markdown + HTML + PDF). Pre-build gates `CM-ITEM-TYPE-TRACKING` + `CM-COVENANT-114-16-PROPAGATION` enforce the mandate.

§11.4.17 — Universal-vs-project classification of new rules (User mandate, 2026-05-14)

Before adding ANY new rule, mandatory constraint, covenant clause, gate, or “MUST”-bearing statement to a project’s Constitution / `CLAUDE.md` / `AGENTS.md` (or to a submodule’s equivalents), the author MUST classify it as **universal** (reusable across any project → goes into this constitution submodule) or **project-specific** (references particular hardware / vendor / package / region → stays in the project / submodule layer). The commit message MUST carry a `Classification:` line stating the choice + one-sentence rationale. Universal rules that leak project-specific assumptions (hardware part numbers, vendor names, geographic regions, internal asset names) MUST be genericised first or downgraded to project-specific. When uncertain, default to project-specific (the narrower scope — lifting to universal later is cheap; the reverse is expensive). Pre-build gate `CM-UNIVERSAL-VS-PROJECT-CLASSIFICATION` audits new rule commits for the classification statement; paired mutation strips it and asserts gate FAILs.

§11.4.20 — Subagent-driven-by-default mandate (User mandate, 2026-05-14)

When the runtime supports subagent delegation (Claude Code Agent tool, Cursor task-runners, Aider sub-sessions, etc.), the primary agent MUST default to subagent delegation for any task that has multi-step scope (≥ 3 phases), parallelisable independent subtasks, long-running diagnostic loops, OR specialised domain workflows (code review, security audit, doc propagation). Foreground-only is reserved for single-file edits, mid-execution operator clarification, critical-state sequencing (commits / pushes / tags), or tasks so quick that subagent overhead

exceeds the work. Sub-discipline: **tight scope** (4-6 tasks, not “do everything”), **checkpoint commits** after each major task, **anti-stall protection** explicit in prompts, **anti-bluff verification** of subagent claims via repo state. Parallel subagents MUST partition non-overlapping files; `commit_all.sh --auto-cascade` bundles both via `git add -A .` Gate `CM-SUBAGENT-DELEGATION-AUDIT` (when implemented in consuming project) flags foreground multi-step work as a §11.4.20 violation.

§11.4.18 — Script documentation mandate (User mandate, 2026-05-14)

Every Bash / shell / POSIX-sh script anywhere in a project (`scripts/` , `bin/` , `tests/` , library directories, deployment hooks, CI helpers — depth-N recursive) MUST carry: (1) an in-source documentation block (Purpose / Usage / Inputs / Outputs / Side-effects / Dependencies / Cross-references) at the top of the file; (2) an external user guide under `docs/scripts/<script-name>.md` covering Overview / Prerequisites / Usage examples / Edge cases / Internal behaviour / Related scripts / Last verified date. When a script is modified, BOTH the in-source block AND the external user guide MUST be updated in the SAME commit. **No documentation ever can be out of sync with its codebase.** Pre-build gate `CM-SCRIPT-DOCS-SYNC` walks every `*.sh` / `*.bash` under script directories, verifies a companion `docs/scripts/<name>.md` exists, AND verifies doc was modified in the same commit (or `doc-mtime ≥ script-mtime` as a softer floor). Paired mutation strips the doc-sync invariant and asserts gate FAILs.

§11.4.19 — Fixed-document column-alignment mandate (User mandate, 2026-05-14)

Every project that maintains an open-work tracker AND a closed-archive tracker MUST keep the two structurally aligned along the same lifecycle axes (Status + Type). For the Fixed archive that means: (1) every `###` / `####` heading in `Fixed.md` (or equivalent) carries a `**Status:**` line and a `**Type:**` line within 8 non-blank lines of its heading; (2) a `Fixed_Summary.md` companion exists with the same column structure as `Issues_Summary.md` (`#` | `Level` | `Status` | `Type` | `One-line description`); (3) all three file formats (`.md` + `.html` + `.pdf`) for BOTH `Fixed.md` and `Fixed_Summary.md` stay in sync via the same single-shot wrapper that handles `Issues` + `Issues_Summary`; (4) closure migration is

atomic — when an Issues entry resolves it moves to Fixed.md in the same commit, disappears from Issues_Summary (open-only), and appears in Fixed_Summary (closed-only). Status values for closed items are drawn from {Fixed (→ Fixed.md) | Fixed – pending device verification | Fixed – RECLASSIFIED} . Type values follow §11.4.16: {Bug | Feature | Task} . Pre-build gate CM-FIXED-COLUMN-ALIGNMENT (5+ invariants) — Fixed_Summary.md exists, table header carries Status+Type columns, mtime (Fixed_Summary ≥ Fixed), generator script present, sync wrapper invokes it, HTML+PDF exports for both Fixed and Fixed_Summary present. Paired mutation strips Status column from Fixed_Summary table header → gate FAILs. Classification: universal (per §11.4.17). No escape hatch.

§11.4.21 — Operator-blocked status + self-resolution exhaustion (User mandate, 2026-05-14)

Operator-blocked is the §11.4.15 Status closed-set's 7th value: {Queued | In progress | Ready for testing | In testing | Reopened | Operator-blocked | Fixed (→ Fixed.md)} . It is a **last-resort classification**, earned only after the agent documents exhaustion of every applicable self-resolution path: (a) CLI / ADB / SSH / API access already available, (b) subagent delegation per §11.4.20, (c) existing repo tooling (scripts / helpers / libraries), (d) captured fallback (synthetic event, asset substitution, mock, §11.4.3 topology SKIP), (e) external research per §11.4.8. Every Operator-blocked item MUST carry an ****Operator-Block-Details:**** line within 8 non-blank lines of its heading stating: **WHAT** (concrete action), **WHY** (each exhausted alternative enumerated), **UNBLOCK CONDITION** (observable signal), **WHO** (handle / contact / doc pointer). Issues_Summary.md lists Operator-blocked as a sortable Status value. Items MUST be re-evaluated every Nth tag cycle (project- defined, recommended ≥3rd cycle) — operator dependencies change. Fake Operator-blocked (no exhaustion audit) is a §11.4 covenant violation at the planning layer, severity-equivalent to a PASS-bluff. Gates: CM-ITEM-OPERATOR-BLOCKED-DETAILS (every Operator-blocked heading has the details line), CM-OPERATOR-BLOCKED-SELF-RESOLUTION-AUDIT (NEW Operator-blocked commits contain “Attempted: a — ...; b — ...; c — ...” trail). Classification: universal (per §11.4.17). No escape hatch.

§11.4.22 — Document-sync commit discipline (User mandate, 2026-05-14)

Every project tracking work items through an Issues / Fixed lifecycle MUST provide a **lightweight commit path** distinct from the full-repo commit wrapper. The lightweight path stages, commits, and pushes ONLY the status-tracking doc set — Issues + Issues_Summary + Fixed + Fixed_Summary + CONTINUATION + their HTML + PDF exports + any auto-generated audit artifact — so doc-status never drifts behind working-tree reality when the full-repo wrapper is unavailable (in-flight rebase, large submodule churn, partial network). The wrapper MUST: (a) auto-invoke the project's export-regeneration pipeline first so Markdown + HTML + PDF stay in sync; (b) stage ONLY the explicit doc-set list — NEVER `git add -A`; (c) use a separate flock disjoint from the full-tree wrapper's lock; (d) push to every parent-repo remote; (e) exit `3` on nothing-to-commit (informational, not error). The wrapper MUST be invocable standalone OR as a delegation flag on the full-tree wrapper (e.g. `commit_all.sh --docs-only`) so operators have a single mental model. Inherits the project's §9 preflight discipline (refuses to run mid-meta-test). Gate `CM-COMMIT-DOCS-EXISTS` verifies wrapper + guide + flag + doc-set enumeration. Paired mutation strips the doc-set array → gate FAILs. Classification: universal (per §11.4.17). Composes with §11.4.12 (export-sync), §11.4.15 (status tracking), §11.4.18 (script documentation), §12.10 (CONTINUATION maintenance). No escape hatch — doc-status drift is a §11.4 PASS-bluff at the documentation layer.

§11.4.24 — Build-resource stats tracking mandate (User mandate, 2026-05-14)

Every project under this Constitution with a build exceeding 1 minute wall-clock MUST run a host-side resource sampler for every build that captures memory used, CPU%, load average, disk read/write at a fixed interval (recommended 5 s) and computes per-metric **min / max / mean / p95** at stop. Per-build summaries appended to a TSV registry; the registry is the single source of truth — the human-readable Markdown report (and its HTML + PDF exports per §11.4.12) is derived. Top of the report MUST surface **ever-values** (min / max / mean across all tracked builds). Per-build entries sorted most-recent-first; each row carries SUCCESS / FAIL / UNKNOWN + reason for FAIL. Sampler MUST itself stay under 50 MB RSS and 5% CPU (Heisenberg-class observer constraint). Stop hook MUST be called from both success AND failure paths of the build wrapper. The Stats.{md,html,pdf}

triple is committed via the project's §11.4.22 lightweight doc-sync wrapper. Gate CM-BUILD-RESOURCE-STATS-TRACKER + paired mutation hiding the monitor aside → gate FAILs. Classification: mixed (per §11.4.17) — the discipline universal, the implementation paths project-specific. Composes with §11.4.12 / §11.4.18 / §11.4.22 / §12.6 / §12.7 / §12.9 (the host-safety forensic anchors are the empirical motivation for this telemetry). No escape hatch — build-resource debugging without time-series data is the bluff this anchor forbids.

§11.4.25 — Full-Automation-Coverage Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

“Make sure that every feature, every functionality, every flow, every use case, every edge case, every service or application, on every platform we support is covered with full automation tests which will confirm anti-bluff policy and provide the proof of fully working capabilities, working implementation as expected, no issues, no bugs, fully documented, tests covered! Nothing less than this does not give us a chance to deliver stable product!”

For every consuming project, no feature / functionality / flow / use case / edge case / service / application on any supported platform may be considered **deliverable** until it is covered by automation tests proving six invariants: (1) anti-bluff posture (captured runtime evidence per §7.1 + §11.4); (2) proof of working capability end-to-end on the target topology (per §11.4.3, not in a mock); (3) working implementation matching the documented promise; (4) no open issues / bugs surfaced by the suite (cross-checked against §11.4.15 / §11.4.16 trackers); (5) full documentation (user manual entry + §11.4.18 for scripts) kept in sync per §11.4.12; (6) four-layer test floor per §1 (pre-build + post-build + runtime + paired mutation). Consuming projects **MUST** publish a coverage ledger (feature × platform × invariant-1..6 × status), regenerated as part of release-gate sweeps, with gaps tracked per §11.4.15. Classification: universal (§11.4.17). No escape hatch. A project that ships a feature without all six invariants is **not delivering a stable product** — severity-equivalent to a §11.4 PASS-bluff at the release-gate layer. See Constitution §11.4.25 for the full mandate (cross-cutting reach, composition, audit requirements).

§11.4.26 — Constitution-Submodule Update Workflow Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

“Every time we add something into our root (constitution Submodule) Constitution, CLAUDE.MD and AGENTS.MD we MUST FIRST fetch and pull all new changes / work from constitution Submodule first! All changes we apply MUST BE committed and pushed to all constitution Submodule upstreams! In case of conflict, IT MUST BE carefully resolved! Nothing can be broken, made faulty, corrupted or unusable! After merging full validation and verification MUST BE done!”

Before ANY modification to `constitution/Constitution.md` , `constitution/CLAUDE.md` , or `constitution/AGENTS.md` , the agent or operator MUST execute the following pipeline in order:

1. **Fetch + pull first** — inside the constitution submodule worktree run `git fetch` against every remote, then `git pull --ff-only` (or `--rebase` if non-FF-mergeable; never `--strategy=ours` / `--allow-unrelated-histories` without explicit authorization). The submodule MUST be at upstream tip BEFORE any local edit.
2. **Apply the change** — classify per §11.4.17 (only universal additions belong here; project-specific clauses stay in the consuming project’s governance). Cite the verbatim user mandate if one originated the change.
3. **Validate before commit** — run `meta_test_inheritance.sh` (or equivalent); verify no merge-conflict markers (`<<<<<<<` , `=====` , `>>>>>>>`); verify Constitution + CLAUDE + AGENTS cross-reference the new clause consistently.
4. **Commit + push to ALL upstreams** — stage only the governance files (NEVER `git add -A` inside the submodule); commit message cites the user mandate + §11.4.17 classification; push to every configured remote. A commit landing on one upstream but not others is a §2.1 violation AND a §11.4.26 violation.
5. **Conflict resolution** — if `pull --ff-only` reports non-fast-forward, merge carefully (preserve union of governance content, no clause silently dropped, re-classify, re-validate). Force-push to “make conflicts go away” is FORBIDDEN (§9.2). Nothing about the constitution may be broken, made faulty, corrupted, or rendered unusable.

6. **Post-merge validation + verification** — after the push lands, `git submodule update --remote --init` and re-run the consumer project's cascade verifier (per CONST-047) to confirm the new clause reaches every owned submodule. Any cascade gap closed in the same change-window.
7. **Bump consuming project pointer** — `.gitmodules`-tracked submodule pointer MUST be advanced to the new constitution HEAD in the SAME commit as any cascade work. Out-of-sync pointers are §11.4.26 violations.

Classification: universal (§11.4.17). No escape hatch. A constitution-submodule change violating §11.4.26 is a release blocker for every consuming project, severity-equivalent to a force-push without §9.2 authorization. See Constitution §11.4.26 for the full mandate (operational scope, cross-cutting reach).

§11.4.31 — Submodule-Dependency-Manifest Mandate (User mandate, 2026-05-15)

Every owned-by-us submodule MUST ship a machine-readable dependency manifest at canonical path `helix-deps.yaml` (or `.json/.toml`) listing its own-org Git SSH dependencies. Schema (per Constitution §11.4.31): `schema_version` , `deps: [{name, ssh_url, ref, why, layout: flat|grouped}]` , `transitive_handling.recursive: true` , `transitive_handling.conflict_resolution: operator-required` , `language_specific_subtree: bool` .

Tooling: `incorporate-submodule <ssh-url>` adds the submodule at its declared canonical path (CONST-051(C) flat/grouped), reads its `helix-deps.yaml`, recurses for each declared dep, aborts on conflicting refs, emits `<root>/helix-manifest.yaml` audit record.

Anti-bluff guarantee: every manifest paired with a Challenge that bootstraps a throwaway consuming project, runs `incorporate-submodule` , asserts produced layout matches manifest, runs the submodule's own tests against the bootstrapped layout, captures wire evidence per §11.4.2. A manifest without this proof is a §11.4.31 violation.

§11.4.31 is the operational complement of §11.4.28 / CONST-051(C): nested own-org submodule chains are FORBIDDEN, manifests are the bridge that lets consumers reconstruct the dependency graph at the parent root.

Classification: universal (§11.4.17). Composes with §1, §3, §11.4.12, §11.4.17, §11.4.18, §11.4.20, §11.4.25, §11.4.26, §11.4.27, §11.4.28, §11.4.29, §11.4.30, CONST-047. See Constitution §11.4.31 for the full mandate.

§11.4.32 — Post-Constitution-Pull Validation Mandate (User mandate, 2026-05-15)

Whenever a project's constitution submodule is fetched + pulled with any content change, the project **MUST** run a full-project + recursive-submodule validation sweep **BEFORE** the new constitution HEAD is treated as canonical for any other work.

Sweep contract (canonical script: `scripts/verify-all-constitution-rules.sh`): re-runs the governance- cascade verifier; for every rule with a programmatic gate (CONST-053 `.gitignore` audit, CONST-051(C) nested-own-org-chain audit, CONST-052 case audit, CONST-050(A) mock-from-production audit, CONST-035 anti-bluff smoke), runs the gate against post-pull tree. Failures produce directed FAIL entries → tracker per §11.4.15 with Status: `Reopened`, Type: `Bug`. Closure requires positive-evidence per §11.4 anti-bluff covenant.

Pull-time invocation: `git submodule update --remote constitution` triggers the sweep automatically (post-update hook or commit wrapper); operator-explicit manual invocation also available.

Anti-bluff: sweep's own meta-test (paired mutation §1.1) plants a known violation of each enforced gate and asserts sweep reports FAIL for the planted gate. A sweep that exits PASS without running every implementable gate is a §11.4.32 violation.

§11.4.32 is the **enforcement engine** for every other §11.4.x and CONST-NNN rule — without it, new rules cascade as anchors but never get enforced in the codebase.

Classification: universal (§11.4.17). Composes with every rule that has a programmatic gate. See Constitution §11.4.32 for the full mandate.

§11.4.30 — .gitignore + No-Versioned-Build-Artifacts Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

“every project module, every Submodule, every service and application **MUST** HAVE proper `.gitignore` file! We **MUST NOT** git version build artifacts, cache

files, tmp files, main .env file(s) or any files containing sensitive data, API keys or token! Any build derivative which we can recreate by executing proper mechanism for generating MUST NOT be versioned! We MUST pay attention what is going to be committed every time we are preparing to execute commit! If any violation is detected it MUST be fixed before commit is executed!"

Every project module / owned-by-us submodule / service / application MUST ship a proper .gitignore covering the forbidden-from-version-control classes:

1. **Build artefacts:** /bin/ , /build/ , /dist/ , /out/ , target/ , *.exe , *.dll , *.so , *.dylib , *.a , *.o , *.class , *.pyc , generator-produced files (when the generator is committed).
2. **Cache files:** __pycache__/ , .pytest_cache/ , .mypy_cache/ , .ruff_cache/ , node_modules/ , .next/ , .nuxt/ , .cache/ , .gradle/ , .terraform/ , language-server caches.
3. **Temp files:** *.tmp , *.swp , *~ , .DS_Store , Thumbs.db , *.orig , *.rej .
4. **Sensitive-data files:** .env , .env.* (allow .env.example placeholder), *.pem , *.key , *.crt , id_rsa* , id_ed25519* , .netrc , secrets/ , api_keys.sh .
5. **Generated reports/logs:** *.log , coverage.out , htmlcov/ , runtime captures unless reference assets.
6. **OS/IDE personal state:** .idea/ , .vscode/ (except shared settings), .history/ .

Anti-bluff invariant: .gitignore line alone is not sufficient — no file matching the forbidden patterns may be currently tracked. A tracked *.log despite the ignore-line is a violation of equal severity to no ignore-line at all.

Pre-commit attention: every commit author (human OR agent) MUST inspect `git diff --staged + git status` BEFORE the commit. Forbidden-class hits abort the commit until fixed (un-stage, add to .gitignore , scrub if already-tracked).

Gate `CM-GITIGNORE-PRECOMMIT-AUDIT` + paired mutation.

Secret-leak intersection: §11.4.30 composes tightly with §11.4.10 + §12.1 (CONST-042) — a .env leak is BOTH a §11.4.30 and a §11.4.10 violation, requiring rotation + post-mortem.

Recreatable-content test: if a documented mechanism regenerates the file from sources, it's a build derivative and MUST be ignored. Generators MUST be committed so consumers regenerate on demand.

Classification: universal (§11.4.17). No escape hatch beyond enumerated exceptions. Severity-equivalent to §11.4 PASS-bluff at the repository-hygiene layer. Composes with §1, §2, §9.1, §11.4.10, §11.4.12, §11.4.17, §11.4.18, §11.4.20, §11.4.25, §11.4.26, §11.4.27, §11.4.28, §11.4.29, CONST-047. See Constitution §11.4.30 for the full mandate.

§11.4.29 — Lowercase-Snake_Case-Naming Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

“naming convention for Submodules and directories (applied deep into hierarchy recursively) - all directories and Submodules MSUT HAVE lowercase names with space separator between the words of ' _ ' character (snake-case)! All existing Submodules and directories which are not following this rule MUST BE renamed! ... NOTE: Rules lowercase / snake-case do apply to all project files as well and references to it and from them!”

Every directory, submodule, and file MUST use lowercase snake_case names (ASCII letters / digits / underscores, words separated by `_`). Existing non-compliant names MUST be renamed as part of the migration window opened by this clause. Every reference (configs, docs, links, source-code imports, governance files) MUST be updated atomically with the rename — reference drift after a rename is a §11.4.29 violation of equal severity to the rename itself.

Exceptions (common-sense, must-not-break-technology). Language- mandated case (Java/Kotlin package paths, Android resource directories, Apple framework dirs, C#/Swift project layouts) is preserved inside the language-root. Submodule root directory follows our convention; language-specific subtree follows its own. Vendor/upstream third-party submodules keep their upstream names. Build artefacts (`node_modules/` , `__pycache__` / , `.git/` , `target/` , `build/` , `bin/`) keep tool-mandated names. “Does renaming break the technology?” trumps the rule.

Upstreams/ → **upstreams/ transition.** Constitution submodule's

`install_upstreams.sh` (exported via `.bashrc` / `.zshrc`) MUST support BOTH `Upstreams/` and `upstreams/` directory layouts during migration. Lowercase wins when both present. Uppercase fallback retires only by deliberate amendment.

Project-Toolkit Upstreamable synchronisation. Upstreamable / Project-Toolkit machinery MUST be fetched+pullled before any rename batch + MUST itself comply with this rule. Lacking BOTH- directory support is a release blocker.

Test coverage of renames. Each rename batch ships with: (i) regression test verifying every reference now resolves; (ii) full CONST-050(B) test-type matrix run on the post-rename tree; (iii) anti-bluff wire-evidence captured. All three or it's a §11.4.29 violation.

Phased execution. Comprehensive brainstorming → phase-divided plan → fine-grained tasks/subtasks → every change covered by every applicable test type. Phases run in parallel with mainstream work (§11.4.20 subagent delegation).

Classification: universal (§11.4.17). No escape hatch beyond the common-sense exceptions enumerated. Severity-equivalent to §11.4 PASS-bluff at the reference-integrity layer. Composes with §1, §1.1, §11.4.12, §11.4.17, §11.4.18, §11.4.20, §11.4.25, §11.4.26, §11.4.27, §11.4.28, CONST-047. See Constitution §11.4.29 for the full mandate.

§11.4.28 — Submodules-As-Equal-Codebase + Decoupling + Dependency-Layout Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

“All existing Submodules in the project that we are controlling and belong to some our organizations (vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory — we can ALWAYS check dynamically using GitHub and GitLab CLIs) are equal parts of the project's codebase! We MUST work on that code as much as we do with main project's codebase! All on equal basis! Equally important! ... We MUST NEVER modify Submodules to bring into them any project specific context since they all MUST BE ALWAYS fully decoupled, project not-aware, fully reusable and modular (by any other project(s)), completely testable! All Submodule dependencies that are used by

Submodule MUST BE accessed from the root of the project! We MUST NOT have nested Submodule dependencies but accessing each from proper location from the root of the project — directly from project's root project_name/ submodule_name or some more proper structure project_name/submodules/ submodule_name!"

Three cooperating invariants:

(A) Equal-codebase. Every owned-by-us submodule (orgs: vasic-digital , HelixDevelopment , red-elf , ATMOSphere1234321 , Bear-Suite , Boat0S123456 , Helix-Flow , Helix-Track , Server-Factory — dynamically discoverable via gh / glab CLIs) is an **equal part** of the consuming project's codebase. Same engineering attention as main: analysis, extension, test creation, gap-filling, bug-fix, documentation (user manuals, guides, diagrams, SQL, websites, all materials). A round that improves main while leaving an owned-submodule deficiency unaddressed is a §11.4.28 violation, severity-equivalent to a §11.4 PASS-bluff at the project-scope layer. Coverage ledgers (§11.4.25) list every owned submodule as in-scope.

(B) Decoupling / reusability. Owned submodules MUST stay fully decoupled, project-not-aware, reusable, modular, completely testable. NEVER inject project-specific context (hardcoded paths, hostnames, asset names) INTO a submodule. When a submodule needs parent-project info, use configuration injection (env var, config file, constructor parameter) — never a hardcoded reach.

(C) Dependency-layout. Every dependency consumed by an owned submodule MUST be accessible from the parent project's root at:

```
<project_root>/<submodule_name>/  
<project_root>/submodules/<submodule_name>/
```

Nested own-org submodule chains are FORBIDDEN. A submodule MUST NOT have its own .gitmodules entries pulling in further owned- by-us repos. Add the dependency at the parent's root path; the submodule reaches it via documented import / SDK / runtime resolver. Third-party submodules exempt.

Gates: CM-OWNED-SUBMODULE-EQUAL-ENGINEERING (release-gate sweep audits parity), CM-OWNED-SUBMODULE-DECOUPLING (pre-commit greps for parent-project context inside the submodule diff), CM-OWNED-SUBMODULE-LAYOUT (pre-merge

verifies canonical location + no nested own-org submodules + dependency lookup from root). Paired mutations (§1.1) for all three. Classification: universal (§11.4.17). No escape hatch. Composes with §1, §3, §11.4.17, §11.4.20, §11.4.25, §11.4.26, §11.4.27, CONST-047. See Constitution §11.4.28 for the full mandate.

§11.4.27 — No-Fakes-Beyond-Unit-Tests + 100%-Test-Type-Coverage Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

“Mocks, stubs, placeholders, TODOs or FIXMEs are allowed to exist ONLY in Unit tests! All other test types MUST interact with real fully implemented System! No fakes, empty implementations or bluffing is allowed of any kind! All codebase of the project MUST BE 100% covered with every supported test type: unit tests, integration tests, e2e tests, full automation tests, security tests, ddos tests, scaling tests, chaos tests, stress tests, performance tests, benchmarking tests, ui tests, ux tests, Challenges (fully incorporating our Challenges Submodule). EVERYTHING MUST BE tested using HelixQA (fully incorporating HelixQA Submodule). HelixQA MUST BE used with all possible written tests suites (test banks) for every applications, service, platform, etc and execution of the full HelixQA QA autonomous sessions! All required dependency Submodules MUST BE added into the project as well (fully recursive!!!).”

Two cooperating invariants:

(A) No-fakes-beyond-unit-tests. Mocks, stubs, fakes, placeholders, `TODO`, `FIXME`, “for now”, “in production this would”, or empty-implementation patterns are PERMITTED only in unit-test sources. Every other test type — integration, E2E, full automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges, HelixQA — MUST exercise the real, fully implemented system against real infrastructure. Production code MUST NOT import mock paths. Gate `CM-NO-FAKES-BEYOND-UNIT-TESTS` + paired mutation.

(B) 100% test-type coverage. Codebase MUST be covered by every supported test type the domain warrants: unit, integration, E2E, full-automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges (vasic-digital/ Challenges submodule fully incorporated), HelixQA

(HelixDevelopment/ HelixQA submodule fully incorporated). HelixQA autonomous sessions drive end-to-end execution of every registered test bank with captured wire evidence per check.

Required dependency submodules (recursive per CONST-047): - Challenges — `git@github.com:vasic-digital/Challenges.git` - HelixQA — `git@github.com:HelixDevelopment/HelixQA.git` - Any other functionality submodules under `vasic-digital/*` / `HelixDevelopment/*` orgs the project depends on.

Pointers bumped to upstream HEAD in same commit as cascade work (§11.4.26 step 7); pointer drift = §11.4.27 violation.

Classification: universal (§11.4.17). Severity-equivalent to a §11.4 PASS-bluff at the release-gate layer. No escape hatch. Composes with §1, §7.1, §11.4.1–§11.4.26 (esp. §11.4.25 — this is its strict expansion into per-type-of-test territory). See Constitution §11.4.27 for full mandate.

§11.4.33 — Type-aware closure-status vocabulary (User mandate, 2026-05-15)

Every project that tracks work items by Type per §11.4.16 MUST close them with the Type-appropriate closure-status word, drawn from this 3-element closed map:

Item **Type:**	Closure **Status:** value
Bug	Fixed (→ Fixed.md)
Feature	Implemented (→ Fixed.md)
Task	Completed (→ Fixed.md)

The (→ Fixed.md) suffix is preserved across all three so the existing migration-discipline tooling (atomic Issues.md → Fixed.md move per §11.4.19) keeps working without per-Type branching. Generators (`generate_issues_summary.sh` , `generate_fixed_summary.sh` , status-counter helpers, the §11.4.23 colorizer) MUST treat the three terminal values as semantically equivalent (all map to “closed, positive evidence captured”) while preserving the literal in the emitted document.

Closing a `Feature` with `Fixed (→ Fixed.md)` or a `Task` with `Implemented (→ Fixed.md)` is a §11.4.33 violation. Pre-build gate (recommended) `CM-CLOSURE-`

VOCAB-TYPE-AWARE walks every Fixed.md heading + every Issues.md heading whose `**Status:**` is one of the three terminal values and asserts the Status-Type match. Composes with §11.4.15 (status tracking), §11.4.16 (type tracking), §11.4.19 (Fixed-document column alignment), §11.4.23 (colourisation). Classification: universal (per §11.4.17). No escape hatch.

§11.4.34 — Reopened-source attribution mandate (User mandate, 2026-05-15)

Every Issues.md (or equivalent project tracker) heading whose `**Status:**` is Reopened MUST carry, within 8 non-blank lines of the heading, a `**Reopened-Details:**` line capturing four sub-facts:

- **By:** AI or User (source-of-truth observer who flipped the status). AI covers in-loop reopens (test failure, gate regression, captured-evidence retrospect). User covers operator-side observations (manual testing, end-user report, design reconsideration).
- **On:** ISO date (YYYY-MM-DD).
- **Reason:** one-line cause classification — chosen from the closed vocabulary { test-failed | manual-testing-detected | captured-evidence-contradicts | end-user-report | cycle-re-discovered | design-reconsidered }. Other values are permitted with explicit Reason: <free text> annotation but the closed list MUST be tried first.
- **Evidence:** path to or short description of the captured artefact justifying the reopen — log file, recording, gate failure ID, operator quote, etc. Reopens without evidence are §11.4.6 / §11.4.7 violations: the reopen IS a demotion-from-Fixed classification change, and demotion requires positive evidence captured under the conditions that re-exposed the defect.

The Issues_Summary.md (or equivalent) Status column MUST distinguish the four Reopened sub-states by source so a sweep query for “reopens by AI in the last 30 days” is mechanically possible. Suggested column rendering: Reopened (AI: test-failed) vs Reopened (User: manual-testing) .

A Reopened entry without `**Reopened-Details:**` is a §11.4.34 violation. Pre-build gate (recommended) CM-ITEM-REOPENED-DETAILS mirrors CM-ITEM-OPERATOR-BLOCKED-DETAILS (§11.4.21 walk pattern). Composes with §11.4.6 (no-guessing — Reason from closed vocabulary), §11.4.7 (demotion-evidence — reopen IS a demotion from Fixed), §11.4.15 (item-status tracking), §11.4.21

(Operator-blocked discipline — same audit-line pattern). Classification: universal (per §11.4.17). No escape hatch.

§11.4.35 — Canonical-root inheritance clarity (User mandate, 2026-05-15)

The constitution submodule's three files (`constitution/Constitution.md` , `constitution/CLAUDE.md` , `constitution/AGENTS.md`) ARE the canonical root — also called the parent files. They contain only universal rules per §11.4.17.

The consuming project's repository-root files (`<project-root>/CLAUDE.md` , `<project-root>/AGENTS.md` , optionally `<project-root>/Constitution.md` or equivalent) are consumer extensions. They open with the inheritance pointer (either the Claude-Code native `@constitution/CLAUDE.md` import or the portable `## INHERITED FROM constitution/CLAUDE.md` heading defined in this file's "How inheritance works" section). They contain only project- specific rules per §11.4.17 — rules that reference particular hardware, vendor names, regulatory regions, internal asset names, or project-private conventions.

When in doubt about which file to edit: universal rule → edit constitution submodule's file; project-specific rule → edit consumer's file. Default consumer-side when uncertain (per §11.4.17, narrower scope is cheap to widen).

Terminology: when prose references "the parent CLAUDE.md" or "the root Constitution," the referent is the constitution-submodule file at `constitution/<filename>` , never the consumer's file. When it references "the project CLAUDE.md" or "this project's AGENTS.md," the referent is the consumer-side file at `<project-root>/<filename>` . AI agents resolve ambiguous references via this rule.

No silent demotion or silent promotion. Moving a rule between layers MUST be a visible commit — `git mv` of a section if it's a clean clone, or an explicit "Lifted from to constitution per §11.4.35" / "Demoted from constitution to per §11.4.35" line in the commit message.

Pre-build gate (recommended) `CM-CANONICAL-ROOT-CLARITY` verifies (a) consumer's `CLAUDE.md` opens with the inheritance pointer (either `@import` or `## INHERITED FROM constitution/CLAUDE.md` heading), (b) the constitution submodule's three files are present at the expected path, (c) no `## INHERITED FROM` block in the constitution submodule's own files (those ARE the source-of-

truth, not consumers). Composes with §11.4.17 (universal-vs-project classification — §11.4.35 defines the file-layer split that §11.4.17 classifies INTO). Reading order: this anchor first, then §11.4.17. Classification: universal (per §11.4.17). No escape hatch.

§11.4.36 — Mandatory `install_upstreams` on clone/add Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

“Every Submodule or Git repository we add or clone MUST BE upstreams installed using Upstreamable utility which MUST BE available through exported paths of the host system (in `.bashrc` or `.zshrc`) using `install_upstreams` command executed from the root of the cloned (added) repository - only if in it is Upstreams or upstreams directory present with bash script files (recipes) for all repository’s upstreams!”

Every clone / add of a Git repository under any consuming project MUST be followed by `install_upstreams` invocation from that repository’s root IF its tree contains an `upstreams/` directory (or legacy `Upstreams/` per §11.4.29 transition) populated with `*.sh` recipe files declaring upstream Git SSH URLs.

`install_upstreams` is a host-system utility on operator’s `PATH` (exported via `.bashrc` / `.zshrc`), implemented in this constitution submodule (`install_upstreams.sh`). The utility reads recipe files, configures every declared upstream as a named git remote, and fans out `origin` push URLs across all declared upstreams.

Skipping the invocation when `upstreams/` IS present silently breaks §2.1 (Multi-upstream push is the norm) — the next push lands on only one upstream. Gate `CM-INSTALL-UPSTREAMS-ON-CLONE` + paired mutation (§1.1).

Automation: `incorporate-submodule` (§11.4.31) and `scripts/init-submodules.sh` patterns auto-invoke `install_upstreams` when applicable. Operator-explicit manual invocation remains supported.

Pre-commit attention: before the first commit in the newly-cloned working tree, verify `git remote -v | grep -c push` reports the expected upstream count.

Classification: universal (§11.4.17). Composes with §2, §2.1, §3, §9.2, §11.4.17, §11.4.20, §11.4.28, §11.4.29, §11.4.30, §11.4.31. See Constitution §11.4.36 for the full mandate.

§11.4.37 — Fetch-before-edit mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

“Make sure that `feedback_fetch_before_edit` memory rule is part of our constitution Submodule - the root Consitution, AGENTS.MD and CLAUDE.MD. Validate and verify that Proejct-Toolkit and all Submodules do inherit all of them!”

The FIRST git-touching action of any session, on any consuming project, MUST be:

```
git fetch --all --prune
git log --oneline HEAD..@{u}           # parent
git submodule foreach --recursive 'git fetch --all --prune --quiet'
```

If `HEAD..@{u}` is non-empty, integrate (ff-merge / rebase / surface to operator per §11.4.4) BEFORE any local edit, scanner run, or test cycle. Multi-agent / multi-upstream codebases (Claude Code + Cursor + Aider + operator sessions in parallel) routinely lap each other; a 30-second fetch prevents the agent from redoing work a parallel session already finished, from filing a false-confidence “completion” of already-done work, and from doubling the multi-upstream conflict surface (§2.1) with sibling commits of the same change.

The check is non-negotiable even when the operator says “do X immediately” — skipping it on the basis of “nothing could have changed in the last N minutes” is a §11.4.6 (no-guessing) violation: remote state is not knowable without a fetch. The fetch+log output (even if empty) is the captured evidence.

Scope: consuming project root + every owned submodule recursively (§11.4.28) + the constitution submodule itself (§11.4.26 step 1 made this explicit for constitution-side edits; §11.4.37 generalises to ALL edits) + any dependency cloned via `incorporate-submodule` (§11.4.31) or `git submodule add` (§11.4.36).

Pre-build gate `CM-FETCH-BEFORE-EDIT-AUDIT` (when implemented in the consuming project) audits the most-recent commit range against the upstream HEAD at the commit's parent — non-aligned parent = FAIL. Paired mutation (§1.1): synthetic commit whose parent was N commits behind the then-current upstream HEAD → gate FAILs.

Classification: universal (§11.4.17). No escape hatch. Composes with §2.1, §11.4.4, §11.4.6, §11.4.20, §11.4.26, §11.4.32. See Constitution §11.4.37 for the full mandate.

§11.4.38 — Installable-Asset Evidence Mandate (User mandate, 2026-05-17)

For any user-distributable build artifact (package, bundle, installer, or container image produced by the build pipeline and distributed to end users), tests and challenges **MUST** open the artifact and verify each user-visible asset is **present** and **non-degenerate**.

A PASS without opening the artifact and verifying the asset chain end-to-end is a §11.4 PASS-bluff. The specific failure mode: source file exists → build packages it → post-build checks pass at the source layer → artifact produced with the asset stripped or misconfigured, and no gate ever opens the artifact to verify.

Each consuming project ships one challenge script per artifact type that opens the produced artifact and verifies every declared user-visible asset. The challenge **MUST** run as part of the project's standard QA gate.

Classification: universal (§11.4.17). No escape hatch. See Constitution §11.4.38 for the full mandate.

§11.4.40 — Full-suite retest before release tag mandate (User mandate, 2026-05-17)

A release tag **MUST NOT** be created until a **COMPLETE** retest with **ALL** existing tests has been executed on a clean baseline **AFTER** every workable item in the batch is done, fixed, polished, and individually verified. Spot-check retests that run only the tests touched by the batch are **FORBIDDEN** — they miss interaction defects between batch fixes and previously-stable code.

The complete retest comprises: (1) pre-build full sweep, (2) post-build full sweep, (3) on-device 4-phase cycle on EVERY owned device, (4) meta-test full mutation sweep, (5) Challenge bank full sweep, (6) Issues.md/Fixed.md state audit, (7) CONTINUATION.md sync check.

Time is essential — typically 12–48h elapsed effort. NOT optional, NOT abbreviated. Skipping is the exact “tests pass but feature broken” failure mode §11.4 prohibits.

Composes with §11.4.4 (per-fix retest still required at fix granularity) + §11.4.7 (full-suite retest is authoritative baseline for closures) + §11.4.39 (per-feature on-device validation runs as step 3 of the full-suite retest).

Classification: universal (§11.4.17). No escape hatch. See Constitution §11.4.40 for the full mandate.

§11.4.41 — Pre-Force-Push Merge-First Mandate (User mandate, 2026-05-17)

Forensic anchor — verbatim user mandate (2026-05-17):

“make sure we bring everything from branches to our side before forc push is done! Afer everything is safely and fully merged and all potential conflicts (if any) resolved, then do force push! make sure nothing isnlost, broken or corrupted on bith sides!”

Any force-push (`--force` , `--force-with-lease` , `+<ref>` , equivalent history-rewrite) authorised under CONST-043 MUST be preceded by a 4-step merge-first pipeline:

1. **Fetch every remote** — `git fetch --all --prune --tags` against origin + every upstream; capture output.
2. **Integrate every divergent commit locally** — rebase / merge / operator-confirmed cherry-pick per the appropriate strategy for every non-empty `HEAD..<remote>/<branch>` range.
3. **Audit the integrated tree** — no conflict markers anywhere (`grep -rn` `'^<<<<<< \|^===== $\|^>>>>>> '` returns empty in governance + source + test files); no file silently dropped; previously-passing tests still pass; captured-evidence artefacts still validate.

4. **Force-push** — only after steps 1-3 produce clean integration evidence: `git push --force-with-lease` (NEVER `--force` alone unless authorised per §9.2 sub-clause 6).

Two-gate composition with CONST-043. §11.4.41 does NOT relax CONST-043's operator-approval requirement — it adds a SECOND mechanical gate. Both required: CONST-043 alone authorises a push that loses remote work; §11.4.41 alone risks pushing without operator awareness.

Three failure modes prevented: (a) remote-side content loss when parallel sessions land work between fetches; (b) stale-state acts when `--force-with-lease` reads stale local refs; (c) conflict-driven corruption when markers get committed verbatim (observed 2026-05-17 in helix_qa + containers governance files).

Verification artefact — `docs/change logs/<tag>.md` “Force-push merge-first audit” section captures fetch output, per-remote divergence log, integration strategy, conflict- marker scan, test delta, push output with lease SHA, CONST-043 authorisation quote. Gate `CM-FORCE-PUSH-MERGE-FIRST` + paired mutation.

Classification: universal (§11.4.17). No escape hatch. Composes with §9.2, §11.4.4, §11.4.6, §11.4.26, §11.4.32, §11.4.37, §11.4.40, CONST-043, CONST-047. See Constitution §11.4.41 for the full mandate. ### §11.4.42 — Iteration-discipline mandate (User mandate, 2026-05-18)

Project work proceeds in priority-ordered iteration cycles. Each cycle has five mandatory steps: (1) select TOP + MIDDLE critical items only (defer LOW until critical batch closed), (2) batch implementation with §11.4.4 four-layer coverage + §11.4.9 batch-source-fixes-before-rebuild, (3) smoke gate (<30 min — batch-touched tests + critical-path regression probe + anti-bluff baseline), (4) ONLY if smoke GREEN and no new operator/user report arrived, run §11.4.40 full retest (12–48 h), (5) release-ready OR loop back to step 1 with new evidence added to queue.

Composes with §11.4.4 (per-fix retest inside step 2), §11.4.7 (closures require same-conditions evidence; step 4 is authoritative baseline), §11.4.9 (source-side batching inside step 2), §11.4.34 (Reopened items attribute source), §11.4.40 (the multi-hour retest IS step 4 — §11.4.42 is the meta-loop conductor).

Anti-bluff coupling: every smoke and full-system PASS MUST carry positive captured evidence per §11.4.2 + §11.4.5. Tests AND HelixQA Challenges bound equally.

No escape hatch — no `--skip-priority-batch` , no `--skip-smoke` , no `--full-suite-only` . Subagents default to the §11.4.42 path.

Classification: universal (§11.4.17). See Constitution §11.4.42 for the full mandate.

§11.4.43 — TDD-Fix-Discipline mandate (User mandate, 2026-05-18)

Every fix MUST follow the 5-step TDD-fix workflow: **RED** (failing test FIRST, real product defect per §11.4.1, captured evidence per §11.4.2) → **LIVE-ADB-PROBE** (try fix on running device via `adb shell` for mutable surfaces — `setprop persist.*` , `settings put` , `pm clear` , `am ...` , boot-script push; INFEASIBLE for kernel / framework / HAL / vendor / init.rc / sepolicy / ro.* / Android.bp — must rebuild; cite `LIVE_PROBE_INFEASIBLE: <reason>` in commit message) → **GREEN** (source patch achieves same effect, batched per §11.4.9, four-layer coverage per §11.4.4) → **VERIFY** (re-run RED test, must PASS under SAME conditions per §11.4.7, captured positive evidence per §11.4.5, 10-iteration reliability per §11.4.42) → **DOCUMENT** (Issues.md → Fixed.md with type-aware closure vocabulary per §11.4.33, CLAUDE.md Applied Fixes row, changelog, guides, HelixQA bank, CONTINUATION.md per §12.10 — all in the SAME commit).

No escape hatch — no `--skip-red-test` , `--no-live-probe` , `--skip-verify` flag. “Test added after the fix” is a §11.4 PASS-bluff: it demonstrates the test agrees with the fix, not that the test catches the bug.

Classification: universal (§11.4.17). Composes with §11.4.1 / §11.4.2 / §11.4.4 / §11.4.5 / §11.4.7 / §11.4.9 / §11.4.40 / §11.4.42. See Constitution §11.4.43 for the full mandate.

§11.4.44 — Document revision header mandate (User mandate, 2026-05-18)

Every tracked document in scope (Issues.md, Issues_Summary.md, Fixed.md, Fixed_Summary.md, CONTINUATION.md, docs/guides/, **docs/research/**, docs/scripts/, **docs/changelogs/**, docs/superpowers/plans/, **docs/hardware/**, all other docs/* tracked Markdown) MUST carry a header block directly below the H1 title

containing two MANDATORY fields: `**Revision:** N` (monotonic positive integer, never reset, never skipped) and `**Last modified:** YYYY-MM-DDTHH:MM:SSZ` (ISO 8601 UTC). Optional fields (Description, Authority, Maintainer, Scope) encouraged but not gated. CLAUDE.md / AGENTS.md / README / LICENSE / VERSION / rendered HTML / rendered PDF artifacts are explicitly OUT of scope (revision tracked via VERSION file or auto-derived from source Markdown).

Auto-bump: `scripts/doc_revision_bump.sh <file>` (idempotent), pre-commit hook for automatic staged-doc bumps, `scripts/testing/sync_issues_docs.sh` auto-bumps Issues_Summary / Fixed_Summary after regeneration, `scripts/commit_docs.sh` calls the bump before stage. CONTINUATION.md's existing `Last updated:` line per §12.10 IS the §11.4.44 `Last modified:` line — composed, not duplicated. HTML/PDF exports inherit revision from source Markdown via pandoc pipeline. No escape hatch — no `--skip-revision-bump` flag exists anywhere.

Pre-build gates `CM-DOC-REVISION-HEADER-PRESENT` + `CM-COVENANT-114-44-PROPAGATION` + paired mutations per §1.1. Composes with §12.10 (CONTINUATION.md header reuse), §11.4.12 (Issues_Summary regen), §11.4.22 (`commit_docs.sh` hook entry), §11.4.23 (HTML colorizer preserves revision), §11.4.18 (companion doc for `doc_revision_bump.sh`).

Classification: universal (§11.4.17). See Constitution §11.4.44 for the full mandate.

§11.4.45 — Integration-status-doc maintenance mandate (User mandate, 2026-05-18)

Every non-trivial domain integration MUST have a `docs/<domain>/<integration>/Status.md` document that (1) exists when more than one fix/test/gate has landed for the integration, (2) carries the §11.4.44 revision header, (3) is auto-synced (HTML + PDF) on every related test cycle and every fix touching the integration, (4) is auto-colored per §11.4.23, (5) has a sync wrapper invocable as `bash scripts/testing/sync_integration_status.sh` or a per-integration thin shell, (6) lives under `docs/<domain>/<integration>/`, (7) includes a captured-evidence- driven status table per §11.4.5 (every claim cites the test log / recording / sink-probe report that backs it), (8) uses the closed status vocabulary PASS / FAIL / SKIP / PENDING_FORENSICS / OPERATOR-BLOCKED, (9) lists operator-blocked items at the top so operators find action items in O(1), (10)

is referenced from `docs/CONTINUATION.md §3` (Active work) when any item is non-terminal.

§11.4.45 is the generic form of §12.10 (CONTINUATION.md) applied to every integration domain. Without this generalisation each new integration re-invents the same sync wrapper, revision-header discipline, captured-evidence requirement, and operator-blocked surface.

Pre-build gates `CM-COVENANT-114-45-PROPAGATION` + `CM-AF-INTEGRATION-STATUS-DOCS` + paired mutations (propagation strip / Revision-line delete / sync-staleness / vocabulary violation). Composes with §11.4.5 (captured evidence), §11.4.12 (sync wrapper pattern reused), §11.4.13 (sink-side evidence is a specific instance), §11.4.15 (status vocabulary), §11.4.22 (`commit_docs.sh` wrapper), §11.4.23 (colorizer), §11.4.44 (revision header), §12.10 (CONTINUATION.md references Status.md paths).

No escape hatch — no `--skip-status-sync`, no `--no-revision-bump-on-status`, no `--allow-stale-html` flag.

Classification: universal (§11.4.17). See Constitution §11.4.45 for the full mandate.

§11.4.46 — Validate-recent-work-before-post-flash-tests mandate (User mandate, 2026-05-18)

After every device flash, the orchestrator MUST first run a recent-work validation pass (targeted on-device tests for items currently in `Issues.md` `In progress` / `Ready for testing` / `Reopened`, `Fixed.md` items closed within last 7 days, CONTINUATION.md §3 active work). Only if that pass is 100% green does the orchestrator proceed to the full post-flash suite.

Helper: `scripts/testing/recent_work_validate.sh --device <serial>` writes `/data/local/tmp/.recent_work_validated` IFF green; the full suite (`test_all_fixes.sh`) refuses to start without it. Marker is invalidated on reboot (stores device boot epoch).

Composes with §11.4.4 (STOP-on-discovery) + §11.4.6 (no-guessing) + §11.4.7 (demotion-evidence) + §11.4.40 (full-suite gate) + §11.4.42 + §11.4.43 + §11.4.44 + §12.10. Each recent-item fix MUST have a paired §11.4.43 RED-then-GREEN — a GREEN with no prior RED is a bluff.

Pre-build gates `CM-COVENANT-114-46-PROPAGATION` + `CM-AF-RECENT-WORK-VALIDATION-GATE` + `CM-AF-VALIDATION-ARTIFACT-FILE` + paired mutations.

Classification: universal (§11.4.17). No escape hatch — no `--skip-validation`, `--full-suite-always`, `--ignore-recent-work` flag. See Constitution §11.4.46 for the full mandate.

§11.4.47 — Firebase data review mandate (User mandate, 2026-05-18)

Before every “bigger working round” (pre-build, pre-flash, pre-tag, daily, post-deployment burn-in) the operator/loop MUST execute `scripts/firebase/review_round.sh`. The pass queries Crashlytics (fatals + non-fatals + ANRs) + Analytics + Performance, classifies findings by severity, dedup-maps to existing Issues.md entries via a three-tier algorithm (exact Firebase Issue-ID match → stacktrace- similarity cluster hash → operator merge review), and drafts new Issue entries for unrecognised findings with full Firebase Console URL refs + §11.4.4(a) systematic-debugging output. Skipping the pass is a §11.4 PASS-bluff — Firebase IS the captured evidence from real end-user devices.

Five mandatory elements: (1) 5-trigger cadence (pre-build / pre- flash / pre-tag blocking, daily / burn-in non-blocking), (2) 3-source query (all three sources), (3) Issues.md output with Firebase metadata (Issue IDs + URL + Cluster Hash / KPI / Funnel), (4) 3-tier dedup, (5) comprehensive systematic-debugging output per Issue.

Pre-build gates `CM-COVENANT-114-47-PROPAGATION` + `CM-AF-FIREBASE-REVIEW-CADENCE` + `CM-AF-FIREBASE-ISSUE-XREF` + 3 paired mutations. Composes with §11.4.4 / §11.4.4(a) / §11.4.6 / §11.4.7 / §11.4.10 / §11.4.12 / §11.4.14 / §11.4.15 / §11.4.16 / §11.4.34 / §11.4.42 / §11.4.43 / §11.4.44 / §11.4.45 / §11.4.46.

No escape hatch — no `--skip-firebase-review`, `--firebase-review-not-applicable`, `--no-issue-from-firebase` flag. Operator MAY filter with `--severity-min` but MUST execute the pass.

Classification: universal (§11.4.17). See Constitution §11.4.47 for the full mandate.

§11.4.48 — UI-driven video testing mandate (User mandate, 2026-05-18)

Every test that asserts video playback on a secondary display MUST traverse the user-equivalent UI path (launcher icon → app home → content list → tile tap →

playback → in-app pause/resume → back button stop). NOT Intent/Broadcast shortcuts (`am start -a VIEW` , `cmd media_session play`). Real `uiautomator dump` element resolution + `input tap X Y` . Coverage: every video-capable app in `PRODUCT_PACKAGES` + every stream type (progressive HTTP / HLS / DASH / RTMP / file-local / DRM) + every codec (H.264/265/VP9/AV1/MPEG-2/MP4V video; AC-3/E-AC-3/TrueHD/DTS/DTS-HD/MLP/Opus/AAC audio). Secondary display verified via `ffprobe-on-captured-mp4` + VOM activeDecoder state. Arvus codec-state cross-check per §11.4.13 + §CG screenshot.

Per §11.4.4 four-layer: pre-build gate `CM-AF-UI-DRIVEN-VIDEO- COVERAGE` + propagation gate `CM-COVENANT-114-48-PROPAGATION` + on-device test framework at `device/rockchip/rk3588/tests/ui_driven/` (Layer 1 helper + Layer 2 per-app drivers + Layer 3 scenarios) + Layer 4 orchestrator `scripts/testing/run_ui_driven_video_suite.sh` + paired meta-test mutations.

Carve-out (User mandate 2026-05-20). The 5 canonical tracker documents — `docs/Issues.md` , `docs/Issues_Summary.md` , `docs/Fixed.md` , `docs/Fixed_Summary.md` , `docs/CONTINUATION.md` — sit at `docs/` root by design. They are architectural constants of the project layout, analogous to AOSP's `Makefile` , `Android.bp` , `OWNERS` files at repo root. Their location is encoded as literal path strings in §11.4.12 + §11.4.15 + §11.4.16 + §11.4.19 + §11.4.44 + §11.4.53 propagation gates plus the helper-script constellation that regenerates them. Moving them would require coordinated amendment of those 6 sister anchors plus 5 pre-build gates plus ~20 helper scripts plus 42 consumer files in a single PWU. Per §11.4.66, that scope is operator-blocked until explicitly authorised. Audit-snapshot files (`docs/audit/anti_bluff_audit.md` , `docs/audit/PRE_SONOS_TAG_READINESS.md` , `docs/audit/D1_WIFI_FAIL_CLASSIFICATION.md` , plus any future audit snapshots) DO move under `docs/audit/` per the §11.4.11 general principle.

Canonical authority: constitution submodule `Constitution.md` §11.4.48.

Non-compliance is a release blocker regardless of context.

§11.4.49 — Dual-approach testing mandate (User mandate, 2026-05-18)

Every feature test exercising a user-visible behaviour MUST ship in TWO variants: a UI-driven variant (`uiautomator`-based, §11.4.48 surfaces A–E) AND an Intent/Broadcast-driven variant (`am start --es` / `am broadcast` -based). Either alone is

a §11.4 PASS-bluff for the OPPOSITE half of the stack — UI catches app-side bugs; Intent catches framework/system-server bugs.

Shared assertion base at `tests/lib/dual_approach_test_base.sh`: `dat_init` / `dat_start_capture` / `dat_assert_codec_state` / `dat_assert_video_frames` / `dat_assert_audio_channels` / `dat_arvus_dashboard_capture` / `dat_cleanup` / `dat_report_finding`. Both variants gather identical evidence into mirror directories `qa-results/dual_approach/<F>/<run-ts>/{ui,intent}/` so the orchestrator diffs results and pinpoints which half of the stack contains a bug.

Kinopoisk 5.1 EAC3 is the canonical first implementation. Both variants are RED per §11.4.43 until the §CN decoder pipeline fix lands.

Pre-build gates: `CM-COVENANT-114-49-PROPAGATION` (anchor across parent + 42 consumer files) + `CM-AF-DUAL-APPROACH-COVERAGE` (shared base contract) + `CM-AF-KINOPOISK-5-1-DUAL-COVERAGE` (both variants exist + share the base). Three paired meta-test mutations.

No escape hatch — no `--ui-only` / `--intent-only` / `--skip_dual` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.49.

Non-compliance is a release blocker regardless of context.

§11.4.50 — Deterministic consistency mandate (User mandate, 2026-05-18)

Every test that PASSes MUST have been executed N times (default N=3 normal tests, N=10 cycle-validation suites) against the same firmware MD5 + same device + same topology and produced IDENTICAL PASS in every iteration. A divergent N-iter run is auto-FAIL — there is no “first PASSEd therefore X was a flake” path.

§11.4.7’s `intermittent` / `transient` / `flake` / `flaky` vocabulary is enforced MECHANICALLY by this mandate, not merely textually.

Coverage: every public API path (Activity / Service / Broadcast receiver / ContentProvider URI / IPC interface / JNI entry / sysprop write / sysfs node / init.rc trigger) MUST have ≥1 dedicated test that drives it. Untested paths surface in a feature-coverage-matrix audit; the threshold ratchets 70 → 85 → 95 → 99 over phases.

Reliability mechanism: `ab_run_n_times <test_name> <N> <fn> [args...]`

helper in the project anti-bluff library loops, captures evidence-hash per iter, asserts all N hashes + exit codes identical. NO operator-facing escape converts divergence to PASS.

Pre-build gates: `CM-COVENANT-114-50-PROPAGATION` + `CM-AF-RELIABILITY-CHECK-WIRED` + `CM-AF-FEATURE-COVERAGE-MATRIX` . Three paired meta-test mutations. No escape hatch — no `--allow-flake` , `--first-pass-suffices` , `--skip-n-iter` , `--skip-coverage-audit` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.50.

Non-compliance is a release blocker regardless of context.

§11.4.51 — Live-ADB-First Maximization Mandate (User mandate, 2026-05-18)

Every fix MUST be classified by rebuild-requirement before commit using the project's per-file-class decision matrix. If `LIVE_ADB_TESTABLE` (on-device test scripts, host scripts, atmosphere- *.sh boot scripts*, *persist.* properties, markdown docs, test fixture assets), the operator MUST first apply the fix to the running device via `adb push` / `setprop` / `pm install -r` / `mount -o remount,rw` , run the §11.4.43 RED test live, capture PASS, THEN commit + rebuild + reflash as belt-and-suspenders re-validation. Commit footer: `LIVE_ADB_VALIDATED: yes` . If `REQUIRES_REBUILD` (kernel, framework Java/AIDL, native C++ in APEX, sepolicy, init.rc, ro.* properties, XML overlays, codec XML in APEX, Android.bp/.mk), the operator proceeds directly to source-side + rebuild. Commit footer: `REQUIRES_REBUILD: <reason>` . Mixed batches use partial.

§11.4.51 REFINES §11.4.43 step 2 with mechanical enforcement. Helper:

`scripts/testing/classify_fix_rebuild_requirement.sh` walks `git diff --name-only` , looks up each file against the matrix, emits per-file classification + recommended commit-message footer. Unmatched paths classify as `REQUIRES_REBUILD: unmatched-path` (safe default per §11.4.6). Pre-build gates: `CM-COVENANT-114-51-PROPAGATION` + `CM-AF-CLASSIFY-FIX-HELPER-EXISTS` + `CM-AF-LIVE-ADB-FIRST-COMMIT-MARKER` . Three paired meta-test mutations. No escape hatch — no `--skip-classify` / `--assume-rebuild` / `--no-footer-required` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.51.

Non-compliance is a release blocker regardless of context.

§11.4.52 — Autonomous-Validation Mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18):

“Make sure we have full automation tests which will do all this work in full automation! IMPORTANT: Make sure that all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can’t be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!”

Every user-facing feature MUST have at least one autonomous validation path: end-to-end via `adb shell` + scripted automation, captured runtime evidence per §11.4.5, PASS/FAIL verdict WITHOUT a human present to drive UI, observe screen, or make decisions. Operator-attended tests are SUPPLEMENTARY, never PRIMARY. A feature whose ONLY validation path is operator-attended is a §11.4.52 violation — the path does not scale to CI, does not run on every commit, does not survive operator unavailability, and produces the exact “tests pass but feature doesn’t work for users” failure mode §11.4 forbids.

Acceptable autonomous paths: programmatic instrumentation APK (SDK-API exercises like `MediaCodec.createDecoderByName` + JSON result file), headless intent dispatch + state poll (`am start --es + dumpsys /proc/<pid>/maps /media.metrics` polling), ADB-driven uiautomator (ONLY if `uiautomator dump | grep -c clickable=true ≥ 1` — near-empty hierarchy proves UI-driven INFEASIBLE and demands fallback to APK/intent), network-side sink probe (Arvus dashboard, Sonos REST, etc. per §11.4.13), HelixQA autonomous QA session (§11.4.27).

Per-feature coverage ledger (§11.4.25) MUST classify each row as `AUTONOMOUS_VERIFIED` / `AUTONOMOUS_DESIGNED` / `OPERATOR_ATTENDED_ONLY` / `NOT_APPLICABLE`. `OPERATOR_ATTENDED_ONLY` is a release blocker until promoted, citing a tracked migration work item per §11.4.15 + §11.4.16. Autonomous paths themselves MUST be anti-bluff: positive captured evidence per §11.4.5, paired meta-test mutation per §1.1.

Composes with §11.4.25 (full-automation-coverage — §11.4.52 strictens invariants 1+2), §11.4.27 (no-fakes-beyond-unit + 100% type coverage — operational layer closing the gap), §11.4.39 (per-feature on-device end-user validation — refined to mandate autonomous drivability), §11.4.43 (TDD-fix — autonomous path RED

before fix, GREEN after), §11.4.48 (UI-driven — fallback to APK/intent when uiautomator hierarchy empty), §11.4.49 (dual-approach — Intent variant often IS the autonomous path), §11.4.50 (deterministic consistency — autonomous paths scale to N iterations), §11.4.51 (live-ADB-first — instrumentation APK is LIVE_ADB_TESTABLE).

Pre-build gates `CM-COVENANT-114-52-PROPAGATION` (anchor literal across canonical files) + `CM-AF-AUTONOMOUS-PATH-PER-FEATURE` (coverage-ledger classification column non-empty + valid value). Paired mutations strip the anchor literal AND inject an `OPERATOR_ATTENDED_ONLY` row without a tracked migration item. No escape hatch — no `--allow-operator-attended-only`, `--skip-autonomous-path`, `--manual-validation-suffices` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.52.

Non-compliance is a release blocker regardless of context.

§11.4.53 — Fixed_Summary parity mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18T17:55Z):

“Note: Just like for Issues we have `Issues_Summary`, for Fixed we MUST HAVE `Fixed_Summary` - like all other docs: ALWAYS in sync and up to date and ALWAYS exported into the PDF and HTML! Add this mandatory rule / constraint into the root (constitution Submodule) `Constitution`, `AGENTS.MD` and `CLAUDE.MD`.”

`docs/Fixed_Summary.md` is the symmetric short-form summary of `docs/Fixed.md`. MUST be regenerated whenever `Fixed.md` changes. HTML + PDF exports MUST travel with the markdown (identical mtimes within `sync_issues_docs.sh` granularity). Stale exports are §11.4.53 violations regardless of whether the underlying `.md` is correct — an operator (or future agent) reading the HTML or PDF gets a divergent view of which items are closed, and the §12.10 CONTINUATION resumption guarantee silently breaks. Same discipline as §11.4.12 `Issues_Summary` applied to `Fixed.md`.

Generator: `scripts/testing/generate_fixed_summary.sh` (canonical, MUST be executable, MUST emit a markdown table whose header columns include `Status` and `Type` per §11.4.19 column-alignment). Auto- sync wrapper: `scripts/testing/sync_issues_docs.sh` regenerates BOTH `Issues_Summary`

AND Fixed_Summary in one shot (stages 1a + 1b), then exports HTML + PDF (stage 2), then colorizes per §11.4.23 (stage 3), then re-renders the PDFs from the colorized HTML (stage 3b). MUST be invoked after any edit to Fixed.md . No --issues-only flag exists, and §11.4.53 prohibits adding one.

Sort order: closure date DESC (most-recent-Fixed first), §-letter / Fix-# secondary. Documented at the top of the generated file.

Composes with §11.4.12 (Issues_Summary parity is the sibling rule; §11.4.12 + §11.4.53 are the canonical pair), §11.4.19 (atomic Issues → Fixed migration triggers Fixed_Summary regen — column-aligned structure), §11.4.23 (visual-cue + grouping colorizer post-processes both summaries), §11.4.33 (type-aware closure vocabulary — Fixed_Summary respects Fixed (→ Fixed.md) / Implemented (→ Fixed.md) / Completed (→ Fixed.md) terminal values literally), §11.4.44 (revision header applies to Fixed_Summary.md), §12.10 (CONTINUATION.md resumption guarantee depends on divergent-summary problem NOT existing).

Pre-build gates CM-FIXED-SUMMARY-SYNC (6 invariants: Fixed_Summary exists; HTML + PDF mtime ≥ md mtime; Fixed_Summary mtime ≥ Fixed mtime; generator script exists + executable; sync wrapper invokes generator) + CM-COVENANT-114-53-PROPAGATION (anchor literal across canonical files + per-consumer propagation). Paired mutations strip the anchor literal AND move the generator aside AND backdate Fixed_Summary mtime. No escape hatch — no --skip-fixed-summary-sync , --issues-only , --summary-not-applicable flag.

Canonical authority: constitution submodule Constitution.md §11.4.53.

Non-compliance is a release blocker regardless of context.

§11.4.54 — ATM-NNN ticket identifier mandate (User mandate, 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

“Every workable item in Issues.md / Issues_Summary.md / Fixed.md / Fixed_Summary.md MUST carry a stable, unique, auto-incremental ATM-NNN ticket identifier. ATM- prefix, monotonic, never renumbered, append-only.”

Every workable item in docs/Issues.md AND docs/Fixed.md MUST carry a [ATM-NNN] ticket identifier in its heading (form ## \$X.Y. [ATM-NNN] <title> , zero-padded ≥3 digits). Identifiers are allocated by scripts/testing/

`assign_atm_ticket_ids.sh` and persisted to an append-only state file `scripts/testing/.atm_ticket_state.json` (jsonl: `atm_id` , `heading_hash` , `type` , `current_location` , `current_status` , `reopens_count` , `created_at` , `last_modified`). Once assigned, an ATM-NNN MUST NEVER be renumbered, reused, decremented, or skipped. `heading_hash` is the binding key — wording reflows preserve the binding via the state-file lookup.

`Issues_Summary.md` and `Fixed_Summary.md` MUST carry an `ATM ID` column as the leftmost data column. Generators (`generate_issues_summary.sh` , `generate_fixed_summary.sh`) emit the column so operators / agents can sort + filter on it.

Composes with §11.4.15 (Status), §11.4.16 (Type), §11.4.19 (column-alignment), §11.4.33 (closure vocabulary), §11.4.12 + §11.4.53 (Issues_Summary + Fixed_Summary regen — helper invoked from `sync_issues_docs.sh`), §11.4.55 (per-item Reopens.md path key), §11.4.57 (README.md doc-link cross-reference key).

Pre-build gates `CM-ATM-TICKET-IDS-COMPLETE` (every heading carries `[ATM-NNN]`) + `CM-ATM-TICKET-IDS-UNIQUE` (no duplicates) + `CM-ATM-TICKET-IDS-MONOTONIC` (no gaps) + `CM-COVENANT-114-54-PROPAGATION` (anchor literal across canonical files). Paired mutations strip an `[ATM-NNN]` heading token, dup an ID in the state file, gap the sequence at NNN=2, strip the anchor literal. No escape hatch — no `--skip-atm-assignment` , `--renumber` , `--no-atm-id-required` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.54.

Non-compliance is a release blocker regardless of context.

§11.4.55 — Reopens-history tracking + per-item Reopens.md doc (User mandate, 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

“Add a Reopens-count column. For any item whose reopens-count > 0, create docs/issues/ATM-NNN/Reopens.md (+ HTML + PDF) with comprehensive reopen history + Fixed cycles. Each reopen MUST include By (AI / User), On, Reason, Evidence, and each Fixed-marking with reasoning chain.”

Every workable item with `reopens_count > 0` MUST have a companion document at `docs/issues/<ATM-NNN>/Reopens.md` (+ HTML + PDF). The document MUST contain: (1) §11.4.44 revision header, (2) item identification (ATM ID, Title, Type, Current Status, Current Location, link back to live heading), (3) cycle counters (Total reopens, Total fixed cycles), (4) chronological timeline — one entry per state-change event, each with By (AI/User per §11.4.34), On (ISO date), Event (Opened/ Reopened/Fixed/Implemented/ Completed), Reason (closed-vocabulary value from §11.4.34 for reopens; captured-evidence summary for closures), Evidence (path or short description), Outcome, (5) reasoning chain for each closure (root-cause analysis, captured-evidence under same conditions per §11.4.7, gate / mutation pair), (6) most-recent state-change pointer.

`Issues_Summary.md` and `Fixed_Summary.md` MUST carry a `Reopens` column; cells with count > 0 hyperlink to the per-item `Reopens.md`. The §11.4.23 colorizer MAY apply a visual cue when `reopens > 2`.

Composes with §11.4.34 (per-event capture in current heading — §11.4.55 is the per-item history aggregation), §11.4.54 (ATM-NNN provides the stable path), §11.4.44 (revision header), §11.4.45 (`Status.md` per-integration analogue), §11.4.53 (`Fixed_Summary` parity — `Reopens` column symmetric on both summaries).

Pre-build gates `CM-REOPENS-DOC-EXISTS-WHEN-COUNT-GT-ZERO` + `CM-REOPENS-DOC-REVISION-HEADER` + `CM-REOPENS-COL-IN-SUMMARIES` + `CM-COVENANT-114-55-PROPAGATION`. Paired mutations delete a `Reopens.md` for a `reopens_count=2` item, strip the revision header, remove the column from `Issues_Summary`, strip the anchor literal. No escape hatch — no `--skip-reopens-doc`, `--collapse-history`, `--reopens-not-applicable` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.55.

Non-compliance is a release blocker regardless of context.

§11.4.56 — `Status_Summary` parity + two-audience format (User mandate, 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

“Every `Status.md` doc gets a `Status_Summary` parity companion ALWAYS in sync, exported to HTML + PDF. Two-page format: page 1 = non-developer

audience (team-specific), page 2 = software engineer summary. Auto-generated after every main Status update.”

For every `docs/<domain>/<integration>/Status.md` (§11.4.45) a companion `Status_Summary.md` MUST exist with: (1) §11.4.44 revision header, (2) **Page 1 — For the** — audience- specific heading (audio team for `docs/dolby/*` , video team for `docs/video/*` , etc.) — plain-language summary, What works (1-3 bullets), What’s broken or pending (1-3 bullets), Operator / team actions if any. NO code references, NO §-letter jargon, NO captured-evidence file paths, NO gate / mutation names. (3) **Page 2 — For software engineers** — §-letter references, gate names, commit hashes, captured-evidence paths, ATM-NNN cross-references. HTML + PDF exports travel with the markdown.

Generator: `scripts/testing/generate_status_summary.sh <Status.md path>` produces both pages. Invoked from `scripts/testing/sync_integration_status.sh` (§11.4.45 sync wrapper) on every Status.md update.

Composes with §11.4.45 (Status.md per integration — Status_Summary.md COMPLEMENTS, never replaces), §11.4.12 + §11.4.53 (parity discipline), §11.4.44 (revision header), §11.4.23 (colorizer for tracked-item references on page 2), §12.10 (CONTINUATION.md — non-developer stakeholders read Status_Summary; engineers read Status + CONTINUATION + Issues).

Pre-build gates `CM-STATUS-SUMMARY-EXISTS-FOR-EVERY-STATUS` + `CM-STATUS-SUMMARY-TWO-AUDIENCE` + `CM-STATUS-SUMMARY-REVISION-HEADER` + `CM-COVENANT-114-56-PROPAGATION` . Paired mutations delete a Status_Summary.md, remove the Page 1 heading, strip the anchor literal. No escape hatch — no `--skip-status-summary` , `--engineer-only` , `--no-audience-split` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.56.

Non-compliance is a release blocker regardless of context.

§11.4.57 — README.md doc-link section + revision metadata (User mandate, 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

“Add a doc-link section to README.md — links to Issues + Issues_Summary + Fixed + Fixed_Summary + CONTINUATION + ALL Status docs + their exports. Each link shows revision + last-modified.”

Every project’s top-level README.md MUST contain a section titled Tracked-Items + Status Documents (heading MUST contain literal Tracked-Items). The section is a markdown table with columns: Document , Last modified (ISO 8601 UTC from §11.4.44 header), Revision (integer from same header), Markdown link, HTML link, PDF link. The section MUST link to: Issues.md + Issues_Summary.md (§11.4.12, §11.4.15, §11.4.16), Fixed.md + Fixed_Summary.md (§11.4.19, §11.4.53), CONTINUATION.md (§12.10), every docs/**/Status.md + its Status_Summary.md pair (§11.4.45, §11.4.56). Status docs are auto-discovered by the generator via `find docs -name 'Status.md' .`

Generator: `scripts/testing/update_readme_doc_links.sh` extracts each doc’s Revision + Last modified , renders the markdown table, replaces the section between markers (`<!-- doc-link-section:begin -->` / `<!-- doc-link-section:end -->`) in README.md. Invoked from `sync_issues_docs.sh` (Issues / Fixed edits) AND `sync_integration_status.sh` (Status edits).

Composes with §11.4.12 + §11.4.19 + §11.4.53 (Issues / Fixed / summaries — README links all four), §11.4.44 (revision header data source), §11.4.45 + §11.4.56 (Status pairs enumeration), §12.10 (CONTINUATION.md explicit link).

Pre-build gates CM-README-DOC-LINK-SECTION-PRESENT (literal Tracked-Items + section markers) + CM-README-DOC-LINK-ROWS-COMPLETE (every canonical doc appears as a row) + CM-README-DOC-LINK-FRESHNESS (Last modified matches source within sync-wrapper granularity) + CM-COVENANT-114-57-PROPAGATION . Paired mutations strip the markers, remove the CONTINUATION row, backdate a Last modified cell, strip the anchor literal. No escape hatch — no `--skip-readme-doc-links` , `--collapse-status-rows` , `--no-freshness-check` flag.

Canonical authority: constitution submodule Constitution.md §11.4.57.

Non-compliance is a release blocker regardless of context.

§11.4.58 — Parallel-development methodology (User mandate, 2026-05-19)

Forensic anchor — direct user mandate (verbatim, 2026-05-19T~05:00Z MSK):

"We MUST DO one comprehensive research and planning in the background in parallel with current mainstream work: our current methodology of the development is very slow. ... We MUST CREATE adjusted improved version of working methodology where multiple workable items could be done in parallel, all come into the central (main) branch as they are done, then we at particular moment rebuild and reflash the System and in background full testing with validation and verification is done. Each parallel work on one workable (or more) item(s) must use parallel agents as much as possible! ... Writing in depth tests (all supported types of the tests) with Challenges and full HelixQA use is MANDATORY! Every test we execute besides executed with success MUST RESULT in proof that actual functionality being tested REALLY DOES WORK with NO BLUFF of any kind! Heavt enforcement of no-bluff / anti-bluff policy IS MANDATORY!"

Project work proceeds through the **Parallel Work Unit (PWU) pipeline** rather than sequential phase-chain. Each PWU is a self-contained workable item with mandatory components: ATM-NNN identifier per §11.4.54, Issues.md entry per §11.4.15+§11.4.16, file-scope manifest, §11.4.43 RED test, source patch, pre-build gate per §11.4.4(b) layer 1, post-flash test per §11.4.4(b) layer 3, paired §1.1 meta-test mutation, §11.4.4(b) layer 4 HelixQA Challenge bank entry, captured-evidence directory per §11.4.5+§11.4.52.

5-stage pipeline: (Stage 1 DEVELOP — parallel PWU agents in isolated worktrees) → (Stage 2 MERGE — serial conductor via `commit_all.sh` flock + §11.4.41 4-step merge-first) → (Stage 3 REBUILD+FLASH — parallel where hardware allows) → (Stage 4 VALIDATE — parallel on D3+D4+meta-test+coverage) → (Stage 5 SWEEP — parallel HelixQA Challenges + Fixed.md migration + README refresh). Stage 1 of round N+1 overlaps with Stages 4-5 of round N — the throughput multiplier.

Synchronization mechanism: 4-layer lock hierarchy (L1 parent flock / L2 per-submodule git ops / L3 contention-path advisory locks for the 10 forbidden cross-PWU paths / L4 per-PWU git worktree). Disjoint-scope PWUs run fully parallel. Conflict detection at Stage 2: `git fetch --all --prune --tags` + scope-overlap check → overlap detected rejects PWU back to Stage 1.

Anti-bluff enforcement at merge time (all four required): C1 §11.4.43 RED-test captured-evidence (proves test catches regression), C2 §1.1 paired meta-test

mutation (proves gate is not bluff), C3 §11.4.50 deterministic-consistency (3 or 10 iterations identical), C4 §11.4.5 captured-evidence (audio WAV / video screen recording / UI uiautomator dump / HelixQA `result.json`). Metadata-only / configuration-only / absence-of-error / grep-without-runtime PASS all REJECTED.

HelixQA mandatory. Every user-visible PWU MUST add a Challenge entry in `tools/helixqa/banks/atmosphere.yaml` referencing the PWU's ATM-NNN + dispatching to its on-device test + scoring PASS only on positive captured evidence per §11.4.5.

Pre-build gates `CM-PWU-LOCK-HIERARCHY` + `CM-PWU-ANTI-BLUFF-COVERAGE` + `CM-PWU-MERGE-QUEUE-DISCIPLINE` + `CM-PWU-PARALLEL-AGENT-LIMIT` + `CM-COVENANT-114-58-PROPAGATION` (anchor literal across 42 consumer files). Paired mutations strip the lock helpers / forge a READY marker without evidence / bypass the merge queue / spawn a 7th concurrent agent / strip the anchor literal — every mutation FAILs its gate.

No escape hatch — no `--skip-merge-queue` , `--allow-bypass` , `--no-anti-bluff-check` , `--unlimited-agents` , `--sequential-phase-chain-` mode flag. Composes with §11.4.4, §11.4.5, §11.4.6, §11.4.9, §11.4.15, §11.4.16, §11.4.19, §11.4.33, §11.4.41, §11.4.42, §11.4.43, §11.4.45, §11.4.49, §11.4.50, §11.4.52, §11.4.54, §11.4.57, §12.6, §12.7, §12.8, §12.10, §9.2.

Canonical authority: constitution submodule `Constitution.md` §11.4.58. Project-specific implementation reference in consumer-side `docs/guides/PARALLEL_DEVELOPMENT_METHODODOLOGY.md` .

Non-compliance is a release blocker regardless of context.

§11.4.59 — README always-sync mandate (User mandate, 2026-05-19)

`README.md` is a §11.4.12-class always-sync document: HTML + PDF exports refresh on every update via `scripts/testing/sync_readme_export.sh` (pandoc + weasyprint); auto-invoked by `sync_issues_docs.sh` so a single doc-sync run refreshes Issues / Issues_Summary / Fixed / Fixed_Summary / CONTINUATION / README (md + html + pdf). README carries a §11.4.44 revision header and a Documentation Map section linking to every Status.md + Status_Summary.md + spec + plan + guide + script-companion doc + changelog + the constitution submodule, plus per-audience navigation. Pre-build gate `CM-README-EXPORT-SYNC` enforces mtime parity (`README.html` + `README.pdf` ≥ `README.md`). Paired meta-test mutation backdates HTML+PDF → gate FAILs. No escape hatch — no

`--skip-readme-sync` , `--no-readme-export` , `--readme-stale-OK` flag.

Composes with §11.4.12 + §11.4.18 + §11.4.44 + §11.4.45 + §11.4.53 + §11.4.56 + §11.4.57 + §12.10.

Canonical authority: constitution submodule `Constitution.md` §11.4.59.

Non-compliance is a release blocker regardless of context.

§11.4.60 — Documentation always-sync composite covenant (User mandate, 2026-05-19)

Eight documentation classes (Issues, Issues_Summary, Fixed, Fixed_Summary, CONTINUATION, README, every Status.md, every Status_Summary.md) MUST be in sync at all times across `.md` + `.html` + `.pdf` artefacts. Per-class anchors §11.4.12 / §11.4.44 / §11.4.45 / §11.4.53 / §11.4.56 / §11.4.57 / §11.4.59 / §12.10 govern individually; §11.4.60 binds them via single composite gate `CM-DOCS-COMPOSITE-SYNC` that FAILs if ANY instance's `.html` or `.pdf` mtime is older than `.md` mtime. Walks `docs/**` recursively for Status.md fleet. Paired mutation backdates `docs/Issues.html` → gate FAILs. No escape hatch — no `--skip-composite-doc-sync` , `--allow-stale-html` , `--summary-not-applicable` flag exists. Composes with §11.4.12 + §11.4.15 + §11.4.16 + §11.4.19 + §11.4.23 + §11.4.33 + §11.4.44 + §11.4.45 + §11.4.53 + §11.4.56 + §11.4.57 + §11.4.59 + §12.10.

Canonical authority: constitution submodule `Constitution.md` §11.4.60.

Non-compliance is a release blocker regardless of context.

§11.4.63 — Workable-items procedure docs as single source of truth (User mandate, 2026-05-19)

Every workable-item action (open / update / close / reopen / migrate) MUST follow the canonical procedure document at `docs/procedures/issues/<Action>.md` . Closed-set of 5 procedure docs: `Creation.md` , `Updating.md` , `Resolution.md` , `Reopening.md` , `Migration.md` . Each carries §11.4.44 revision header + HTML + PDF exports (refreshed by `scripts/testing/sync_procedure_docs_export.sh` , invoked as the final stage of `sync_issues_docs.sh`). ALL work — new features, behavioural changes, tasks (refactor / doc / infra / gate / audit / cleanup), bug fixes, investigation, documentation — MUST flow through these procedures. No ad-hoc procedure permitted.

Pre-build gate `CM-PROCEDURES-DOCS-PRESENT` checks all 5 procedure docs exist + carry revision header + have current HTML/PDF exports + sync helper is wired into `sync_issues_docs.sh`. Paired mutation: rename one procedure doc → gate FAILs. Propagation gate `CM-COVENANT-114-63-PROPAGATION` enforces this anchor literal in every `CLAUDE.md` / `AGENTS.md`.

Canonical authority: constitution submodule `Constitution.md` §11.4.63.

Non-compliance is a release blocker regardless of context.

MANDATORY HOST-SESSION SAFETY (Constitution §12)

Every script, test, helper, and AI agent **MUST** respect host-session safety. Non-compliance is a release blocker.

Forbidden — directly OR indirectly

1. Suspending the host (`systemctl suspend` , etc.).
2. Hibernating / hybrid-sleeping the host.
3. Logging out the user (`loginctl terminate-session` , `kill -u <user>` , anything targeting `user@<uid>.service`).
4. Unbounded-memory operations inside `user@<uid>.service` cgroup — any command expected to exceed ~4 GiB RSS **MUST** be wrapped in a bounded execution scope.
5. Programmatic `rkill` toggles, lid-switch handlers, power-button handlers — these cascade into idle-actions.
6. Disabling session managers “to make things faster”.

Required safeguards for heavy scripts

1. Source the project’s host-safety helper library.
2. Call its pre-flight check and abort if it fails.
3. Wrap any subprocess expected to exceed ~4 GiB RSS in a bounded execution scope.
4. Cap parallelism to fit available RAM.

§12.6 Memory-Budget Ceiling — 60% MAXIMUM

Forensic anchor — verbatim user mandate:

“First make sure that whatever we do through our procedures related to this project MUST NOT use more than 60% of total system memory! All processes MUST be able to function normally!”

Project procedures MUST NOT use more than **60%** of total system RAM. The remaining 40% is reserved for the operator’s other workloads. No escape hatch — bypassing this is the bluff §11.4 forbids.

§12.10 Continuation document maintenance

`docs/CONTINUATION.md` MUST exist at the project root and reflect the live state of the work. Every non-trivial state change updates it in the same commit. Any agent must be able to resume work exactly where the previous session left off by reading this single file.

MANDATORY ABSOLUTE DATA SAFETY — ZERO RISK (Constitution §9)

Every destructive repository operation (history rewrite, force-push, branch deletion, bulk file removal, submodule de-init, object pruning) MUST follow the full §9 safety protocol WITHOUT EXCEPTION:

1. **Backup first, always** — hardlinked mirror of `.git` to a sibling backup directory (`cp -al .git <backup>/repo.git.mirror`) is near-instant and uses zero additional disk.
2. **Record metadata** — refs, tags, submodule pointers, HEAD commit, HEAD tree hash, tree content sha256.
3. **Define expected post-op state.**
4. **Run the destructive operation** — never with `--no-verify` , never with `--force` that bypasses hooks, never auto-force on failure.

5. **Post-op gate** — HEAD tree identical, all tags preserved, all submodule pointers intact, per-entry archive integrity 100%, pre-build gates green. If any check fails → restore immediately.
6. **Force-push authorization** — force-push is NEVER automatic. Each force-push event requires explicit human authorization AND requires the post-op gate to have passed.
7. **Audit trail** — every history rewrite gets a `docs/changelogs/` “Force-push audit” section.

Hardlinked backup is so cheap (zero disk, <2 s) that there is NEVER an excuse to skip it.

MANDATORY COMMIT & PUSH CONSTRAINTS

1. **Use the project’s official commit wrapper** for the main repo (e.g. `scripts/commit_all.sh`).
2. **NEVER use `git add` , `git commit` , or `git push` directly** on the main repo unless the project Constitution explicitly carves out a use case.
3. **Multi-upstream push is the norm** — every commit pushed to ALL configured upstream remotes. The constitution submodule ships an `install_upstreams.sh` (and an `Upstreams/` directory) that configures all remotes locally.
4. **NEVER skip hooks** (`--no-verify` , `--no-gpg-sign`) unless the user explicitly authorizes it for the session.

MANDATORY TESTING CONSTRAINTS

Tests MUST be run at every stage WITHOUT EXCEPTION:

1. **Pre-build / pre-merge verification** — BEFORE every build.
2. **Post-build / packaging verification** — AFTER every build.
3. **Post-deploy verification** — AFTER every deploy / flash.

NEVER skip tests. NEVER mark a test as “broken — disable for now” without fixing the underlying issue. NEVER ship a release with unresolved WARNs.

Code conventions

Universal conventions applicable to most projects:

- Use the project's preferred language for new code. Don't mix languages in one module without explicit Constitution permission.
- Static analyzers / linters / type-checkers MUST run clean (zero warnings) before commit.
- Style is set by the project's formatter; do not hand-edit style in PRs.
- Public APIs are documented at the source.

When in doubt

- Read `constitution/Constitution.md` for the canonical text.
 - Cross-reference the project's `CLAUDE.md` / `AGENTS.md` for project- specific extensions.
 - If still unclear, ask the operator. Do NOT guess. Do NOT bluff.
-

§11.4.65 — Universal Markdown export mandate (User mandate, 2026-05-19)

Every Markdown document inside the project that is NOT part of an application or service's source-code tree MUST have synchronized `.html` and `.pdf` siblings. Includes: project-root `*.md` , `docs/**/*` , `scripts/**/*` (doc-format companion docs), owned-submodule top-level `README.md` / `CLAUDE.md` / `AGENTS.md` / `CHANGELOG.md` and their `docs/**/*` , `constitution/**/*` , owned HelixQA submodules' equivalents. Excludes: `external/**` , `prebuilts/**` , `packages/modules/**` , `kernel-5.10/**` , `out/**` , `build/**` , application/service source-code trees, and third-party submodules NOT in the owned set. Every edit triggers regeneration via `scripts/testing/sync_all_markdown_exports.sh` (pandoc HTML + weasyprint PDF, `timeout 60` per file, capped at 500 candidates). HTML + PDF mtime MUST be $\geq$ source `.md` mtime at all times.

Pre-build gates `CM-UNIVERSAL-MARKDOWN-EXPORT-SYNC` + `CM-COVENANT-114-65-PROPAGATION` . Paired meta-test mutations. Composes with §11.4.12 / §11.4.18 / §11.4.23 / §11.4.44 / §11.4.45 / §11.4.53 / §11.4.59 / §11.4.60 / §11.4.63 / §11.4.64.

No escape hatch — no `--skip-md-exports` , `--no-pdf-only` , `--md-export-not-applicable` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.65.

Non-compliance is a release blocker regardless of context.

§11.4.66 — Blocker-resolution interactive-clarification mandate (User mandate, 2026-05-19)

When any task is blocked (operator decision, hardware access, external authorization, ambiguous scope), the agent MUST: (1) research what's doable from the agent side without operator input; (2) calculate minimum-viable operator input; (3) construct 2–4 mutually-exclusive options with one marked “Recommended” and each stating what the agent does after that answer; (4) present via the platform's interactive question mechanism (`AskUserQuestion` on Claude Code) — NEVER free-text “what would you like?” for closed- set decisions; (5) after the answer, resume work without follow-up round-trips. Composes with §11.4.6 / §11.4.7 / §11.4.40 / §11.4.41 / §11.4.42 / §11.4.52. No silent waiting; no bulk-text questions when interactive options would do.

Pre-build gate `CM-COVENANT-114-66-PROPAGATION` enforces the anchor literal across the 42-file consumer fleet. Paired meta- test mutation strips the literal → gate FAILs. No escape hatch — no `--skip-ask` , `--silent-wait` , `--free-form-only` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.66.

Non-compliance is a release blocker regardless of context.

§11.4.67 — Shell-script target-shell-parseability mandate (User mandate, 2026-05-19)

Forensic anchor — direct user mandate (verbatim, 2026-05-19): “any issue we spot must be fixed, bash scripts as well if they are broken!” + “Make sure that this is mandatory rule!”

Every shell script that may be invoked under a target shell other than the one in its shebang MUST parse cleanly under that target shell. Forensic incident: `device/rockchip/rk3588/tests/test_all_fixes.sh:114` used bash-only `exec > >(tee -a "$f") 2>&1` on a `sh script.sh` callsite — Android mksh parses the whole script BEFORE executing, so the runtime `[ -n "$`

`{BASH_VERSION:-}" ]` guard could not save it. Fixed by wrapping in `eval 'exec > >(tee ...) 2>&1'` so the parser sees only a string.

Closed-set scope: every tracked `.sh` under `device/rockchip/rk3588/tests/`, `scripts/`, `scripts/testing/` (and equivalent paths in owned submodules).
OUT of scope: `external/`, `prebuilts/`, `packages/modules/`, `kernel-5.10/`, `out/`, `build/`, `scripts/legacy/`. Mandatory invariants: (1) every in-scope script parses under `sh -n`; (2) bash-only constructs (`>(...)`, `<(...)`, `[[ ]]`, `<<<`, arrays, `${var^^}`, etc.) MUST be wrapped in `eval` OR guarded by bash-only loading; (3) shebangs honest — `#!/bin/bash` only if bash actually expected; (4) fix at source per §11.4.1, never at callsites. Composes with §11.4.1 / §11.4.4 / §11.4.6 / §11.4.50 / §11.4.51.

Pre-build gate `CM-SCRIPT-TARGET-SHELL-PARSEABLE` runs `sh -n` on every in-scope script. Propagation gate `CM-COVENANT-114-67-PROPAGATION` enforces the anchor literal across the 44-file consumer fleet. Paired mutations: inject bash-only outside `eval` → parse gate FAILs; strip `11.4.67` literal → propagation gate FAILs. No escape hatch — no `--skip-parseability-check`, `--bash-only-script`, `--runtime-guard-suffices` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.67.

Non-compliance is a release blocker regardless of context.

§11.4.68 — Positive sink-side / downstream evidence mandate (User mandate, 2026-05-20)

A test asserting audio or video routing PASS MUST capture and verify positive sink-side or downstream evidence — never config-only, never metadata-only, never PCM-open-state-only. Closed enumeration of acceptable evidence: (1) sink-side codec-state showing non-empty codec matching the expected regex during playback (Arvus / Sonos / Yamaha REST API); (2) PCM `hw_ptr` delta strictly positive across the playback window; (3) ALSA ELD demonstrating negotiated channel count + format; (4) `ffprobe-on-captured-mp4` with non-zero frames matching expected codec; (5) recording-analyzer matched event per §11.4.2 / §11.4.5 timeline; (6) `tinycap` RMS amplitude above floor.

Failure modes specifically forbidden: `arvus_probe_present` returning 1 silently dropping the test to SKIP; empty codec-state treated as evidence; `dumpsys media.audio_flinger` policy-side device field used as proof PCM actually opened; config-XML-only PASS without runtime evidence. All four are the §11.4 PASS-bluff

pattern materialised on D3 2026-05-20 when the test suite reported audio- routing PASS while the user heard nothing and Arvus Codec-In-Use was empty.

Mandatory protections: (1) sink-side library helpers expose `*_require_reachable` returning exit code 2 (OPERATOR-BLOCKED) when sink unreachable; (2) test harness propagates 2 to summary as OPERATOR-BLOCKED, release tags blocked while OPERATOR-BLOCKED exists; (3) audio-routing tests rewritten to capture ≥ 1 positive sink/downstream evidence per the enumeration above; (4) anti-stickiness post-stop — test re-probes sink and asserts codec-state transitioned to N.E. / 0-frames-delta. No escape hatch — no `--skip-sink-evidence`, `--allow-empty-codec`, `--sink-unreachable-is-pass`, `--metadata-only-suffices` flag.

Composes with §11.4.2 / §11.4.5 / §11.4.13 / §11.4.14 / §11.4.46 / §11.4.49 / §11.4.50 / §11.4.52. Pre-build gates `CM-COVENANT-114-68-PROPAGATION` + `CM-POSITIVE-SINK-EVIDENCE-PER-AUDIO-TEST` with paired meta-test mutations per §1.1.

Canonical authority: constitution submodule `Constitution.md` §11.4.68.

Non-compliance is a release blocker regardless of context.

§11.4.69 — Universal sink-side positive-evidence taxonomy + mechanical enforcement (User mandate, 2026-05-20)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

“THIS MUST HAPPEN NEVER AGAIN!!! We MUST HAVE this all working! Not just for audio but for every single piece of the System!!! Proper full automation when executed with success MUST MEAN that manual testing will be as much positive at least regarding the success results! ... Solution MUST BE universal, generic that solves working flows for all System components and for all future and all existing projects! ... Everything we do MUST BE validated and verified with rock-solid proofs and anti-bluff policy enforcement and fulfillment!”

Universal generalisation of §11.4.68 (audio-specific) across every user-visible feature class. Closes the PASS-bluff pattern where tests reported green while end users hit broken features (2026-05-19 → 20 D3 audio “82/84 PASS” + empty Arvus Codec-In-Use).

The mandate. Every user-visible feature MUST map to one entry in the closed-set §11.4.69 sink-side evidence taxonomy (audio_output, audio_input, video_display, network_throughput, network_connectivity, bluetooth_a2dp, bluetooth_pair, touch_input, sensor, gpu_render, storage_read, storage_write, mediacodec_decode, mediacodec_encode, miracast, cast, boot_service, package_install, permission_grant, wifi_link, wifi_throughput, ethernet_link, display_topology, drm_playback, subtitle_render — open to additions). Every PASS for a feature in the taxonomy MUST cite a captured-evidence artefact path matching the required evidence shape.

Helper contracts (additive during grace; mandatory after 2026-06-19):

- `ab_pass_with_evidence <description> <evidence_path>` — the new canonical PASS helper. Verifies path exists AND non-empty; emits `PASS: <description> [evidence: <path>]`.
- `ab_skip_with_reason <description> <closed-set-reason>` — reasons: `geo_restricted`, `operator_attended`, `hardware_not_present`, `topology_unsupported`, `network_unreachable_external`, `feature_disabled_by_config`. Forbids `network_unreachable_external` for any taxonomy feature with a sink-side probe.
- Bare `ab_pass` deprecated — WARN pre-grace, FAIL post-grace (2026-06-19).

Mechanical enforcement. Three pre-build gates + three paired §11.1 meta-test mutations:

- `CM-SINK-EVIDENCE-PER-FEATURE` — walks tests for `# §11.4.69 FEATURE: <class>` annotation + verifies taxonomy probe + `ab_pass_with_evidence` use.
- `CM-NO-FAIL-OPEN-SKIP` — audits sink-side probe helpers; FAILs if any code path converts empty/unreachable response to PASS-counting SKIP for a feature class with a sink-side probe.
- `CM-AB-PASS-WITH-EVIDENCE-EVERYWHERE` — pre-grace WARN, post-grace FAIL on bare `ab_pass` calls.

Composes with §11.4.1 (FAIL-bluffs forbidden), §11.4.2 (recorded-evidence), §11.4.5 (audio + video 5-layer quality), §11.4.6 (no-guessing), §11.4.13 (sink-side captured-evidence), §11.4.27 (no-fakes-beyond-unit), §11.4.50 (deterministic consistency), §11.4.52 (autonomous-validation), §11.4.68 (audio-specific sink-side — §11.4.69 is the universal generalisation).

No escape hatch — no `--skip-evidence` , `--config-only-pass` , `--allow-fail-open-skip` , `--legacy-ab-pass-permitted` flag. The discipline exists because the 2026-05-20 forensic incident demonstrated the failure: tests reported audio-routing PASS while the user heard nothing and the Arvus Codec-In-Use field was empty.

Propagation gate `CM-COVENANT-114-69-PROPAGATION` enforces this anchor literal across the ~44-file consumer fleet. Paired mutation strips the literal → gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.69.

Non-compliance is a release blocker regardless of context.

§11.4.70 — Subagent-driven execution is the default (User mandate, 2026-05-20)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

“Always do if possible Subagent-driven! Add this into our root (constitution Submodule) `Constitution.md`, `CLAUDE.md` and `AGENTS.md`. This should be the default choice ALWAYS!”

When executing implementation plans authored via `superpowers:writing-plans` (or any equivalent task-decomposed execution flow), the **default execution model is subagent-driven** per `superpowers:subagent-driven-development` . Inline execution via `superpowers:executing-plans` is permitted ONLY when (a) the task is trivial AND fits in a single sub-300-line edit, OR (b) the operator explicitly requests inline execution at brainstorm-handoff time.

Subagents bring an isolated context window per task (no conductor context bloat), a structurally separated review seam (conductor reviews subagent output, eliminating self-review blind spots), parallel-PWU compatibility (§11.4.58 — subagents ARE the parallel work units), and resumability across operator absence (subagents resume from on-disk plan + spec inputs).

Composes with §11.4.4 (four-layer coverage), §11.4.6 (no-guessing — subagent's captured output IS the evidence), §11.4.42 (iteration discipline), §11.4.43 (TDD-fix), §11.4.50 (deterministic consistency), §11.4.51 (LIVE_ADB_FIRST), §11.4.58 (parallel- development PWU).

No escape hatch — `--inline-execution-required`, `--no-subagents`, `--monolithic-execution` are NOT permitted flags. Skipping subagent-driven for non-trivial work without recorded operator authorisation is itself a §11.4 PASS-bluff. Pre-build gate `CM-COVENANT-114-70-SUBAGENT-DEFAULT-PROPAGATION` enforces this anchor literal across the ~44-file consumer fleet. Paired meta-test mutation strips the literal → gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.70.

Non-compliance is a release blocker regardless of context.

§11.4.71 — Pre-Push Fetch + Investigate + Integrate Mandate (User mandate, 2026-05-20)

Everyday-push variant of §11.4.41 (force-push merge-first). Before pushing to any upstream for any repository (main repo or submodule), the agent MUST follow the 5-step pre-push cycle: (1) `git fetch --all --prune --tags`, (2) `git pull --no-rebase <remote> <branch>` for each remote whose tip differs from local, (3) investigate the diff vs OUR previous HEAD by reading every foreign commit's body (what changed, why, how it affects OUR project), (4) integrate mandatory changes with full §11.4.4(b) four-layer anti-bluff test coverage producing REAL captured-evidence proofs, (5) THEN push to every configured remote in cascade order.

Composes with §11.4.41 (force-push case) + §11.4.26 / §11.4.32 / §11.4.37 / §11.4.40 / §11.4.42 / §11.4.43 / §11.4.4(b) / §11.4.5 / §11.4.6.

Applies to parent repo + constitution submodule + every owned submodule + every nested submodule + every HelixQA dependency. Audit-trail per push reconstructable from `docs/changelogs/<tag>.md` + per-repo `git log` evidence.

No escape hatch — no `--skip-fetch`, `--no-investigate`, `--fast-push`, `--trust-upstream` flag. Pre-build gate `CM-COVENANT-114-71-PROPAGATION` + paired mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.71.

Non-compliance is a release blocker regardless of context.

§11.4.72 — Audio Top-Priority Mandate (User mandate, 2026-05-20)

Direct user mandate (verbatim): “Make sure all fixes for audio are always top priority in main working stream!” The conductor (main working stream — Claude Code session, AI agent, or human operator) MUST treat audio fixes as the highest-priority class on the serial dispatch queue. Audio scope includes §EU HDMI rejection, §EM

multichannel HDMI, §ET Arvus integration, §EV/§EW/§EX D3 audio defects, HiFi (Fix #74, #76), AC3 (Fix #106), ES8388 (Fix #103/§104/#105), multichannel LPCM (Fix #112), and every future audio-stack improvement.

Composes with §11.4.42 (iteration-discipline — audio sits at apex of priority order) + §11.4.58 (parallel-development PWU — background research subagents run concurrently but do NOT preempt audio on the main-stream serial dispatch queue).

Operative rule: any time the conductor faces a choice between dispatching an audio task vs a non-audio task on the SAME serial resource, the audio task wins. No escape hatch — no “but this non-audio task is faster” or “but this research is more interesting” override. Audio-stack regressions are user-perceptible and high-impact (D3 silent post-flash is a release blocker), while research and refactors can wait.

Canonical authority: constitution submodule `Constitution.md` §11.4.72.

Non-compliance is a process violation regardless of context.

§11.4.73 — Main-specification document versioning + revision discipline (User mandate, 2026-05-20)

Direct user mandate (verbatim): “Make sure everything we add now in previous and upcoming requests IS ALWAYS applied to the main specification — if we have one. Since all these are not major changes we could increase Specification version per change for secondary version instead of the primary. Primary version MUST BE increased for much bigger levels of changes! Document MUST BE updated ALWAYS to follow the versioning rules we are applying here + revision and other properties we have!”

Applies only when a project recognises a main specification document (e.g. `docs/specs/**/specification.V<n>.md`). Two-axis versioning: **primary** (V1/V2/V3/...) bumps for major rewrites (old primary archived to `<spec-dir>/archive/`); **secondary** (`Revision`) bumps for additive operator requirements, refinements, polish (matches the §11.4.61 metadata-table `Revision` integer). Every operator-mandated requirement MUST land in the spec as part of the work that implements it. Cross-doc propagation copies MUST reference the active spec file, not a stale archived version. Composes with §11.4.44, §11.4.61, §11.4.59, §11.4.65.

Canonical authority: constitution submodule `Constitution.md` §11.4.73.

Non-compliance is a release blocker.

§11.4.74 — Submodule-catalogue-first discovery + extend-don't-reimplement (User mandate, 2026-05-20)

Direct user mandate (verbatim): “We MUST ALWAYS check which already developed features / functionalities do exist as a part of our comprehensive Submodules catalogue located in `vasic-digital` and `HelixDevelopment` organizations on GitHub and GitLab both! Project MUST BE aware of all its existence so we do not implement same things multiple times. For any missing features we MUST IMPLEMENT them properly and extend those Submodules further!”

Before scaffolding any new module / package / helper / utility, the contributor (human or AI agent) MUST: (1) survey the `vasic-digital` + `HelixDevelopment` orgs on GitHub + GitLab — the canonical inventory is `submodules-catalogue.md` (142 repos categorised); (2) reuse an existing Submodule when it covers $\geq 80\%$ of the functionality; (3) extend in-place via upstream PR when 80%+ matches but features are missing — never duplicate the 80% in-project; (4) document the survey result in the relevant tracker row with a `Catalogue-Check: reuse|extend|no-match <org/repo>@<sha>` line.

Every Submodule in the catalogue is subject to the same development-cycle rules (§11.4 anti-bluff, §1.1 paired mutations, §11.4.10 credentials, §11.4.61 metadata + ToC, §11.4.65 universal export, §11.4.73 spec versioning, §2.1 multi-mirror push, §3 propagation order).

Canonical authority: constitution submodule `Constitution.md` §11.4.74.

Non-compliance is a process violation; severe cases (duplicate implementation landed without catalogue check) are release blockers.

§11.4.75 — Mechanical Enforcement Without Exception (User mandate, 2026-05-20)

Direct user mandate (verbatim): “Why do these violations still happen!? This is a serious problem! We cannot rely on stability nor consistency if we cannot respect our Constitution, mandatory rules and constraints! Is there a way to make this always respected, followed and applied without exception fully and unconditionally!? WE MUST HAVE THIS WORKING FLAWLESSLY!!! Do investigate the root causes of such problems! Once all problems are identified WE MUST apply proper mechanisms for this not to happen NEVER EVER AGAIN!”

The §11.4 covenant historically relied on agent + operator vigilance. Three forensic incidents in 2026-05-19 → 20 (two stalled remediation subagents + post-§11.4.66 propagation drift requiring catch-up commit `f93b25a92eb`) demonstrated that late-binding enforcement at `pre_build_verification.sh` time fires hours-to-days AFTER the violator commit has already reached every remote. §11.4.75 closes the gap with FIVE independent mechanical enforcement layers — bypassing any single layer does not bypass the discipline:

1. **Local pre-commit git hook** — refuses staged `.md` lacking sibling `.html` + `.pdf` (and other staged-only invariants).
2. **commit_all.sh integration** — the canonical project commit script invokes the same checks + auto-runs `sync_all_markdown_exports.sh` to self-repair before commit (`_constitution_sibling_check` function).
3. **Local pre-push git hook** — re-runs siblings + propagation-gate subset on every commit in the push.
4. **post-commit auto-repair hook** — detects orphan `.md` in just-committed manifest, auto-generates siblings via pandoc + weasyprint, creates a `chore(§11.4.75): auto-export ...` follow-up commit. Idempotent + recursion-guarded.
5. **Local-only equivalent** (Phase 39.GF, User mandate 2026-05-20). Remote CI surfaces (GitHub Actions, GitLab pipelines, Jenkins, CircleCI, etc.) are DISABLED — the workflow file is preserved at `.github/workflows/constitution-compliance.yml.disabled-local-only` (NOT `.yml` ; GitHub Actions ignores it). Layer 5 enforcement migrated to the LOCAL pre-build-verification + meta-test ritual the operator MUST execute before tagging per §11.4.40. Layers 1-4 remain authoritative; Layer 5 is the operator's local final gate. A future re-enable PWU may re-establish remote CI.

Helper contracts (mandatory): `scripts/install_git_hooks.sh` (idempotent installer wired into `scripts/setup.sh`), `scripts/git_hooks/{pre-commit,pre-push,post-commit,commit-msg}`, `_constitution_sibling_check` in `scripts/commit_all.sh`. The `commit-msg` hook enforces a Bypass-rationale: `<reason>` footer when `--no-verify` is detected (touch-file marker `.git/ATMO_LAST_BYPASS_ATTEMPT`); `docs/audit/bypass_events.md` accumulates the audit trail per §11.4 captured-evidence requirement.

Five pre-build gates with paired §1.1 meta-test mutations: `CM-COVENANT-114-75-PROPAGATION` (anchor literal across canonical files), `CM-GIT-HOOKS-INSTALL-`

SCRIPT (installer present + executable), CM-GIT-HOOKS-SOURCE-DIR (4 hook bodies present + executable), CM-COMMIT-ALL-SIBLING-CHECK (`_constitution_sibling_check` in `commit_all.sh`), CM-CI-WORKFLOW-PRESENT (CI workflow + ≥ 3 required jobs).

Composes with §1.1 (paired meta-test mutations), §9 (data safety), §11.4 (end-user-quality covenant — Layer 5 CI makes the §11.4 promise mechanical), §11.4.41 (this anchor's renumber from §11.4.74 → §11.4.75 was itself driven by §11.4.41 merge-first discipline), §11.4.65 (universal Markdown export — Layer 1+3+4 are §11.4.65's mechanical seam), §11.4.66 (interactive clarification), §11.4.67 (target-shell-parseability — hooks parse under bash AND mksh), §11.4.71 (pre-push fetch + integrate — Layer 3 + 5 honour the merge-first pipeline), §11.4.72 (audio top-priority — hooks hold no lock themselves, so do not race against in-flight audio commits), §11.4.73 (main-spec versioning — when spec exists, hooks include it), §11.4.74 (submodule-catalogue-first — hooks can be added to the canonical catalogue for cross-project reuse).

No escape hatch — no `--skip-hooks` , `--bypass-enforcement` , `--allow-orphan-md` , `--ci-not-applicable` , `--mechanical-enforcement-not-needed` flag exists. The `--no-verify` route IS the deliberate audit-trail bypass; §11.4.75 makes the audit trail mechanical via the `commit-msg` footer requirement.

Canonical authority: constitution submodule `Constitution.md` §11.4.75.

Non-compliance is a release blocker regardless of context.

§11.4.76 — Containers-submodule mandate (User mandate, 2026-05-20)

Direct user mandate (verbatim): “For any work or requirements of running services or codebase inside the Containers (Docker / Podman / Qemu / Emulators, and so on) we MUST USE / INCORPORATE the Containers Submodule properly:

`https://github.com/vasic-digital/containers` (`git@github.com:vasic-digital/containers.git`). Containers Submodule contains all means for us to Containerize our code and services! If any feature or Containing System is missing or not supported we MUST EXTEND IT properly like we do all of our projects! No bluff work is allowed of any kind!”

§-slot history note. Originally drafted as §11.4.75, renumbered to §11.4.76 per §11.4.71 fetch-before-push after the concurrent `0a70083` landing of §11.4.75 (Mechanical Enforcement). Same collision-resolution pattern as §11.4.75's own renumber from drafted §11.4.74.

For ANY containerized workload (Docker / Podman / Qemu / Kubernetes / container-backed emulators), every consuming project MUST: (1) install `vasic-digital/containers` (`digital.vasic.containers`) as a Git submodule; (2) consume via `replace` directive during development + pinned commit SHAs in production; (3) boot infra on-demand via `pkg/boot` + `pkg/compose` + `pkg/health` so operators are never required to start `podman machine` / `docker compose up` manually — the boot is part of the test entry point (the **on-demand-infra invariant**); (4) extend the Submodule (PR upstream) for missing runtimes / lifecycle primitives — never reimplement in-project (per §11.4.74); (5) anti-bluff: integration tests claiming to exercise containerized components MUST actually boot them via the Submodule — short-circuit fakes that bypass boot are a §11.4 violation.

Tracker rows touching containerization MUST record `Catalogue-Check: extend vasic-digital/containers@<sha> (or reuse ); no-match requires` demonstrating the Submodule cannot model the workload.

Planned anti-bluff gate `CM-CONTAINERS-USED` scans container-touching PRs for `digital.vasic.containers/...` imports. Paired mutation strips the import + asserts FAIL.

Composes with §11.4.74 (catalogue-first), §11.4.75 (mechanical enforcement — this clause IS one of the things mechanical enforcement protects), §3 (propagation), §11.4.31 (Submodule-Dependency-Manifest), §11.4.36 (`install_upstreams` on clone), §11.4.28 (Submodules-As-Equal-Codebase), §1.1 (paired-mutation gates protect the Submodule's own primitives).

Canonical authority: constitution submodule `Constitution.md` §11.4.76.

Non-compliance: reinventing compose orchestration in-project is a release blocker.

§11.4.77 — Regeneration-mechanism-required mandate (User mandate, 2026-05-20)

Direct user mandate (verbatim): “We must be sure that after excluding anything from Git versioning we still have the mechanism which will out of the box obtain or re-generate missing content!”

Forensic anchor. 2026-05-20T15:00Z: a consuming project's audio Tier 1 `commit_all.sh` stalled 4 h on `git add -A` scanning 274 GiB `.git-backup-*` + 159 GiB `RKTools/linux/` + 167 GiB `qa-results/` — all untracked but un-

gitignored. Bare `.gitignore` fix would orphan every fresh clone (missing RKTools, missing test infra, missing build outputs).

Every `.gitignore` entry excluding (a) >~100 MiB OR (b) any artefact essential to building / running / testing the project MUST carry a documented + automated mechanism to either **re-obtain** (download from authoritative source: vendor tarball, SDK installer, npm / pip / cargo / go-mod / container registry, dedicated git submodule, S3/GCS) OR **re-generate** (run from tracked source code via build pipeline, code-gen, asset render, captured-evidence replay, container build, kernel build). Required artefacts per qualifying entry: (1) `.gitignore-meta/<entry-slug>.yaml` declaring pattern + mechanism-type + script-path + expected-disk-usage + vendor-url-or-source + integrity hash + requires-network + requires-credentials; (2) entry in `scripts/setup.sh` (post-clone bootstrap) that runs the mechanism non-interactively; (3) pre-build gate verifying regenerated content present OR stamp `.gitignore-meta/.regenerated/<slug>.ok` recent; (4) README + relevant `docs/guides/*.md` describing the mechanism + manual fallback + time/disk budget + per-§11.4.10 credentials.

No escape hatch: bare `.gitignore` additions without the mechanism are themselves a §11.4 PASS-bluff variant — codebase appears complete to casual eye but fresh clone cannot build / run. No `--skip-regen-mechanism`, `--gitignore-is-enough`, `--operator-already-has-content` flag.

Planned anti-bluff gate `CM-GITIGNORE-REGEN-MECHANISM` scans every `.gitignore` addition for matching `.gitignore-meta/` sibling + verifies `script-path` exists + parses YAML + checks bootstrap reference. Paired §1.1 mutation strips one required YAML key → gate FAILs.

Composes with §11.4.6 (no-guessing — mechanism MUST be verified working on sandbox clone, not assumed), §11.4.65 (universal Markdown export — generated HTML/PDF siblings are a `re-generate` instance), §11.4.66 (interactive clarification — ASK operator if mechanism unknown), §11.4.71 (pre-push fetch + integrate — re-validate vendor integrity before push), §11.4.74 (catalogue-first — extend a reusable downloader Submodule rather than reimplement), §11.4.75 (Mechanical Enforcement — pre-commit + CI replay layers refuse bare additions), §11.4.76 (Containers-submodule — container images regenerated via `vasic-digital/containers`), §9 / §9.2 (zero-risk data safety — pre-test mechanism in sandbox with hardlinked-backup), §3 (propagation — consuming submodules inherit §11.4.77).

Canonical authority: constitution submodule `Constitution.md` §11.4.77.

Non-compliance is a release blocker regardless of context.

§11.4.78 — CodeGraph code-intelligence mandate (User mandate, 2026-05-20)

Direct user mandate (verbatim): “Make codegraph MANDATORY CHOICE for this purpose for all of our project ... All project which do not have configured and installed codegraph yet MUST DO IT and MUST USE IT!”

Every consuming project worked on by AI coding agents MUST install, initialize, and use **CodeGraph** (`https://github.com/colbymchenry/codegraph` , `npm`

`@colbymchenry/codegraph`) — a local SQLite semantic code-knowledge-graph exposed to agents over MCP (100% local, no cloud, no external API). (1) Install globally via `npm` — no `sudo` (the `npm` prefix MUST be user-writable). (2)

`codegraph init` + `codegraph index` : `.codegraph/config.json` is tracked, `.codegraph/codegraph.db` is gitignored with `codegraph index` as its §11.4.77 regeneration mechanism; the `config.json` `exclude` list MUST exclude — non-negotiably — every credential/secret path per §11.4.10 (a secret reaching the index is a §11.4.10 violation). **CRITICAL — CodeGraph filter mechanism:** CodeGraph v1.x is zero-config. It does NOT use `config.json` `include` / `exclude` fields for file discovery. Filtering is done by a built-in ignore matcher seeded with defaults (`node_modules` , `vendor` , `dist` , `build` , `target` , `.venv` , `Pods` , `.next`) PLUS the project root `.gitignore` . To control indexing, use root-anchored `.gitignore` patterns with `!` negations for re-inclusion under default-skipped parents (e.g., `!/vendor/widevine_*` re-includes owned vendor code). **ALL**

SOURCE CODE MUST BE INDEXABLE — including AOSP platform trees (`frameworks/` , `packages/` , `external/` , `art/` , `bionic/` , `cts/` , `kernel-5.10/` , `hardware/*/` , `vendor/` , `device/*/`). Only actual rubbish MAY be excluded: build outputs (`/out/` , `/prebuilts/` , `/rockdev/`), caches (`.gradle/` , `__pycache__`), credentials/secrets (`/secrets/` , `/qa-results/` , `/recordings/`). A forked-AOSP project with 1M+ tracked files will produce a correspondingly large index (30-40 GB for first init), which is expected and correct — the index must represent the full codebase the AI agent cross-references. (3)

Wire the `codegraph serve --mcp` server into every CLI agent the developers use — project-scoped + committed where supported (Claude Code `.mcp.json` , OpenCode `opencode.json` , Qwen Code `.qwen/settings.json` , Crush `.crush.json`), host-local otherwise (Kimi CLI `~/.kimi/mcp.json`); every config references the bare `codegraph` command on `PATH` (no hardcoded host path). (4) Cover the integration with an anti-bluff verification suite whose per-agent end-to-end

layer uses an **unforgeable challenge** — a fact obtainable only by calling a CodeGraph MCP tool (e.g. the index node count via `codegraph_status`) so an agent answering from its own file-reading tools cannot produce a false PASS; an agent that genuinely cannot be driven end-to-end (missing credentials / quota / environment incompatibility) is a documented SKIP gap per §11.4.3, never a faked PASS. (5) Document everything in `docs/CODEGRAPH.md` , kept in sync per §11.4.12 / §11.4.65. CodeGraph is consumed as the published npm package (§11.4.74) — not added as a git submodule, and it adds no Git remote.

Composes with §11.4.3, §11.4.10, §11.4.12, §11.4.65, §11.4.30, §11.4.74, §11.4.77, §11.4 (anti-bluff — CI green is necessary, never sufficient), §1.1. Planned gate `CM-CODEGRAPH-WIRED` + paired §1.1 mutation (strip a secret-exclusion from `config.json` → gate FAILs).

Canonical authority: constitution submodule `Constitution.md` §11.4.78.

Non-compliance is a process violation; a project worked on by AI agents without CodeGraph installed, wired, and anti-bluff-verified is in breach of this mandate.

§11.4.79 — Own-org submodules MUST be included in the CodeGraph index (User mandate, 2026-05-21)

Direct user mandate (verbatim, 2026-05-21): “All Submodules we use in the project and that are part of organizations to which we have the full access via GitHub, GitLab and other CLIs MUST BE included into the codegraph database and initialized / scanned / synced!”

Refines §11.4.78 step 2’s exclude-list with a per-submodule-ownership split: (a)

Own-org submodules — full write access via the project’s CLIs (canonical orgs:

`vasic-digital` on GitHub/GitLab/GitFlic/GitVerse + `HelixDevelopment` on

GitHub) — MUST be **INCLUDED** in the index. (b) **Third-party submodules** (the

§11.4.74 `no-match` → `vendor` path, e.g. `gopkg.in/telebot.v3`) — MUST be

EXCLUDED. Operational steps: (1) `git submodule update --remote --merge`

to pull latest from every submodule before re-indexing — respect load-bearing pins

on third-party submodules (e.g. roll back if `--remote` advances `gopkg.in/`

`telebot.v3` past the v3 import-path pin). (2) Adjust `.codegraph/config.json`

exclude list to keep own-org paths in scope. (3) Re-index via `scripts/`

`codegraph_setup.sh` . (4) Verify via `scripts/codegraph_validate.sh` with at

least one probe that resolves a symbol living ONLY inside an own-org submodule.

(5) Paired §1.1 mutation: temporarily add the own-org submodule to exclude → validate MUST FAIL on the cross-submodule probe → restore.

Composes with §11.4.74 (catalogue ownership classifier), §11.4.78 (CodeGraph parent mandate), §11.4.10 (credentials still excluded regardless), §1.1 (paired mutation), §107 (an index that lies about reachable symbols is a §107 PASS-bluff against AI agents).

Canonical authority: constitution submodule `Constitution.md` §11.4.79.

Non-compliance is a process violation; severe cases (own-org submodules silently excluded WITHOUT an audit trail in `.codegraph/config.json` comments) are release blockers.

§11.4.80 — CodeGraph regular-update + sync automation mandate (User mandate, 2026-05-21)

Direct user mandate (verbatim, 2026-05-21): “We MUST regularly check for the updates and execute codegraph npm updates so the latest version of it is always installed on the host machine! [...] Make sure we have proper full automation bash scripts which will run regularly and that these are part of the constitution Submodule since they MUST BE available to all projects which do respect and follow our constitution rules and mandatory constraints! Make sure all updates, sync processes we do and important codegraph related events are all documented under docs/codegraph in Status and Status_Summary documents (another area / context to regularly update and sync) and regularly export them like all other Status docs into the PDF and HTML!”

Three deliverables (all in this constitution submodule): (1) `scripts/codegraph_update.sh` — npm-installs latest `@colbymchenry/codegraph` after npm-registry version check; appends old/new version to `docs/codegraph/Status.md` ; §107 anti-bluff: verifies `codegraph --version` reflects the new version after install (npm exit 0 ≠ working binary). (2) `scripts/codegraph_sync.sh` — after a successful update, runs `codegraph status` → `codegraph sync .` → `codegraph status` → project's `scripts/codegraph_validate.sh` in the consuming project; appends every step's output to BOTH the project's and the constitution's `docs/codegraph/Status.md` . (3) `docs/codegraph/Status.md` + `Status_Summary.md` append-only ledgers tracking all CodeGraph-related events; exported to `.html` + `.pdf` siblings per §11.4.65.

Operational cadence: weekly floor (per §11.4.45 status-digest cadence). Scripts are inherited by reference (per §3 submodule inheritance) — consuming projects invoke them at `${CONST_DIR}/scripts/codegraph_*.sh`, never copy.

Composes with §11.4.78 (CodeGraph parent), §11.4.79 (own-org submodule inclusion), §11.4.10 (credentials excluded), §11.4.45 (status-digest cadence), §11.4.53 (Fixed_Summary backfill), §11.4.65 (multi-format export), §107 (observed-version evidence per update; observed PASS/FAIL per sync), §1.1 (paired mutation: downgrade installed version → script detects drift → restore).

Canonical authority: constitution submodule `Constitution.md` §11.4.80.

Non-compliance is a process violation; severe cases (consuming project has not run `codegraph_update.sh` in >2 weeks AND has open AI-agent work) are release blockers.

§11.4.81 — Cross-platform-parity mandate (User mandate, 2026-05-21)

Direct user mandate (verbatim, 2026-05-21): “Any Linux-only blocker / issue we have MUST BE created macOS and other supported platforms equivalent! So, depending on platform proper implementation will be used for particular OS! EVERYTHING MUST BE PROPERLY EXTENDED AND UPDATED!”

Every consuming project whose supported-platforms manifest lists more than one OS MUST, for every feature/test/gate/challenge/mutation that depends on platform-specific primitives, ship a per-OS-equivalent implementation chosen at runtime via `uname -s` (or equivalent platform detection). Three sub-mandates: **(A) Per-OS implementation REQUIRED.** Linux `cgroup/systemd/ /proc` primitives MUST have documented per-OS equivalents (POSIX `setrlimit / ulimit`, macOS `launchd`, BSD `rctl`, Windows Job Object) chosen via runtime dispatch — same operator-path invocation, OS-correct mechanism. **(B) Per-OS tests REQUIRED.** Every gate test that exercises platform-dependent behaviour MUST have `case "$(uname -s)" in` branches with positive captured evidence per §11.4.2 + §11.4.5 in each branch. SKIP-with-reason is acceptable ONLY when the platform genuinely cannot enforce the invariant. **(C) Honest kernel-gap citation + adjacent equivalent test REQUIRED.** Where a Linux primitive has NO macOS/other-OS equivalent due to a documented kernel limitation (canonical: XNU does NOT enforce `RLIMIT_AS` for unprivileged processes), the test MUST detect the gap at runtime, SKIP with exact kernel reason + reproducer + link to honest-gap doc, AND provide an ADJACENT test exercising the closest invariant the platform CAN

enforce (e.g. `RLIMIT_CPU + SIGXCPU` as the macOS proxy for “process is bounded”). The adjacent test MUST itself be anti-bluff with a paired §1.1 mutation.

Per-OS equivalence catalogue (canonical, not exhaustive): `systemd-run --user --scope` ↔ POSIX `ulimit -t -u` / `launchd`; `cgroup MemoryMax` ↔ XNU gap (use `RLIMIT_CPU` adjacent) / `rctl` / Job Object; `cgroup TasksMax` ↔ `RLIMIT_NPROC` ; `/proc/<pid>/oom_score_adj` ↔ no equivalent on Darwin/BSD (no kernel OOM-killer).

Composes with §11.4.1 (FAIL-bluffs forbidden — Linux-PASS / Darwin-SKIP without honest reason is a §11.4.81 violation), §11.4.2 (recorded evidence per branch), §11.4.3 (§11.4.81 strictens §11.4.3: topology SKIP only when kernel CANNOT, not when “we didn’t implement yet”), §11.4.4 (test-interrupt triggers when discovering a Linux-only test without per-OS equivalent), §11.4.5 (captured evidence per platform), §11.4.6 (no guessing about platform capability — prove via reproducer), §11.4.20 / §11.4.70 (per-OS branches are parallel-subagent dispatchable), §11.4.27 (100% test-type coverage applies per-platform), §11.4.69 (sink-side evidence per platform), §107 (multi-platform projects must guarantee end-user usability on every platform). Pre-build gate `CM-CROSS-PLATFORM-PARITY` (when implemented): scans every gate test for `case "$(uname -s)"` blocks, asserts a non-SKIP branch (or honest-gap citation per (C)) exists for each platform in the supported-platforms manifest. Paired §1.1 mutation: strip a Darwin branch → gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.81.

Non-compliance is a release blocker on multi-platform projects. No escape hatch.

§11.4.82 — Iteration-speedup discipline mandate (User mandate, 2026-05-22)

Forensic anchor — direct user mandate (verbatim, 2026-05-22): “How can we speed-up this whole development and fixing process? ... Do not forget to all speed optimizations critical rules and mandatory constraints MUST BE all added into our root (constitution Submodule) `Constitution.md`, `CLAUDE.md`, `AGENTS.md` and `QWEN.md` and all other relevant constitution Submodules files!”

Iteration cycle time is a first-order quality enabler. Slow cycles bound the project’s defect-discovery rate; fast cycles multiply it. The 2026-05-22 forensic session witnessed ~3 hours of operator wait time on a job whose intrinsic compute was ~90 min — every wasted minute came from a missing speedup discipline.

Every consuming project's build / test / commit / debug pipeline MUST adopt the following speedup disciplines AS MANDATORY (each independently enforceable):

- **(A) Phase 1 forensic before any speculative source patch** — before any non-trivial source patch, complete `superpowers:systematic-debugging` Phase 1 to identify FACT-grade root cause. Speculative patches without Phase 1 evidence are §11.4.6 + §11.4.82 violations.
- **(B) Live-ADB-First (or live-equivalent) before any rebuild** — strengthens §11.4.51 to a release-blocker mandate. Skipping live-probe for a `LIVE_ADB_TESTABLE` change is a §11.4.82 violation.
- **(C) Pre-flight before launching rebuild orchestrators** — 30-second pre-flight verifies device reachability, sink reachability, host memory/disk, no stale locks, no orphan processes. A rebuild against a broken precondition wastes 45 min.
- **(D) Persistent build caches outside containers** — `ccache` + Soong / `sccache` / Gradle daemon state bind-mounted to host, NOT in ephemeral container layer. Cold rebuild ~40 min → warm rebuild 5-15 min.
- **(E) Module-only rebuild for loadable-module-only changes** — `make -C kernel M=<driver-path> modules` for `CONFIG_*=m` drivers (~2-5 min). Full rebuild required only for `=y` built-ins.
- **(F) Parallel multi-device testing** — every owned validation device runs the autonomous cycle concurrently with separate `qa-results/<TS>/<device-tag>/` outputs.
- **(G) Subagent scope discipline + worktree isolation** — subagents ≤30 min budget, single-responsibility, intermediate output emitted as they progress, `isolation: "worktree"` by default.
- **(H) Lock-file + stale-process hygiene** — detect and clean `.git/index.lock`, `.lock` files, orphan `git` /build processes on session start. Disable auto git-gc (`gc.auto 0`) in concurrent multi-agent repos.
- **(I) Cycle telemetry per §11.4.24** — every iteration logs commit hash, per-phase wall-clock, speedup-discipline flag set, outcome. Aggregated weekly to surface which disciplines deliver the biggest empirical return.

Composes with §11.4.4, §11.4.6, §11.4.9, §11.4.20 / §11.4.70, §11.4.24, §11.4.42, §11.4.43, §11.4.50, §11.4.51, §11.4.52, §11.4.58, §12.7, §107. Pre-build gate `CM-ITERATION-SPEEDUP-DISCIPLINE` (when implemented): audits the most recent N cycles for telemetry citing which of (A)-(I) applied. Paired §1.1 mutation: strip the speedup-flag column → gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.82.

Non-compliance is a release blocker. No escape hatch — no `--skip-phase1-forensic`, `--no-pre-flight`, `--rebuild-everything-always`, `--unlimited-subagent-scope`, `--ignore-locks`, `--no-telemetry` flag exists.

§11.4.83 — docs/qa/ end-user evidence mandate (User mandate, 2026-05-22)

Forensic anchor — verbatim user mandate (2026-05-22):

“every feature that ships MUST carry a recorded e2e communication transcript + any attached materials under `docs/qa/<run-id>/` (per-feature subdirectories). A feature with no QA transcript is itself a §107 PASS-bluff — it claims to work but has no auditable runtime evidence. Bot-driven automation MUST preserve full bidirectional communication threads as proof.”

Every feature that ships MUST carry a recorded end-to-end communication transcript plus any attached materials (screenshots, request/response payloads, audio, file uploads) committed under `docs/qa/<run-id>/` — one directory per feature run. A feature with no QA transcript is itself a §11.4 / §107 PASS-bluff: it claims to work but has no auditable runtime evidence that an end user actually exercised the feature through the same interface they will use in production.

Operative rule. (1) Every consuming project MUST maintain a `docs/qa/` tree. Each new feature lands under `docs/qa/<run-id>/` where `<run-id>` is monotonic + greppable (timestamp, HRD-NNN, ATM-NNN, other workable-item ID per §11.4.54). (2) Transcripts MUST be full bidirectional — every prompt/command sent + every response received. One-sided is not a transcript. (3) Attached materials MUST be committed in-repo (no external-only links — that is §11.4.13 sink-side violation). (4) Bot-driven / agent-driven QA automation MUST preserve the full conversation thread as the proof artefact. (5) CI / release gates MUST refuse to tag a version that has any feature-shipping commit without its matching `docs/qa/<run-id>/` directory.

Composes with §11.4.2, §11.4.5, §11.4.13, §11.4.65, §11.4.69, §107, §1.1.

Canonical authority: constitution submodule `Constitution.md` §11.4.83.

Non-compliance is a release blocker. No `--qa-evidence-optional` escape hatch.

§11.4.84 — Working-tree quiescence rule for subagent commits (User mandate, 2026-05-22)

Short tag: `working-tree quiescence` .

Forensic anchor — verbatim user mandate (2026-05-22):

“no subagent commit may proceed while any concurrent mutation gate is in flight in the same checkout. Before `git add` , the committing agent MUST `grep` its own working tree for mutation markers (`MUTATED` for `paired` , `// always pass` , `return json.Marshal` shortcut paths, etc.). Any unexplained file in the staging area triggers ABORT.”

No subagent (or main-thread) commit may proceed while any concurrent mutation gate, paired-mutation experiment, or other in-flight mutation is live in the same checkout. Before `git add` , the committing agent MUST `grep` its own working tree for mutation markers (`MUTATED` for `paired` , `// always pass` , `return json.Marshal` shortcut paths, `// MUTATION / # MUTATION` annotations, `_mutated_*` filename suffixes, etc.) and explicitly account for every modified file in the staging area. Any unexplained file → ABORT.

Lesson (forensic case study). A consuming project’s logo-fix subagent (Herald commit `72e81ab` , 2026-05-21) ran in a checkout where a paired §1.1 mutation gate had temporarily introduced an `// always pass` shortcut into a JWT verify path. The subagent’s `git add` swept the mutation residue into the same commit as the unrelated logo fix, and the resulting commit was pushed to all four mirrors before any other agent caught it. The fix (Herald `d5bd360` , “SECURITY FIX: restore commons_auth/middleware.go JWT verify”) landed within the hour, but the window during which production-equivalent binaries shipped with a bypassed JWT verify is a real security-defect window. The lesson is now constitutional.

Operative rule. (1) Pre- `git add` MUST `grep` for mutation markers + cross-check `git status --porcelain` against the subagent’s declared scope; unaccounted entries → ABORT. (2) Any active mutation gate MUST be serialised — `mutate` → `assert FAIL` → `restore` → `assert PASS` — and the working tree MUST be verifiably clean BEFORE any unrelated commit. (3) Concurrent subagents in the SAME

checkout MUST coordinate through a lockfile (`.git/MUTATION_IN_PROGRESS`); the cleaner solution is `git worktree add` per subagent (composes with §11.4.20/§11.4.70). (4) Post-commit `mutation-residue-scanner` MUST run before push; any commit containing a mutation marker → push BLOCKED.

Composes with §1.1, §11.4.20, §11.4.70, §11.4.27, §11.4.10, §11.4.71, §107.

Canonical authority: constitution submodule `Constitution.md` §11.4.84.

Non-compliance is a release blocker. A mutation marker that lands in a tagged commit is a critical defect regardless of how briefly it persisted.

§11.4.85 — Stress + Chaos Test Mandate (User mandate, 2026-05-24)

Short tag: `stress-chaos-mandate` .

Forensic anchor — verbatim user mandate (2026-05-24):

“Every fix or improvement you do MUST BE covered with full automation stress and chaos tests so we are sure nothing can break the functionality and all edge cases are monitored and polished and additionally fixed if that is needed! Everything must produce rock solid proofs and follow fully no-bluff policy!”

Every fix or improvement landed in a consuming project MUST ship with full-automation **stress** AND **chaos** test suites that exercise edge cases, sustained load, concurrent contention, and failure-injection. Happy-path coverage alone is a §11.4 / §107 PASS-bluff at the resilience layer.

Stress (closed-set, mechanically auditable): sustained load ($N \geq 100$ iterations OR ≥ 30 s wall-clock, per-iteration latency p50/p95/p99 recorded) + concurrent contention ($N \geq 10$ parallel invocations, no deadlock, no resource leak) + boundary conditions (empty / max / off-by-one input — every boundary produces a categorised result).

Chaos (closed-set, mechanically auditable, applied per fix-class appropriateness): process-death injection (kill primary or upstream mid-call, categorised recovery) + network-fault injection (drop/delay/reorder, `category=network|upstream` per §11.4.69) + input-corruption injection (corrupt `.env` / config / input file mid-test,

detected + reported) + resource-exhaustion injection (disk full, OOM, FD exhaustion — refuse cleanly OR degrade gracefully, NEVER crash) + state-corruption injection (mid-flight lock loss, partial-write fault — recovery restores consistent state).

Anti-bluff (mandatory). Every stress + chaos test PASS MUST cite a captured-evidence artefact path per §11.4.5 + §11.4.69 (per-iteration `latency.json` , `categorised_errors.txt` , `state_delta_snapshot.json` , `recovery_trace.log`). Helper library `stress_chaos.sh` provides `ab_stress_run` , `ab_stress_concurrent` , `ab_chaos_kill_pid_during` , `ab_chaos_drop_network_during` , `ab_chaos_corrupt_file_during` , `ab_chaos_oom_pressure_during` , `ab_chaos_disk_full_during` , each composing with `ab_pass_with_evidence` / `ab_skip_with_reason` per §11.4.69. Chaos-injection cleanup is non-negotiable — corrupt-restore, disk-fill-cleanup, process-restart MUST run in `trap '...' EXIT` ; cleanup failure = §11.4.14 violation.

4-layer coverage per §11.4.4(b): pre-build gate (test files exist + executable + `sh -n` + `bash -n` parseable; library exists; fix's pre-build gate cites the stress+chaos test path) + paired meta-test mutation per §1.1 (strip chaos-injection or evidence-capture → gate FAILs) + on-device test (if `LIVE_ADB_TESTABLE` per §11.4.51, dispatched against a real device with captured evidence under `qa-results/<run-id>/stress_chaos/`) + HelixQA Challenge entry (if user-visible feature per §11.4.4(b) layer 4).

Composes with §11.4 / §107 (resilience IS end-user quality), §11.4.1 (FAIL-bluffs forbidden — set-u crashes from chaos step are bluffs), §11.4.5 (captured-evidence content quality applies to latency distribution + error categories), §11.4.6 (no guessing — categorised errors only), §11.4.43 (TDD RED-first under load/chaos), §11.4.50 (N iterations produce identical exit + identical evidence-hashes), §11.4.52 (autonomous validation), §11.4.69 (universal sink-side positive-evidence taxonomy), §11.4.83 (recovery transcripts ARE end-user-channel proofs).

Canonical authority: constitution submodule `Constitution.md` §11.4.85.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-stress` , `--no-chaos` , `--happy-path-suffices` , `--stress-test-later` flag exists.

§11.4.86 — Roster/corpus-backed Status-doc auto-sync mandate (User mandate, 2026-05-25)

Forensic anchor — verbatim user mandate (2026-05-25):

“Make sure that assets and players Status docs are ALWAYS regularly updated and in sync like all others Status docs — any time we add or modify the assets content(s) or we change or add new / remove existing pre-installed video and audio player apps! This MUST WORK OUT OF THE BOX!”

Some Status docs (§11.4.45) are backed by a **tracked roster** (installed apps/components) or a **tracked asset corpus** (test/media asset directory) rather than narrative alone. Their freshness MUST NOT depend on operator vigilance — the moment a roster/corpus member changes (player app added/removed/renamed; asset added/modified/removed) the Status doc + Status_Summary + HTML + PDF MUST resync **out of the box**, mechanically.

Mechanism (all must hold): (1) **drift-proof fingerprint** — sha256 of the sorted member list (NOT mtime), persisted in a sidecar beside the Status doc; (2) **sync helper** that regenerates the fingerprint + re-exports HTML+PDF (via the §11.4.65 exporter), wired so sync is automatic; (3) **pre-build gate** that FAILs when the live fingerprint differs from the persisted one (mirrors §11.4.12 `CM-ISSUES-SUMMARY-SYNC` + §11.4.45 `sync_integration_status`); (4) **paired §1.1 mutation** that corrupts the fingerprint and asserts the gate FAILs.

Composes with §11.4.12 / §11.4.45 (roster/corpus specialisation) / §11.4.53 / §11.4.56 / §11.4.57 / §11.4.59 / §11.4.60 / §11.4.65 / §11.4.6. Classification: universal (§11.4.17) — the consuming project supplies the specific docs, roster/corpus sources, helper, and gate name per §11.4.35.

Canonical authority: constitution submodule `Constitution.md` §11.4.86.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-roster-sync`, `--allow-status-drift`, `--roster-sync-not-applicable` flag exists.

§11.4.87 — Endless-loop autonomous work + zero-idle agent dispatch + anti-bluff testing mandate (User mandate, 2026-05-26)

When the operator instructs an AI agent to “continue in endless loop fully autonomously” (or any semantically-equivalent phrasing), the agent MUST treat this as a HARD-CONTRACT covenant: (A) continue working until `docs/Issues.md` Status-column has zero non-terminal entries AND `docs/CONTINUATION.md` §3 Active work is empty AND no background subagent is mid-execution AND no external dependency is in-flight; (B) dispatch background subagents for parallelisable work — main + every subagent operate concurrently, “waiting for results” is the ONLY acceptable idle reason; (C) every closure lands four-layer test coverage per §11.4.4(b) with captured-evidence (audio/video/network/UI/sysfs — “physical proofs”); (D) the §11.4 anti-bluff covenant family (§11.4.1/§11.4.2/§11.4.6/§11.4.7/§11.4.27/§11.4.50/§11.4.52/§11.4.68/§11.4.69/§11.4.83) is the operative truth-discipline — tests AND HelixQA Challenges bound equally per the historical forensic anchor “we had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can’t be used”; (E) loop terminates ONLY on all-conditions-met, explicit operator STOP, host-session-safety demand, or scheduled wake on a known-future-actionable signal.

Composes with §11.4 / §11.4.1 / §11.4.2 / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.7 / §11.4.20 / §11.4.27 / §11.4.42 / §11.4.43 / §11.4.50 / §11.4.52 / §11.4.58 / §11.4.68 / §11.4.69 / §11.4.70 / §11.4.83 / §11.4.85 / §11.4.86 / §12.10. Pre-build gate `CM-COVENANT-114-87-PROPAGATION` + paired §1.1 meta-test mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.87.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--idle-OK`, `--skip-endless-loop`, `--bluff-permitted-for-this-task`, `--metadata-only-test-suffices`, `--no-physical-proof-required` flag exists.

§11.4.88 — Background-push mandate: commit-lock release immediately after commit, push runs detached (User mandate, 2026-05-26)

Forensic anchor: 2026-05-26 a single `commit_all.sh` held its flock ~5 hours because `do_push` ran synchronously after the commit landed — every subsequent commit

was blocked on a slow mirror push that had nothing to do with the local commit's durability. Implementation seam for §11.4.87(B) zero-idle.

The mandate: (A) `.git/.commit_all.lock` MUST be released IMMEDIATELY after `git commit` returns 0 — the commit is durable on local disk regardless of remote push outcome; (B) push runs detached via `nohup ./push_all.sh ... > <log> 2>&1 & + disown` — orchestrator's exit code reports COMMIT success, NOT push success; (C) `push_all.sh` acquires per-remote flock `.git/.push.<remote>.lock` so concurrent invocations targeting same remote serialize but different-remote invocations run in parallel; (D) backgrounded push failures land in `qa-results/push_failures/<ts>_<remote>.log` — next autonomous-loop tick checks per §11.4.87(A) “no external dependency in-flight” gate; (E) synchronous-push escape: explicit `--sync-push` CLI flag preserves legacy behaviour for §11.4.41 force-push merge-first audit paths.

Composes with §2.1 / §9.2 / §11.4.41 / §11.4.42 / §11.4.71 / §11.4.87. Pre-build gates `CM-COVENANT-114-88-PROPAGATION` + `CM-BACKGROUND-PUSH-WIRED` + paired §1.1 meta-test mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.88.

Non-compliance is a release blocker. Synchronous push (without `--sync-push`) = §11.4 PASS-bluff at execution layer. No escape hatch beyond `--sync-push` for force-push events.

§11.4.89 — Background test execution mandate (User mandate, 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27): “Any tests we are executing, especially long test cycles, MUST BE performed in background in parallel with main work stream! This MUST NOT block our capabilities to work on queued workable items. Main work stream can be blocked or sit iddle only if absolutely needed and if it depends hard on results of some background execution.”

Forensic incident: 2026-05-27 conductor invoked `pre_build_verification.sh` synchronously with foreground `timeout 360`, blocking main stream for 6-7 minutes on §JV/§JW/§JX/§JY scaffolding work. Symmetric anchor to §11.4.88 (background push) at the test-execution layer.

Mandate: (A) Long-running tests (>30s expected: `pre_build`, `meta_test`, `test_all_fixes`, `recent_work_validate`, HelixQA banks, 4-phase cycles, full-suite

retests, audio_loop_supervisor, dual_display_record) MUST run via `nohup ... >`
`<log> 2>&1 & + disown` with log placed under known dir (`qa-results/`
`<test_id>_<ts>.log`); (B) Main stream proceeds to §11.4.42 priority queue
immediately; (C) Hard-dependency gating: poll exit-status file or `pgrep -af`
`<test>` before steps that need exit code — surface as §11.4.66 interactive options
if test still running; (D) Failures land in `<log>` files, next loop tick checks; (E)
Foreground execution permitted ONLY for <30s tests OR explicit operator
authorisation; (F) Per-script flock serialises same-script invocations, different-script
invocations parallel.

Composes with §11.4.42 / §11.4.66 / §11.4.82 / §11.4.84 / §11.4.85 / §11.4.87 /
§11.4.88. Pre-build gates `CM-COVENANT-114-89-PROPAGATION` + `CM-BACKGROUND-`
`TEST-EXECUTION-WIRED` + paired §1.1 meta-test mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.89.

Non-compliance is a release blocker. No escape hatch beyond explicit per-
invocation operator authorisation.

§11.4.90 — Obsolete status + per-item obsolescence audit (User mandate, 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27): “Bug No 6 - Albums cover
etc. seems obsolete after latest request for new behavior when audio and video
content is being played ... mark obsolete tickets with some light gray background ...
text - the description to be strikethrough styled ... review all existing open or
resolved workable items if they are obsolete - not valid any more ... There MUST
NOT be any mistake! No bluff is allowed of any kind!”

§11.4.15 Status closed-set extended with terminal `Obsolete` (→ `Fixed.md`) value
(orthogonal to Type per §11.4.16). Obsolescence reasons (closed vocabulary):
`superseded-by-design-change` | `superseded-by-later-mandate` | `feature-`
`removed` | `duplicate-of` | `unsupported-topology` | `not-reproducible` (`not-`
`reproducible` = a reported defect that does NOT reproduce on the canonical tree/
baseline — an environment/isolated-worktree artifact, not a real product defect;
triple-check evidence captures the canonical-tree non-reproduction). Every
Obsolete heading MUST carry `**Obsolete-Details:**` line (Since + Reason +
Superseding-item + Triple-check evidence) within 8 non-blank lines. §11.4.23
colorizer adds `cell-status-obsolete` class — light-gray `#E0E0E0` background +
strikethrough description. Audit cadence: every release-gate sweep per §11.4.40 +

§11.4.42. Triple-check non-negotiable per operator mandate. Composes §11.4.15 / §11.4.16 / §11.4.19 / §11.4.21 / §11.4.23 / §11.4.33 / §11.4.34 / §11.4.40 / §11.4.42 / §11.4.66 / §11.4.71. Pre-build gates `CM-COVENANT-114-90-PROPAGATION` + `CM-ITEM-OBSOLETE-DETAILS` + `CM-OBSOLETE-COLORIZER-WIRED` + paired §1.1 mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.90. Non-compliance is a release blocker.

§11.4.91 — Summary-doc clarity mandate (User mandate, 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27): “Summary docs - Issues_Summary some not clear one line descriptions - like ‘Composes with’ ... For each workable item we MUST HAVE clearly understandable meaning ... every team member can clearly understand what that particular workable item is exactly about! There cannot be misunderstanding or unclarity of any kind and no bluff allowed!”

Every summary entry (Issues_Summary, Fixed_Summary, README doc-link, Status_Summary page 1+2, all one-liners) MUST contain self-contained meaningful description ≥ 6 words OR ≥ 40 chars naming SUBJECT + PROBLEM/GOAL.

Forbidden one-liner anti-patterns: section labels (`Composes with` , `Closure criteria` , `Fix direction` , etc.); bare metadata fragments (`Critical` , `Bug` , `In progress` , etc.); section-marker echoes; §-letter alone. Generators

(`generate_issues_summary.sh` / `generate_fixed_summary.sh` / `update_readme_doc_links.sh` / `generate_status_summary.sh`) MUST extract from H1/H2 heading line per §11.4.54 ATM-NNN convention, NEVER from arbitrary downstream text. Generators refuse anti-pattern rows — emit (`MISSING DESCRIPTION – fix source heading`) placeholder with visual highlight. Pre-build gate `CM-SUMMARY-CLARITY-DESCRIPTIONS` scans every summary; anti-pattern match = FAIL. Audit cadence: every §11.4.40 + §11.4.42 sweep. Composes §11.4.12 / §11.4.19 / §11.4.23 / §11.4.44 / §11.4.53 / §11.4.56 / §11.4.57 / §11.4.59 / §11.4.60 / §11.4.65 / §11.4.74.

Canonical authority: constitution submodule `Constitution.md` §11.4.91. Non-compliance is a release blocker.

§11.4.92 — Multi-pass change-evaluation discipline (User mandate, 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27): “Every change to the project or codebase we do MUST BE evaluated in several passes and in in-depth analysis for potential new issues or problems it can introduce! ... no bluff of any kind! After we do change or set of changes this mandatory steps MUST BE taken!”

Every non-trivial change MUST pass 5-pass evaluation BEFORE commit-ready: **(Pass 1)** Main-task verification — change achieves stated goal, captured-evidence per §11.4.5/§11.4.69; **(Pass 2)** Regression-blast-radius analysis — enumerate every direct dependency, demonstrate no contract break; **(Pass 3)** Cross-feature interaction analysis — parallel features sharing state/timing/hardware/shell environment audited; **(Pass 4)** Deep-research validation per §11.4.8 — external precedent OR “NO external solution found — original work” + CodeGraph queries per §11.4.78/§11.4.79; **(Pass 5)** Anti-bluff confirmation per §11.4/§11.4.1/§11.4.6/§11.4.27/§11.4.50/§11.4.52/§11.4.69/§11.4.83 — no new bluff surface introduced. Documentation per pass (commit footers OR docs/ entries OR qa-results/ evidence). Only AFTER all 5 passes complete may commit/push/test/release proceed. Trivial exemption: typo / revision-bump / MD-export-regen IF zero source touched AND commit message cites exemption explicitly. Composes §11.4.4 / §11.4.5 / §11.4.6 / §11.4.8 / §11.4.20 / §11.4.27 / §11.4.42 / §11.4.43 / §11.4.50 / §11.4.52 / §11.4.69 / §11.4.78 / §11.4.79 / §11.4.82 / §11.4.83 / §11.4.85 / §11.4.87 / §11.4.89. Pre-build gates `CM-COVENANT-114-92-PROPAGATION` + `CM-MULTI-PASS-EVALUATION-EVIDENCE` + paired §1.1 mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.92. Non-compliance is a release blocker.

§11.4.93 — SQLite-backed single-source-of-truth for workable items (User mandate, 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27): “There MUST be single source of truth for all of our workable items - SQLite database ... proper scripts (we recommend Go programs) ... reduce a chance for sync to be broken ... generate always all docs from DB or to re-generate Db from all docs we have in opposite direction”.

Text-based Issues/Fixed/Summary/CONTINUATION constellation converted to SQLite-DB-backed single source of truth. DB at `docs/.workable_items.db`

(gitignored per §11.4.30 + §11.4.77 regen mechanism). Schema mandatory tables: items (atm_id PK + Type + Status incl. Obsolete + Severity + title + description ≥40chars + created/modified + composes_with JSON + current_location); item_history (append-only audit per §11.4.34 By/Reason/Evidence); obsolete_details (§11.4.90); operator_block_details (§11.4.21); firebase_metadata (§11.4.47); meta (schema version + last sync + integrity hash). Go binary at `cmd/workable-items/` : sync md-to-db / db-to-md / diff / validate / add / close. Bidirectional regen byte-identical round-trip (closed-set tolerance for whitespace/section-order). `commit_all.sh` refuses on diff non-empty. `sync_issues_docs.sh` invokes Go binary. `pre_build` runs `workable-items validate` . Anti-bluff: unit + integration + stress (1000-row insert + 10 concurrent writers) + chaos (mid-write SIGKILL + corrupt-DB recovery + disk-full) + paired §1.1 mutation + HelixQA Challenge CME-WORKABLE-ITEMS-001. 6-phase migration: §LA Issues entry → Go binary scaffold + DDL → md-to-db → db-to-md round-trip CI → generator shims → text-direct edits prohibited. Cross-project: Go binary lives in constitution submodule (`constitution/scripts/workable-items/`) per §11.4.74. Composes §11.4 / §11.4.12 / §11.4.15 / §11.4.16 / §11.4.17 / §11.4.19 / §11.4.21 / §11.4.27 / §11.4.30 / §11.4.33 / §11.4.34 / §11.4.42 / §11.4.43 / §11.4.44 / §11.4.45 / §11.4.50 / §11.4.52 / §11.4.53 / §11.4.54 / §11.4.55 / §11.4.56 / §11.4.57 / §11.4.58 / §11.4.60 / §11.4.65 / §11.4.74 / §11.4.83 / §11.4.85 / §11.4.86 / §11.4.87 / §11.4.89 / §11.4.90 / §11.4.91 / §11.4.92. Pre-build gates `CM-COVENANT-114-93-PROPAGATION` + `CM-WORKABLE-ITEMS-DB-PRESENT` + `CM-WORKABLE-ITEMS-MD-DB-IN-SYNC` + paired §1.1 mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.93. Non-compliance is a release blocker — text-based-only trackers are §11.4 PASS-bluff at data-architecture layer.

§11.4.94 — Zero-idle priority-first parallel-by-default operating mode (User mandate, 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27): “We MUST NEVER sit iddle / wait or sleep if there is possibility for us to work on something ... Always check if there is a possibility to work on something while we are not working actively on something! Pick always by priority - most critical workable items and other tasks MUST BE done first! ... Stay still / iddle if nothing is left to be done at all or waiting for something that is blocking us / you!!!”

§11.4.94 binds

§11.4.20+§11.4.42+§11.4.58+§11.4.70+§11.4.72+§11.4.82+§11.4.87+§11.4.88+§11.4.89 into single always-on enforcement: (A) Idle ONLY when every queued item genuinely blocked on external dep (hardware / network upstream / build/test completion conductor cannot accelerate) OR operator STOP OR §12 host-safety; “don’t see what to do” is NEVER valid; (B) before ANY wake/sleep MUST survey parallel-work feasibility per §11.4.42+§11.4.72+§11.4.87, identify non-contending items, dispatch in parallel per §11.4.20/§11.4.70 (subagent) + §11.4.58 (PWU disjoint scope) + §11.4.89 (background long tests); (C) priority order MANDATORY — pick highest-severity+§11.4.72 audio-first the conductor can autonomously progress; (D) subagent-driven default for non-trivial; (E) background default for >30s wall-clock work via nohup+disown; (F) stability-preserving — composes with §11.4.92 multi-pass + §11.4.84 quiescence + §12.6-§12.9 host safety; (G) progress updates surfaced when operator asks OR at milestone boundaries. Anti-pattern forbidden: scheduling wake without queue survey; “nothing to do — sleeping” with non-trivial items progressable; serialising parallelisable work; picking lower priority over higher. Pre-build gates `CM-COVENANT-114-94-PROPAGATION` + `CM-PARALLEL-WORK-AUDIT` + paired §1.1 mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.94. Non-compliance is a release blocker.

§11.4.95 — Workable-items SQLite DB TRACKED in git, NEVER gitignored (User mandate, 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27): “We shall not Git ignore our workable items SQLite DB since it is our single source of truth ... workable items SQLite DB regularly committed and pushed to all upstreams!”

§11.4.93’s earlier “gitignored per §11.4.30” clause is AMENDED — the DB at `docs/workable_items.db` is TRACKED in git, NEVER gitignored. It IS authoritative source data, NOT a build artefact. Every `workable-items sync md-to-db` that mutates state MUST stage+commit+push the DB alongside the MD regen per §11.4.19 atomic-move + §2.1 multi-upstream push. WAL-checkpoint (`PRAGMA wal_checkpoint(TRUNCATE)`) required before commit-stage so transient `.db-wal` + `.db-shm` sidecars (gitignored per §11.4.30) are safely discardable. §11.4.77 regeneration mechanism does NOT apply — DB IS the source. Destructive DB ops require §9.2 hardlinked-backup + operator authorization; §11.4.41 force-push merge-first applies if DB history ever needs rewrite. Pre-build gates `CM-`

COVENANT-114-95-PROPAGATION + CM-WORKABLE-ITEMS-DB-TRACKED + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.95. Non-compliance is a release blocker.

§11.4.96 — Safe-parallel-work-with-long-build catalogue + mandate (User mandate, 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27): “Are there except AOSP build process any other active jobs being done at the moment? Can we work on something in parallel while build is in progress so we slowly cleanup our slate? ... If yes, and that was not already clear by our root Constitution, we should add all mandatory details into it ... do as much as possible work in background in parallel with main work stream and oreferably using subagents-driven approach!”

Operational catalogue for the canonical long-running workload (5-7h AOSP containerised build per §12.9):

SAFE during build: (A) MD/docs work under docs/+README+CLAUDE/AGENTS/QWEN+Status+Issues/Fixed+changelogs+research notes; (B) generator/helper script work under scripts/ scripts/testing/ scripts/llm/ scripts/firebase/; (C) pre-build + meta-test gate authoring + paired §1.1 mutations; (D) on-device test scripts under tests/test_*.sh; (E) constitution submodule edits + push to 6 remotes; (F) any submodule commit + push per §11.4.88; (G) D3/D4 read-only live-ADB probes (dumpsys/getprop/cat /proc/asound/screencap/logcat); (H) subagent dispatch per §11.4.20/§11.4.70 + §11.4.84 quiescence; (I) web research + external API queries with §11.4.10 credentials; (J) workable-items DB ops per §11.4.93+§11.4.95; (K) pre-build + meta-test execution backgrounded per §11.4.89.

UNSAFE during build: (α) git checkout/reset –hard/clean -df on source tree (use git worktree instead); (β) mass file deletes/renames under device/+frameworks/+hardware/+vendor/+kernel-5.10/+external/+packages/+bionic/+bootable/+build/+system/+cts/+tools/+prebuilts/; (γ) submodule pointer updates affecting built APKs (defer to between-build windows); (δ) out/ directory mutations; (ε) make clean/m clobber/rm -rf out/; (ζ) container destruction; (η) disk-filling operations breaching §12.9 free-space minimum; (θ) §12 host-session-safety breaches.

Conductor responsibility: before EVERY pause point during long build, consult catalogue, identify (A)-(K) queue items per §11.4.42+§11.4.72, dispatch ≥1 per

§11.4.20/§11.4.70 subagent default + §11.4.89 background. “Build running, nothing else to do” NEVER true per §11.4.94+§11.4.96. Pre-build gates `CM-COVENANT-114-96-PROPAGATION` + `CM-PARALLEL-WORK-DURING-BUILD-AUDIT` + paired §1.1 mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.96. Non-compliance is a release blocker.

§11.4.97 — Maximum-use-of-idle-time + progress-update cadence (User mandate, 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27): “keep it working, we should do as much as possible, if not it all but as much as we can as long as there is iddle time! it MUST be used! ... keep us updated about all progress and all phisycal proofs and gathered data as you progress through all open workable items!”

Operating-mode capstone strengthening §11.4.87+§11.4.94+§11.4.96: (A) every minute of conductor idle time during which work could autonomously progress AND not genuinely blocked = §11.4.97 violation; “as much as possible, if not it all but as much as we can” is operative — dispatch CONTINUOUSLY through entire idle window, not just at scheduled wakes; (B) progress-update cadence — emit operator-facing 1-line update at every: commit landed / subagent return / constitutional anchor / captured evidence / milestone closure; no operator prompt required; (C) continuous physical-proof gathering per §11.4.5+§11.4.6+§11.4.69 — every autonomous closure cites captured-evidence; evidence path goes into §11.4.93 `item_history.evidence_path` when DB lands; (D) composes with §11.4.5/6/13/20/27/42/50/52/69/70/72/83/85/87/88/89/94/96; (E) idle-only-when-blocked closed-set unchanged from §11.4.94(A). Pre-build gates `CM-COVENANT-114-97-PROPAGATION` + `CM-IDLE-TIME-AUDIT` + paired §1.1 mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.97. Non-compliance is a release blocker.

§11.4.98 — Full-Automation Anti-Bluff Mandate — Live tests MUST be re-runnable end-to-end without manual intervention (User mandate, 2026-05-28)

Forensic anchor — verbatim user mandate (2026-05-28): “Make sure we have full automation testing of all scenarios with real bot, main group and users without any manual intervention or contribution of real user! Everything MUST BE fully automatic and autonomous! These tests MUST BE able to rerun endless times when needed! ... Make sure there is no false positives in testing! Every test and its results MUST obtain real proofs of everything working! No bluff is allowed!”

Composes with §11.4 + §11.4.2 + §11.4.5 + §11.4.50 + §11.4.85 + §11.4.87 + §11.4.89 + §11.4.94 — closes the **manual-intervention gap** they did not explicitly forbid. A live/integration/e2e/Challenge test requiring a human action during execution (typing a chat message, clicking a UI, hand-triggering a webhook, anything beyond test start + PASS/FAIL report) is **by definition a §11.4 PASS-bluff at the automation layer**, regardless of how thorough the manual run is — cannot run continuously in CI, cannot validate regressions between manual runs, human dependency masks drift.

1. Binding rule: every test this Constitution governs — unit/integration/e2e/Challenge/stress/chaos/live — MUST be fully self-driving end-to-end; reports PASS/FAIL/SKIP-with-reason without any further human action after startup. (B) Single permissible exception: one-time credential bootstrap OUTSIDE test execution (`.env` from vault, shell exports in `~/.bashrc` , OAuth approval at first install, MTPROTO session activation at first run) — configuration, not test driving. (C) Concrete requirements for live messenger/channel/agent tests: (1) no “operator MUST type a message” prompts — drive programmatically (MTPROTO for Telegram, real-user-token API for Slack, IMAP-test-account for email, webhook fixture, in-process loopback — never human keystrokes); (2) no hard-coded session UUIDs that collide with active dev session (Herald 2026-05-28 lesson: `claude --resume <UUID>` on same UUID as dev session returns silent exit -1); (3) no 60s human-response windows (§11.4.50 determinism violation); (4) re-runnability proof — PASS at `-count=3` consecutive automated invocations with self-cleaning state; (5) §11.4.98 obsolescence audit — every existing test classified COMPLIANT vs NON-COMPLIANT; (6) no false-positive PASS — silent-skip-reported-as-PASS forbidden, stale-evidence forbidden, SKIP-with-reason per §11.4.3 is correct. (D) Composes with §11.4.85 + §11.4.89 + §11.4.87 + §11.4.94 — together =

continuously-validated, fully-automated, non-flake, anti-bluff regime. (E)
Inheritance per §11.4.35 — every consuming repo's CLAUDE.md/AGENTS.md/
QWEN.md restates citing literal anchor 11.4.98 ; pre-build gate CM-
COVENANT-114-98-PROPAGATION enforces literal presence; paired §1.1
mutations strip → gates FAIL. (F) Enforcement: commit adding manual-action
test BLOCKED at release-gate; manual-dependency test not rewritten within 30
days graduates to §11.4.90 Obsolete citing §11.4.98 as obsolescence reason.

Canonical authority: constitution submodule `Constitution.md` §11.4.98. Non-compliance is a release blocker.

§11.4.99 — Latest-Source Documentation Cross-Reference Mandate — instructions/guides/manuals MUST be verified against latest official online sources BEFORE publication (User mandate, 2026-05-28)

Forensic anchor — verbatim user mandate (2026-05-28): “Make sure we ALWAYS check against latest versions of services we use web / online docs before creating instructions! This situation is illustration of how we can misguide ourselves or get banned! ... These are mandatory rules / constraints and the result is consistency and safety of created instructions, guides and manuals!”

Case study (Herald 2026-05-28): first-draft Herald MTPProto setup guide recommended VoIP/Google-Voice/Twilio fallback AND omitted the `recover@telegram.org` pre-login email — both directly contradicted Telegram's official docs + gotd/td maintainer guidance, and following the guide could have caused a permanent Telegram account ban. Misguidance-by-stale-docs is the same severity class as a §11.4 PASS-bluff at the documentation layer.

Composes with §11.4.4 + §11.4.5 + §11.4.8 + §11.4.92 Pass 4 + §107 + §11.4.98.
Closes the gap §11.4.92 Pass 4 alludes to but does not explicitly mandate. (A) Pre-commit cross-reference: every operator-facing instruction document MUST (1) fetch the LATEST official online docs of the service/library via WebFetch/MCP/direct browsing (NEVER training data, memory, or prior committed docs); (2) cross-reference each instruction against the source; (3) seek secondary authoritative sources when the official docs are sparse/silent (library maintainer SUPPORT.md, official changelogs, vetted community FAQs); (4) cite source URL + date in a “## Sources verified” footer on the doc; (5) cite the cross-reference in the commit-message footer (`Sources verified <date>: <urls>`). (B) Negative findings:

gaps/silences/contradictions MUST be documented explicitly so the next reader doesn't assume absence-of-contradiction is authoritative agreement. (C) Re-verification cadence: documents older than 6 months are STALE — re-verify before citing as operator authority, at every vN.0.0 release boundary, on service breaking-change announcements, or when an operator reports an error following the guide. (D) Risk-classified services (Telegram/WhatsApp messengers, cloud APIs, payment systems, AI/LLM providers, code-hosting services, package managers) — 90-day max staleness; explicit safety warnings cited against latest policies. (E) Composes with §11.4.92 Pass 4 but is INDEPENDENT — cannot substitute. (F) Inheritance per §11.4.35 — restate literal anchor 11.4.99 in every consumer's CLAUDE/AGENTS/QWEN; pre-build gate CM-COVENANT-114-99-PROPAGATION (when implemented) enforces; paired §1.1 mutations strip → gates FAIL. (G) Enforcement: commit without "Sources verified " doc-footer AND commit-message-footer BLOCKED at release-gate; stale-beyond-grace docs graduate to §11.4.90 Obsolete with Reason=stale-documentation .

Canonical authority: constitution submodule Constitution.md §11.4.99. Non-compliance is a release blocker.

§11.4.100 — RETIRED. Demoted to consumer project (a consuming project's video-color/visual-quality fidelity) per §11.4.17/§11.4.35 — project-specific (RK3588/MPV/Arvus), not universal. See the consuming project's Constitution/CLAUDE/AGENTS/QWEN.

§11.4.101 — Autonomous-decision-over-blocking mandate (User mandate, 2026-05-28)

Forensic anchor — verbatim user mandate (2026-05-28):

"when working in endless working loop fully autonomously try to decide most properly about points which would block execution and wait for us. If we haven't answered now work would be blocked whole night! If possible and if that will not cause any issues make proper and most reliable and safe decision so we achieve maximal efficiency and work gets fully done!"

When operating in an autonomous / endless-loop mode (per §11.4.87), the agent MUST minimize operator-blocking and instead make the safe, reliable, reversible decision itself — so work is NOT stalled (e.g. overnight) waiting for input. §11.4.87

says keep working; §11.4.101 says HOW to clear the decision points that would otherwise force a stop-and-wait.

Decision rule (closed-set — proceed autonomously when ALL hold): (a) the action is reversible OR has a captured pre-op backup per §9.2; (b) the agent can determine the safe choice from captured evidence per §11.4.6 (no guessing — `LIKELY` is not a determination); (c) a wrong choice's blast radius is bounded AND recoverable; (d) it composes with anti-bluff §11.4, host-safety §12, data-safety §9.

Block-only-when rule (BLOCK via §11.4.66 ONLY when ALL hold): the action is irreversible AND high-blast-radius AND the safe choice cannot be determined from evidence — e.g. external-account state the agent cannot inspect, hardware it cannot access, destructive ops without backup, force-push (also §9.2 + §11.4.41), spending / sending to third parties. `operator-blocked` per §11.4.21 is reached only after this rule fires AND the self-resolution-exhaustion audit completes.

Maximize-progress-while-blocked: an unavoidable block parks one work unit, it does not pause the loop — the agent MUST keep progressing every NON-blocked item in parallel per §11.4.87 + §11.4.94. Posing the question and going idle is a §11.4.94 + §11.4.97 violation.

Composes with §11.4.6 / §11.4.21 / §11.4.40 / §11.4.41 / §11.4.66 / §11.4.87 / §11.4.94 / §9.2 / §12. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-101-PROPAGATION` enforces the literal anchor `11.4.101` across the consumer fleet; paired §1.1 meta-test mutation strips the literal → gate FAILs (gate-code = separate work item). No escape hatch — no `--always-block-on-decision`, `--never-decide-autonomously`, `--skip-decision-rule`, `--block-without-self-resolution` flag exists.

Canonical authority: constitution submodule `Constitution.md` §11.4.101.

Non-compliance is a release blocker regardless of context.

§11.4.102 — Mandatory systematic-debugging activation + always-loaded skill-discovery + plugin-dependency availability (User mandate, 2026-05-29)

Forensic anchor — verbatim user mandate (2026-05-29):

“Make sure that we ALWAYS trigger / start the”/superpowers:systematic-debugging” skills when any issues happen! If this is possible to activate and use in this situations out of the box when we spot problems / issues / bugs / misalignments / inconsistencies we MUST activate the skill(s) and make strongest efforts in full in depth analysis / debugging and determine root causes of all problem or obtain relevant data and information we need! ... we MUST make sure that “/using-superpowers” skill is ALWAYS loaded, applied and used! All dependencies (plugins) that Claude Code or other market places are offering MUST BE installed if these are not already available for loading and use!”

Three cooperating invariants — the difference between guess-and-retry and investigate-to-root-cause-first.

(A) Mandatory systematic-debugging activation. On ANY spotted issue / bug / test failure / gate failure / regression / misalignment / inconsistency / unexpected behaviour, the agent MUST activate `superpowers:systematic-debugging` (or the platform-equivalent structured-debugging discipline) **BEFORE proposing, writing, or applying any fix** — the **Iron Law: NO FIXES WITHOUT ROOT CAUSE INVESTIGATION FIRST**. Full four-phase arc: root-cause (read the real failure, reproduce, gather facts) → pattern (classify the defect, search for the same pattern elsewhere) → hypothesis (form a falsifiable cause, prove/disprove with captured evidence) → implementation (design the fix only against the proven root cause). Guess-and-retry, symptom-patching, and re-running a failed test hoping it passes (“probably transient / flaky”) WITHOUT a completed investigation are §11.4.102 violations. Real-world signal: calling a failure `transient` / `flaky` / `intermittent` / `probably-timing` without captured forensic evidence is simultaneously a §11.4.6 (no-guessing) and §11.4.7 (demotion-evidence) violation — §11.4.102 makes the corrective response mechanical (the classification is forbidden until the systematic-debugging arc proves it).

(B) Mandatory always-loaded using-superpowers . `superpowers:using-superpowers` (or the platform-equivalent skill-discovery / capability-index discipline) MUST be loaded and applied at session start and consulted before any task. Operative rule: before acting on ANY request, survey available skills; if ANY skill could apply — even at 1% relevance — it MUST be invoked rather than improvised

from memory. Skipping skill-discovery at session start, or improvising a task a loaded skill exists for, is a §11.4.102 violation.

(C) Mandatory plugin / dependency availability. Every skill plugin / marketplace package / capability dependency the project relies on (Claude Code, its skill marketplaces, or any other runtime's plugin ecosystem) MUST be installed + loadable BEFORE the dependent work proceeds. A missing plugin that blocks a mandated skill (e.g. `superpowers` absent so `systematic-debugging` cannot launch) is a **release-blocker** until installed + confirmed loadable. Install mechanism: the runtime's own plugin/marketplace install path — for Claude Code the in-session `/plugin` marketplace flow (add marketplace → install plugin → confirm the skill appears in the available-skills list); for other runtimes the documented package/extension installer. Anti-bluff: confirm by observing the skill in the live capability list, never assume install succeeded (install exit 0 ≠ skill loadable, per the §11.4.80 lesson).

Composes with §11.4.4 (test-interrupt — first action after STOP is launch `systematic-debugging`), §11.4.6 (no-guessing — (A) is the procedural enforcement producing the facts §11.4.6 demands), §11.4.7 (demotion-evidence — the arc captures same-conditions evidence), §11.4.8 (deep-web-research — inside the hypothesis phase), §11.4.43 (TDD-fix — RED test written against proven root cause), §11.4.70 (subagent-driven — the investigation is a natural subagent dispatch), §11.4.82(A) (generalises Phase-1-forensic-before-speculative-patch to ANY fix + adds skill-discovery + plugin-availability), §11.4.92 (feeds Pass 1 + Pass 4). Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-102-PROPAGATION` (literal `11.4.102` across consumer fleet) + paired §1.1 meta-test mutation (strip literal → gate FAILs; gate-code = separate work item). No escape hatch — no `--skip-systematic-debugging`, `--guess-and-retry-OK`, `--symptom-patch-permitted`, `--skip-skill-discovery`, `--plugin-optional`, `--missing-plugin-is-warning` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.102.

Non-compliance is a release blocker regardless of context.

§11.4.103 — Continuous parallel-stream working routine (User mandate, 2026-05-29)

Forensic anchor — verbatim user mandate (2026-05-29):

“Do this working approach continuously and make it part of regular working routine and add it to the Constitution Submodule documented fully.”

Promotes the proven multi-stream operating pattern into the project’s **standing default working routine** (not a per-request opt-in). Binds §11.4.87/§11.4.88/§11.4.89/§11.4.94/§11.4.96 and adds the load-bearing invariant: **the main work stream MUST always stay FREE.**

(A) Main stream stays FREE. ALL commit AND push operations run detached (`nohup ... & + disown` , per §11.4.88 commit-lock-release-immediately + detached-push) — the main stream returns to the priority queue the moment the local commit is durable, never blocking on a push or a slow mirror. **(B) ≥3 parallel background streams at all times + auto-backfill** (User mandate 2026-05-29; raised from ≥2) — run **at least three** subagent-driven background streams (per §11.4.70/§11.4.20, isolated per §11.4.58 PWU worktree + §11.4.84 quiescence) alongside the main stream whenever three-plus non-contending actionable items exist; **the moment any one stream is FULLY done, a new stream MUST immediately start and take its place** (claim next-highest-priority non-contending item), so the active-stream count NEVER drops below 3 while actionable items remain. Idle below 3 is permitted ONLY when no remaining non-contending actionable items OR all remaining are externally blocked (§11.4.94/§11.4.97/§11.4.101). Standing band 3–6, bounded above by §12.6 60% memory + §11.4.58 6-agent cap. **(C) Most-critical + most-visible first; audio always top** per §11.4.72 + §11.4.42 priority order. **(D) Safe-during-build scope only** — while the 42 GB containerised AOSP rebuild (§12.9) or any heavy `gradle / m -j` build runs, streams restrict to the §11.4.96 SAFE catalogue (investigation/forensic/docs/test-authoring/gate-authoring/read-only probes/submodule edits/research/DB ops/backgrounded pre-build+meta-test); NEVER a second concurrent heavy build (§12.8). **(E) Heavy anti-bluff on every closure** — root cause proven or UNCONFIRMED: / UNKNOWN: / PENDING_FORENSICS: per §11.4.6, captured evidence per §11.4.5+§11.4.69, deterministic consistency per §11.4.50, paired §1.1 mutations, §11.4.102 systematic-debugging on any spotted problem. **(F) Idle ONLY when genuinely externally blocked** (hardware/network upstream/in-flight build-test-push completion) OR operator STOP OR §12 host-safety, per §11.4.94(A) + §11.4.97; “nothing visible to do” with progressable items is NEVER valid; a block parks one work unit, not the loop (§11.4.101).

Composes with §11.4.58 / §11.4.70 / §11.4.72 / §11.4.87 / §11.4.88 / §11.4.89 / §11.4.94 / §11.4.96 / §11.4.97 / §11.4.101 / §11.4.102 / §11.4.42 / §11.4.84 / §12.6 / §12.7 / §12.8 / §12.9 / §9.2. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-103-PROPAGATION` (literal `11.4.103` across consumer fleet) + paired §1.1 meta-test mutation (strip literal → gate FAILs; gate-code = separate work item). No escape hatch — no `--block-main-stream`, `--synchronous-commit`, `--synchronous-push`, `--single-stream-only`, `--skip-parallel-streams`, `--serialise-actionable-work`, `--idle-without-queue-survey` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.103.

Non-compliance is a release blocker regardless of context.

§11.4.104 — Participant identity, attribution & notification-tagging (User mandate, 2026-05-31)

Forensic anchor — verbatim user mandate (2026-05-31):

“Every supported messenger must relate messages to participants (Subscribers/Users); the same logical person may have a different username on every messenger. Workable items must carry who created them and who they are assigned to. Notifications must @-tag the right participant — but never the operator (who drives the system) and never the system agent.”

MANDATORY for every consumer that ships a messenger/notification surface (Herald + flavor binaries are the reference impl; others inherit per §11.4.35).

Detailed spec: Herald `docs/design/PARTICIPANT_ATTRIBUTION.md` — restated,

not redefined. **(A) Participant identity.** Every messenger MUST relate messages to a **Participant** (logical Subscriber/User); the SAME person MAY have a DIFFERENT username per messenger — modelled as a logical subscriber (canonical messenger-neutral `handle`, `kind` ∈ {human, agent, service}) + per-channel aliases (`channel`, `channel_user_id`, the `@username` used for tagging).

Canonical handle closed set: `Claude` (reserved system-agent sentinel; never tagged) OR a subscriber `@username`. **(B) Operator = env var, not a DB flag** — the one human who drives via the agent CLI, designated by

`HERALD_<CHANNEL>_OPERATOR_USERNAME`

(e.g. `HERALD_TGRAM_OPERATOR_USERNAME`); a normal Participant whose handle

equals that value. **(C) Workable items MUST carry `created_by` + `assigned_to`** (canonical handles): CLI-prompt → Operator; system/agent-detected → `Claude` ; received-message → sender's resolved `@username` . `assigned_to` defaults to Operator, overridable. Legacy items carry `""` and MUST still parse + validate. **(D) Tagging matrix:** tag `assigned_to` if it is a human ≠ Operator; tag `created_by` if it is a human ≠ Operator ≠ `Claude` ; NEVER tag `Claude` (system) or the Operator (no self-ping); de-dup; resolve each handle to the channel `@username` , skip if not on that channel. **(E) Anti-bluff (composes §11.4):** real SQLite round-trip with the new columns byte-identical (incl. legacy fixtures WITHOUT the fields); tagging matrix proven by a truth-table + a cell-flip mutation forcing FAIL; E2E real-event → real-message asserting the exact `@username` s + a NEGATIVE case proving the Operator is NOT tagged; evidence under `docs/qa/<run-id>/` .

Composes with §11.4 + §11.4.1..§11.4.16 (anti-bluff covenant) / §11.4.5 / §11.4.69 / §11.4.50 / §11.4.91 / §11.4.93 / §11.4.95 / §1.1. Classification: universal (§11.4.17) — projects with no messenger surface inherit it latently (binds the moment they ship one, per the §11.4.96 pattern). Propagation gate `CM-COVENANT-114-104-PROPAGATION` (literal `11.4.104` across consumer fleet) + paired §1.1 meta-test mutation (strip literal → gate FAILs; gate-code = separate work item). No escape hatch — no `--skip-attribution` , `--no-participant-tagging` , `--tag-operator-anyway` , `--attribution-later` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.104; detailed spec Herald `docs/design/PARTICIPANT_ATTRIBUTION.md` .

Non-compliance is a release blocker.

§11.4.105 — Natural-language intent recognition & clarification (User mandate, 2026-05-31)

Forensic anchor — verbatim user mandate (2026-05-31):

“Users must NOT need to know command syntax (no `COMMAND: ...` prefix). They send a clear natural-language message; the System determines the intent. The System recognizes the commands it has; if none matches it infers the exact intent; if it is totally unable it replies, tags the user (`@user ...`), and

asks to clarify precisely. We MUST always do our best to determine exact intent so we never annoy end users. This is a CORE part of the System.”

MANDATORY for every consumer that ships a messenger/command surface (Herald + flavor binaries are the reference impl; others inherit per §11.4.35).

Detailed spec: Herald `docs/design/INTENT_RECOGNITION.md` — restated, not redefined. **(A) No required command syntax.** Users MUST NOT be required to know any command syntax (no `COMMAND:` prefix); they send plain natural language and the System determines the intent. **(B) Three-tier resolution** (first that succeeds wins): TIER 1 — recognize the System’s existing command set from natural language → action (confident deterministic match); TIER 2 — when no command matches, infer the exact intent (LLM dispatch maps language → action), NEVER guessing; TIER 3 — when neither a command nor a confident intent can be determined, REPLY to the message, TAG the sender (`@username` , resolved via the §11.4.104 IdentityResolver) and ask a PRECISE clarifying question NAMING the candidate intents — no guessing, no silent drop. **(C) Never guess, never drop:** a wrong action is worse than a clarifying question (composes §11.4.6 no-guessing); a message is NEVER silently dropped; only genuine ambiguity reaches Tier 3, which always replies-tags-and-asks. **(D) Anti-bluff (composes §11.4):** every tier ships unit + integration + E2E + full-automation tests with real captured evidence — Tier 1 truth-table (natural-language → action+fields, plus conservative negatives that MUST fall through to “no match”); Tier 3 E2E whose recorded reply body is EXACTLY `@<sender> <specific question>` + a NEGATIVE proving a clear command does NOT trigger clarify; a paired §1.1 mutation breaking the confidence guard (false-match) OR dropping the clarify tag MUST FAIL a test; evidence under `docs/qa/<run-id>/` .

Composes with §11.4 + §11.4.1..§11.4.16 (anti-bluff covenant) / §11.4.6 (no-guessing) / §11.4.104 (clarify reply tags the sender) / §11.4.5 / §11.4.69 / §11.4.98 / §1.1. Classification: universal (§11.4.17) — projects with no messenger surface inherit it latently (binds the moment they ship one, per the §11.4.96 pattern).

Propagation gate `CM-COVENANT-114-105-PROPAGATION` (literal `11.4.105` across consumer fleet) + paired §1.1 meta-test mutation (strip literal → gate FAILs; gate-code = separate work item). No escape hatch — no `--require-command-syntax` , `--guess-intent-ok` , `--skip-clarify` , `--drop-on-ambiguous` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.105; detailed spec Herald `docs/design/INTENT_RECOGNITION.md` .

Non-compliance is a release blocker.

§11.4.106 — Docs Chain — mechanical documentation/DB sync engine (Operator mandate, 2026-05-31)

Forensic anchor — operator mandate (2026-05-31): Docs Chain is the canonical mechanical enforcer of the documentation-sync mandates; consumers MUST use the engine instead of ad-hoc per-project doc-sync scripts, register their chains via per-context YAML, and never accept a faked transform.

MANDATORY for every consumer. Docs Chain (the `vasic-digital/docs_chain` engine — a universal Go bidirectional document-and-database dependency-propagation engine) is the canonical mechanical enforcer of the documentation-sync mandates. Detailed spec: the engine docs `~/Projects/docs_chain → docs/CONSTITUTION_INTEGRATION.md` (distribution + inheritance + anchor mapping table) + `docs/USE_CASE_CATALOGUE.md` (chain recipes) — restated, not redefined. **(A) Use the engine, never ad-hoc scripts** — consumed **by reference** (the flat-layout sibling `~/Projects/docs_chain / the constitution-exposed path`), inherited like §11.4.80's `codegraph_*` scripts: referenced, NEVER copied; ad-hoc `sync_*` / `generate_*`_summary / `update_readme_doc_links` scripts are superseded and retired per registered context. **(B) Consumer-owned contexts** — the engine is project-agnostic; the consumer registers its chains as data via `.docs_chain/contexts/*.yaml` (§11.4.28 decoupling); `state.json` + `*.docs_chain.tmp` are gitignored. **(C) Anchors it mechanizes** (per the `CONSTITUTION_INTEGRATION` mapping table): §11.4.12 / §11.4.53 / §11.4.45 / §11.4.56 / §11.4.57 / §11.4.59 / §11.4.60 / §11.4.65 / §11.4.86 / §11.4.93 / §11.4.95 / §12.10 / §11.4.44 — with content-hash change detection (NOT mtime, §11.4.86), atomic-rename + SQLite-txn commit + rollback (§9.2), both-dirty `sync` → conflict-not-silent-merge (§11.4.6), `verify` as the deterministic CI/pre-build gate (§11.4.50), per-run captured evidence to `qa-results/docs_chain/<run-id>/` (§11.4.69). **(D) NOT a replacement for authoring discipline** — the source author still writes the §11.4.44 revision header; the engine only keeps exports in sync. **(E) Anti-bluff (composes §11.4):** the engine NEVER fakes a transform — a missing pandoc/weasyprint surfaces a typed `ToolAbsentError` + honest §11.4.3 SKIP-with-reason, never a fake PASS / partial write; every `sync` / `verify` carries real captured evidence.

Composes with §11.4 + §11.4.1..§11.4.16 (anti-bluff covenant) / §11.4.6 (no-guessing — conflict not silent merge) / §11.4.28 (engine/context decoupling) / §11.4.80 (inherited-by-reference, never copied) / §9.2 / §11.4.50 / §11.4.69 / §11.4.5 / §1.1, plus the sync anchors it mechanizes (§11.4.12/.53/.45/.56/.57/.59/.60/.65/.86/.93/.95/.44, §12.10). Classification: universal (§11.4.17) — projects with no derived-export/DB-sync surface inherit it latently (binds the moment they ship one, §11.4.96 pattern). Status (§11.4.6): engine Phases 1–3 AND the CLI/YAML loader (Phase 4) IMPLEMENTED+tested — `cmd/docs_chain/main.go` registers `sync / verify / diff (alias)/ doctor`, `go test -count=1 ./...` GREEN 7/7 packages; this CLI/loader is in the working tree but UNTRACKED and the engine is NOT yet a registered git submodule (in-tree at `./docs_chain`, HEAD = parent HEAD); only submodule distribution (Phase 6) remains PLANNED + OPERATOR-GATED — wire as phases land, claim no unshipped behaviour. Propagation gate CM-COVENANT-114-106-PROPAGATION (literal `11.4.106` across consumer fleet) + paired §1.1 meta-test mutation (strip literal → gate FAILs; gate-code = separate work item). No escape hatch — no `--ad-hoc-sync-ok`, `--skip-docs-chain`, `--fake-transform`, `--sync-evidence-optional` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.106; detailed spec the Docs Chain engine docs `docs/CONSTITUTION_INTEGRATION.md` + `docs/USE_CASE_CATALOGUE.md` (`vasic-digital/docs_chain`).

Non-compliance is a release blocker.

§11.4.107 — Anti-bluff AV/test-validation techniques mandate (User-driven research, 2026-06-02)

Forensic anchor (genericised, 2026-06-02): a test PASSEd on a SINGLE captured frame showing “a picture” on the target output — but the picture was a FROZEN / STALE frame from the previously-played content (stale-producer / stuck-decoder), so the feature was broken for the user while the test was green; a sibling incident FLASHED media on the WRONG output for ~1 s before routing with no test sampling that window; a third class shipped a comparator that PASSEd its own deliberately-degraded fixture (the analyzer, not the feature, was the bluff). §11.4.5 mandates captured evidence + a presence pass; §11.4.107 raises the bar to **liveness + correct-routing + self-validated-analyzer**.

Every test asserting audio/video output is genuinely playing/advancing for the end user MUST satisfy ALL of: (1) **a single captured frame is NOT proof** — prove LIVE, ADVANCING frames over a steady-state window via a **freeze-detection oracle** (near-duplicate / `freezedetect` -class filter OR perceptual-hash adjacent-frame distance, **NOT byte-identical compare** — byte-identity is only a zero-cost pre-filter); (2) an **independent frame-advance counter from the platform's compositor/decoder telemetry** must increase across the window (a different-domain oracle — flat counter $\Rightarrow$ stuck decoder $\Rightarrow$ FAIL even if pixels appear to move); (3) **loading/buffering is a distinct state** — wait for genuine playback-start before judging liveness, never false-FAIL a still-loading stream nor false-PASS a spinner; timeout+unreachable $\Rightarrow$ SKIP-with-reason per §11.4.3, timeout+reachable $\Rightarrow$ FAIL; (4) **not-stale-from-previous cross-check** — new content's first frame $\neq$ previous content's last frame; (5) **measured FPS / no-lost-frames within tolerance**; (6) **no-flash-on-the-wrong-output** — sample the non-target output at high frequency during a routing transition, any content frame there $\Rightarrow$ FAIL (content-protection regime classified explicitly, never guessed); (7) **drive through the realistic feed/UI path, not deep-link shortcuts** (shortcuts bypass the transition paths where bugs live; UI-not-introspectable $\Rightarrow$ operator-attended fallback per §11.4.52, never fake-PASS); (8) **metamorphic relations solve the oracle problem when there is no golden source** (same content on output-A vs output-B must match; paused $\Rightarrow$ counter stops; $2\times$ speed $\Rightarrow \sim 2\times$ advance rate); (9) **full-reference quality metrics (SSIM/VMAF/ ΔE_{2000}) vs a golden source for owned content**; (10) **mutation-test every analyzer with a golden-good + golden-bad fixture pair** so the analyzer itself provably cannot bluff (an analyzer that PASSES its golden-bad fixture is a bluff gate — §1.1 applied to the analyzers); (11) **per-channel audio RMS/loudness (EBU R128) + XRUN/underrun census** — a single aggregate RMS misses a dead channel; (12) **OCR overlay/subtitle detection needs a per-word confidence floor + ROI** to avoid BOTH false-positive (false-FAIL) and false-negative (false-PASS); (13) **thresholds calibrated on the project's own fixtures, not hardcoded from literature** (§11.4.6 no-guessing).

Honest gap (§11.4.6): true photon FPS at the sink has no clean software oracle — compositor/decoder counters measure the presentation pipeline; flag the gap, never claim it.

4-layer coverage per §11.4.4(b): pre-build gate (a `CM-AV-LIVENESS-NO-FROZEN-FRAME` -class gate asserting every output-is-playing test references the liveness battery not a single frame + an analyzer-self-validation gate wiring the golden-good/golden-bad fixtures into meta-test) + on-device/runtime test + paired §1.1 meta-test

mutation (single-frame-only assertion → gate FAILs; analyzer that PASSes its golden-bad fixture → self-validation FAILs) + HelixQA Challenge. Every PASS via `ab_pass_with_evidence` citing the motion / not-stale / frame-advance / fps / per-channel-loudness / metamorphic / self-validation artefacts (`video_display` / `audio_output` / `subtitle_render` per §11.4.69) — never a single screenshot.

Classification: universal (§11.4.17) — platform-neutral AV/test-validation techniques reusable by ANY project validating media playback or any pixel/audio output; the project supplies its concrete capture mechanism + calibrated thresholds per §11.4.35. Composes with §11.4.5 (its strict expansion), §11.4.6, §11.4.50, §11.4.68, §11.4.69, §11.4.85, plus §11.4.2 / §11.4.3 / §11.4.13 / §11.4.48 / §11.4.52 / §11.1. Propagation gate `CM-COVENANT-114-107-PROPAGATION` enforces the literal anchor `11.4.107` across the consumer fleet; paired §1.1 meta-test mutation strips the literal → gate FAILs (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.107.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--single-frame-proves-playback`, `--skip-liveness`, `--byte-identical-freeze-OK`, `--no-frame-advance-counter`, `--skip-not-stale-check`, `--allow-wrong-output-flash`, `--deep-link-shortcut-OK`, `--unvalidated-analyzer-OK`, `--aggregate-rms-suffices`, `--hardcoded-thresholds-OK` flag exists.

§11.4.108 — Four-layer fix-verification + runtime-signature-as-definition-of-done mandate (systematic-debugging Phase 4.5, 2026-06-03)

Forensic anchor (genericised, 2026-06-03): across one batch, multiple fixes were “green” at every gate yet NOT working for the end user — a change present in source and passed by both the source pre-build gate AND the post-build gate never reached the boot-time command-line embedded in the deployed boot artifact (output feature dead despite all-green); sibling fixes correctly built into the system image were masked by stale per-user overlay copies from a previous deployment (the running code was the stale shadow, not the fresh build); deployment did not wipe the mutable overlay so the shadow survived; validation ran against whatever was running — the stale shadow — and reported green on code that was never exercised. Per `superpowers:systematic-debugging` Phase 4.5: each fix revealing a fresh “fixed-but-not-working” in a DIFFERENT place is NOT

independent bugs — it is ONE architectural VERIFICATION flaw; patching each symptom is thrashing.

A fix crosses FOUR distinct layers, and “fixed” at one does NOT imply fixed at the next: (1) **SOURCE** (committed in the source file — what a grep-the-source pre-build gate checks), (2) **ARTIFACT** (the change’s BYTES actually landed in the produced build artifact — image / bundle / installer / embedded command-line), (3) **RUNTIME-ON-CLEAN-TARGET** (active on a CLEAN/fresh deployment — the layer the end user experiences — with no stale overlay shadowing the deployed code), (4) **USER-VISIBLE** (the feature works for the end user — the §11.4.5/§11.4.69 captured-evidence layer). Green at layer 1 is the cheapest, least conclusive signal and says nothing about layers 2–4. A gate verifying ONLY the source layer is itself a §11.4 bluff surface.

The mandate (ALL must hold): (1) **Runtime-signature-as-definition-of-done** — a fix is DONE only when its declared runtime signature verifies on a CLEAN/fresh deployment; source-committed ≠ artifact-contains-it ≠ active-on-clean-target ≠ user-visible-working. (2) **Every fix declares ONE machine-checkable runtime signature** — a single observable on a clean target proving the fix is BOTH active AND working (a running-system property, a downstream/sink-side report per §11.4.13, a §11.4.5/§11.4.69 captured-evidence assertion, or a counter/state delta — NEVER a re-grep of the source); this **registry of per-fix runtime signatures is the SINGLE SOURCE OF TRUTH for “fixed”** — it REPLACES “a gate greps the source” as the definition of done. (3) **Gates span all four layers** — source (pre-build), artifact (post-build — assert the change’s BYTES landed in the artifact, not merely that the source still contains the change), runtime-on-clean-target (post-deploy — assert the runtime signature on a freshly-deployed clean target), user-visible (§11.4.5/§11.4.69). (4) **Eliminate the stale-deployment / shadow layer by construction** — deployment MUST yield a CLEAN state (wipe the mutable overlay) OR a pre-validation assertion MUST prove `running-artifact == built-artifact` BEFORE any validation runs; validation against possibly-stale deployed state is INVALID and any PASS it produces is a §11.4 PASS-bluff (the test exercised code that was never deployed). (5) **Meta-rule** — ≥ 3 “fixed-but-not-working” discoveries in one cycle signal an architectural VERIFICATION flaw, NOT three independent bugs; on the 3rd, STOP patching symptoms (§11.4.4), fix the VERIFICATION pipeline, and re-certify EVERY item through it on a clean target. (6) **A batch is “validated” only after COMPREHENSIVE per-item runtime-signature verification on a clean baseline** — NOT after the touched items’ own tests pass.

Classification: universal (§11.4.17) — platform-neutral verification-pipeline disciplines reusable by ANY project that builds an artifact, deploys it, and validates the deployed result; the project supplies its concrete artifact format, clean-deployment / equal-artifact mechanism, and per-fix runtime-signature observables per §11.4.35. Composes with §11.4.1 / §11.4.2 / §11.4.4 (clause 5's STOP-and-fix-pipeline is its §11.4.108 specialisation; "clean baseline" = clause 4's clean deployment) / §11.4.5 / §11.4.6 (every layer asserted by evidence, never assumed to propagate) / §11.4.27 / §11.4.40 (§11.4.108 adds the cross-layer per-item runtime-signature dimension) / §11.4.46 (clean-baseline / equal-artifact pre-flight) / §11.4.50 / §11.4.52 / §11.4.69 (a runtime signature is a taxonomy-class observable) / §11.4.102 (Phase 4.5 architectural-flaw recognition IS clause 5's trigger). Propagation gate `CM-COVENANT-114-108-PROPAGATION` (literal `11.4.108` across the consumer fleet) + recommended per-fix gate `CM-RUNTIME-SIGNATURE-REGISTRY` + paired §1.1 meta-test mutations (strip the literal → propagation gate FAILs; downgrade a fix to source-only verification → registry gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.108.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--source-green-is-done` , `--skip-artifact-byte-check` , `--validate-against-running-state` , `--no-clean-deployment` , `--skip-runtime-signature` , `--spot-validate-touched-only` flag exists.

§11.4.109 — Mandatory Anti-Forgetting Enforcement: PreToolUse Guard Hook + Subagent Constitutional Preamble + Orchestrator Pre-Action Checklist (Operator mandate)

Short tag: `anti-forgetting-enforcement` . **UNCONFIRMED forensic anchor** (pending operator's verbatim mandate quote). Background: emulator subagents ran raw host-direct `emulator` / `adb` because the orchestrator forgot to inject the Containers-submodule rule. A rule forgotten at dispatch is not enforcement. Fix: (A) a `PreToolUse` guard hook (`constitution/scripts/hooks/guard-forbidden-commands.sh`) that blocks host-direct emulator, force-push/bypass, sudo, and host-power commands at the tool-call boundary regardless of agent memory; (B) a canonical `docs/AGENT_GUARDRAILS.md` preamble the orchestrator pastes verbatim

into every subagent dispatch; (C) an **ORCHESTRATOR PRE-ACTION CHECKLIST** in the same document. Hook = the floor; preamble = the ceiling.

Consuming projects MUST: (1) wire `constitution/scripts/hooks/guard-forbidden-commands.sh` as a `PreToolUse` hook in `.claude/settings.json` (or equivalent runtime settings); (2) maintain `docs/AGENT_GUARDRAILS.md` containing the `SUBAGENT CONSTITUTIONAL PREAMBLE` and `ORCHESTRATOR PRE-ACTION CHECKLIST` headings, with the anchor literal `11.4.109`; (3) provide a hermetic hook test suite (≥ 20 cases: every blocked class exits 2, every allowed command exits 0, escape hatch fires for non-power classes, host-power rejects even with escape marker). The hook is inherited by reference — NEVER copied locally (a copy diverges silently).

Gates: `CM-ANTI-FORGETTING-ENFORCEMENT` (hook present + wired + guardrails doc present + test present) + `CM-COVENANT-114-109-PROPAGATION` (anchor literal `11.4.109` across every consumer `CLAUDE.md` / `AGENTS.md` / `QWEN.md`). Paired §1.1 mutations: remove hook entry → gate (2) FAILs; delete guardrails doc → gate (3) FAILs; strip hook from constitution → gate (1) FAILs; strip `11.4.109` → propagation gate FAILs.

Classification: universal (§11.4.17). Composes with §11.4.6 / §11.4.10 / §11.4.75 / §11.4.76 / §11.4.78 / §11.4.79 / §11.4.80 / §11.4.81 / §11.4.84 / §11.4.98 / §11.4.102 / §12.

Canonical authority: constitution submodule `Constitution.md` §11.4.109; reference implementation `constitution/scripts/hooks/guard-forbidden-commands.sh` + `constitution/docs/AGENT_GUARDRAILS.md`.

Non-compliance is a release blocker. No escape hatch — no `--skip-pretooluse-hook`, `--no-guardrails-doc`, `--anti-forgetting-optional`, `--single-layer-sufficient` flag exists.

§11.4.110 — Pre-build build-readiness verdict + change-impact clash detection mandate (operator mandate, 2026-06-03)

Forensic anchor (genericised, 2026-06-03): a fix shipped a new system-property *read* but introduced no matching security-policy grant + no property-context type entry — the read was silently denied at runtime + the feature was dead, while every pre-build gate stayed green because the gate only grepped the source file. The

defect class generalises: a change introduces a new dependency on a *second* artifact (security policy, context file, service registry, interface freeze-snapshot, symbol table, build-graph node) that the pre-build never cross-checks. This is §11.4.108's SOURCE → ARTIFACT gap shifted LEFT to pre-build time — most such clashes are statically catchable from the change diff itself, before any build. The mandate (ALL hold): (1) a single deterministic **READY-FOR-BUILD verdict** gates the rebuild (orchestrator refuses to start the build unless READY); (2) a **diff-driven change-impact + clash detector** cross-checks every newly-introduced second-artifact dependency (new property read ⇔ property-context type + read-grant; new service ⇔ service-context entry; new init service ⇔ security label; new/changed stable interface ⇔ freeze-snapshot updated; new native-lib dep ⇔ module/prebuilt resolves; new policy rule ⇔ every type/attribute defined; two batch changes on the same seam ⇔ collision acknowledged); (3) **coverage-completeness is a gate** — every changed file maps to ≥1 gate + ≥1 deployed-target test + ≥1 paired §1.1 mutation, baseline ratchets upward per §11.4.50; (4) **two-speed honesty** — grep-speed always-on gates vs REQUIRES_BUILD heavy gates (build-graph parse-only dry-run, full neverallow compilation, ABI diff) as diff-gated opt-in stages, bounded per §12.6/§12.7; (5) every gate + wired analyzer is **anti-bluff by paired §1.1 mutation**; (6) **honest boundary** — a READY verdict proves static internal-consistency + ready-to-build, NOT that the feature works on the deployed target; the regime empties the *preventable* defect class, not the *all-defects* class (runtime/USER-VISIBLE remains §11.4.108's job).

Classification: universal (§11.4.17) — platform-neutral pre-build-rigor disciplines reusable by ANY project that builds an artifact from source; the consuming project supplies its concrete property/service/policy registries, build-graph parse-only command, interface-freeze mechanism, and changed-file → gate mapping per §11.4.35. Composes with §11.4.1 / §11.4.4 / §11.4.6 / §11.4.9 / §11.4.27 / §11.4.50 / §11.4.67 / §11.4.75 / §11.4.92 / §11.4.108 (§11.4.110 is the SOURCE → ARTIFACT half shifted left to pre-build; §11.4.108 owns RUNTIME-ON-CLEAN-TARGET → USER-VISIBLE — together they span all four layers). Propagation gate `CM-COVENANT-114-110-PROPAGATION` (literal `11.4.110`) + recommended per-family gates `CM-READY-FOR-BUILD-VERDICT` / `CM-CHANGE-IMPACT-CLASH-DETECTOR` / `CM-COVERAGE-COMPLETENESS-GATE` / `CM-BUILDGRAPH-DRYRUN-WIRED` / `CM-SEPOLICY-NEVERALLOW-WIRED` + paired §1.1 mutations (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.110. Non-compliance is a release blocker. No escape hatch — no `--source-green-is-`

`ready` , `--skip-clash-detector` , `--skip-coverage-gate` , `--no-ready-verdict` , `--grep-proves-neverallow` , `--skip-buildgraph-dryrun` , `--build-without-ready-verdict` flag.

§11.4.111 — Resolve-by-stable-name-not-by-enumeration-index mandate (research-derived, 2026-06-03)

Forensic anchor (genericised, 2026-06-03): a platform bound an audio output to a kernel-enumerated device index (`card=0`); a second device of a different class enumerated FIRST at boot and took slot 0, shifting the intended device to slot 1 — the static index binding now pointed at the wrong device (policy mis-attached the sink, mis-assigned its TYPE, the output switcher labelled the AV receiver “Wired Headphone” + routing collapsed to stereo). The lower layer of the SAME stack already resolved the device correctly **by name** (scanning the controller-name registry) — proving the brittleness was the *index* binding, not the resolution capability. Generalises to every enumerated resource whose ordinal is assigned at discovery/boot/hotplug time (non-deterministic across reboots, device additions, topology changes). The mandate: any binding to a hardware device / resource handle / enumerated entity (audio cards, display connectors, network interfaces, storage devices, GPU render nodes, input/camera devices, container/process slots) MUST resolve by a **stable identifier** (name / UUID / serial / label / controller-name / content-hash / sink-reported identity) and MUST NOT bind by enumeration index / ordinal / slot, UNLESS the platform documents that ordinal as deterministically pinned AND the pin is itself captured + asserted as part of the binding. Where a stable identifier exists at one layer, every other layer binding the same resource MUST use the same identifier (mixed by-name-here / by-index-there is the structural weak link forbidden here). Honest boundary (§11.4.6): when only an ordinal exists, pin it deterministically via the platform’s own mechanism, capture the pin in the binding’s §11.4.108 runtime signature, and document the residual fragility as `UNCONFIRMED: -class risk` — never silently trust an unpinned ordinal.

Classification: universal (§11.4.17) — platform-neutral binding-robustness discipline reusable by ANY project binding to enumerated hardware/resources/handles; the consuming project supplies its stable-identifier mechanism (ALSA card name, DRM connector name, NIC predictable name, block-device UUID, etc.) + the layers that must agree per §11.4.35. Composes with §11.4.6 (no-guessing — “the index is *usually* stable” is the exact guess forbidden) / §11.4.8 (mature stacks resolve by name/UUID — reproducing a known-brittle index binding when the by-name path is

documented is a §11.4.8 omission) / §11.4.69 (sink-side evidence verifies the by-name binding's correctness) / §11.4.108 (the by-name binding's runtime signature asserted on a clean target across the topology that broke the index) / §11.4.110 (a new ordinal binding in a diff is a statically-catchable clash class). Propagation gate CM-COVENANT-114-111-PROPAGATION (literal 11.4.111) + recommended gate CM-RESOLVE-BY-NAME-NOT-INDEX + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md §11.4.111. Non-compliance is a release blocker. No escape hatch — no --allow-index-binding , --ordinal-is-stable-enough , --skip-name-resolution , --trust-unpinned-index flag.

§11.4.112 — Structural-impossibility won't-fix classification mandate (research-derived, 2026-06-03)

Forensic anchor (genericised, 2026-06-03): deep research (§11.4.8) into relocating protected (content-protection / secure-surface) video to a secondary display PROVED — from authoritative platform/HDCP docs + reproducible captured behaviour (a secure surface is blanked on any output lacking the secure flag; mirror/screenshot of a secure layer returns black) — that the goal is **structurally impossible by platform design**, not a missing feature or unsolved engineering problem. Without a durable classification, such a goal is re-investigated every cycle (re-read the same sources, re-run the same probes, re-derive the same impossibility — compounding wasted effort). The mandate: when deep research per §11.4.8 PROVES (cited authoritative sources AND, where applicable, reproducible captured evidence) a goal is structurally impossible on the target platform (forbidden by platform design / hardware-protocol constraint / documented kernel-or-API limitation — NOT merely unimplemented or hard), the goal MUST be: (1) classified won't-fix + closed per §11.4.90 with closure reason structurally-impossible ; (2) documented with the impossibility evidence — cited authoritative source URLs (per §11.4.99 latest-source verification) + the reproducible probe/captured evidence — in the tracker entry + relevant docs/guide; (3) NOT re-attempted in future cycles — a reopen MUST cite NEW evidence the platform constraint changed (per §11.4.34 + §11.4.7), never merely re-derive the same impossibility; (4) paired with the correct posture — the entry states what the project DOES instead, so “impossible” is never confused with “broken/unhandled”. Honest boundary (§11.4.6): structurally-impossible is reserved for *proven*

platform/hardware/protocol impossibility — “could not find a way” / “very hard” / “no time” are `operator-blocked` (§11.4.21) or open work, NOT won't-fix; mislabelling them to avoid the work is a §11.4 planning-layer bluff. A future platform change can make the impossible possible; the classification is durable but not eternal.

Classification: universal (§11.4.17) — platform-neutral effort-conservation + honesty discipline reusable by ANY project; the consuming project supplies the specific impossible goal, its platform constraint, and the cited evidence per §11.4.35.

Composes with §11.4.6 (FACT with cited evidence, never “probably can't”) / §11.4.7 (reopening requires NEW positive evidence) / §11.4.8 (“NO external solution found — structurally impossible” is the citation) / §11.4.34 (reopen attribution) / §11.4.90 (`structurally-impossible` is a closure reason in the closed vocabulary) / §11.4.99 (latest-source so the verdict is not stale). Propagation gate `CM-COVENANT-114-112-PROPAGATION` (literal `11.4.112`) + recommended gate `CM-WONT-FIX-STRUCTURAL-IMPOSSIBILITY` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.112. Non-compliance is a release blocker. No escape hatch — no `--wont-fix-without-proof` , `--reattempt-closed-impossible` , `--skip-impossibility-evidence` , `--impossible-equals-broken` flag.

§11.4.113 — Absolute no-force-push + merge-onto-latest-main mandate (User mandate, 2026-06-03)

Forensic anchor — verbatim user mandate (2026-06-03): “Any force-push is strictly forbidden! We must for every Submodule take as a base latest commit on Submodule’s main (or master) branch, then on top of it carefully to merge all changes that have to be pushed! Once all merging is carefully done we perform commit and push to all Submodule’s upstreams!”

Force-push is STRICTLY FORBIDDEN with NO exception — `git push --force` , `--force-with-lease` , `+<ref>` , or any history-rewriting overwrite of a remote ref, against EVERY repository this Constitution governs (main repo, this constitution submodule, every owned + nested submodule, every upstream). No operator-approval path, no “after a merge-first audit” path. The mandated 6-step integration procedure for any repo/submodule whose local has commits to publish OR whose mirrors diverged: (1) `git fetch --all --prune --tags` all remotes; (2) set the base to the LATEST commit on the canonical `main` / `master` branch (the most-

advanced mirror tip); (3) carefully MERGE every change to be published on top of that base — union, preserve BOTH sides, NEVER `-s ours` / rebase / reset that drops commits (per §9 no-commit-loss); (4) resolve every conflict carefully — no conflict markers, no file dropped, gates/tests still pass; (5) commit the merge (stage only intended files, NEVER `git add -A` in a submodule per §11.4.30); (6) push to ALL upstreams — each push is a fast-forward because the merge commit descends from every mirror tip, so NO force is ever needed (if an upstream still rejects, return to step 1 for it, merge its new tip, re-validate, re-push). **TIGHTENS §11.4.41 / §11.4.71 / §9.2 / CONST-043 — the force-push escape hatch is REMOVED:** even WITH operator approval, even after a clean merge-first audit, force-push is forbidden, because the merge-onto-latest-main path is always available so force is never necessary. Those clauses' merge-first/fetch-first machinery stays in force as the integration discipline; only their terminal "...then force-push" step is struck.

Classification: universal (§11.4.17) — a platform-neutral integration discipline reusable across every repository. Composes with §2.1 (multi-upstream push — step 6 fans out) / §9 / §9.2 (absolute data safety — this is the no-loss push discipline that makes force unnecessary) / §11.4.4 / §11.4.6 (remote state unknowable without the step-1 fetch) / §11.4.26 (constitution-update conflict resolution IS this procedure) / §11.4.37 (fetch-before-edit → fetch-before-push) / §11.4.40 / §11.4.41 (merge-first stays, force-push step removed) / §11.4.71 (same tightening) / §11.4.88 (background-push still fast-forward-only) / CONST-043 (no force-push authorisable). Propagation gate `CM-COVENANT-114-113-PROPAGATION` (literal `11.4.113`) + recommended gate `CM-NO-FORCE-PUSH-ABSOLUTE` (scan tracked scripts/hooks for `push --force` / `--force-with-lease` / `push +<ref>` + reject; a §11.4.109-class `PreToolUse` guard blocks the class at the tool-call boundary) + paired §1.1 mutation (inject a `git push --force` into a tracked script → gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.113. Non-compliance is a release blocker. No escape hatch — no `--force` , `--force-with-lease` , `--allow-force-push` , `--force-push-authorised` , `--skip-merge-onto-main` flag.

§11.4.114 — Last-known-good-tag regression isolation mandate (1.1.8-dev remediation, 2026-06-03)

When a previously-working feature/behaviour is observed broken, the FIRST diagnostic action MUST be to identify the last release tag (or commit/build/deploy)

at which it was KNOWN-GOOD and diff/bisect the broken state against it — BEFORE any open-ended root-cause hunt or speculative fix. The known-good revision is the regression oracle: it bounds the search to the commits between good and now (`git diff <good-tag>..HEAD --stat` of the feature's files = captured evidence), tells you it is a regression REPAIR not a from-scratch design problem, and gives each suspect file a behavioural oracle. When the operator volunteers a known-good tag, that lead is load-bearing and MUST be acted on first. Default to a SURGICAL forward-fix (keep the post-good-tag features, revert ONLY the broken sub-part) over a wholesale revert that loses the batch's other working features — unless the operator prefers wholesale revert. Honest boundary (§11.4.6): “it worked before” is a HYPOTHESIS until the known-good tag is identified AND the feature is confirmed working there; “probably regressed in the last batch” without the diff is a guess; a feature that NEVER worked is not a regression. Composes §11.4.4 / §11.4.6 / §11.4.7 / §11.4.40 / §11.4.43 / §11.4.102 / §11.4.108. Propagation gate `CM-COVENANT-114-114-PROPAGATION` (literal `11.4.114`) + recommended gate `CM-REGRESSION-ISOLATED-AGAINST-KNOWN-GOOD` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.114. Non-compliance is a release blocker. No escape hatch — no `--skip-known-good-diff` , `--root-cause-from-scratch` , `--assume-regression-source` , `--wholesale-revert-without-isolation` flag.

§11.4.115 — RED-baseline-on-the-broken-artifact + polarity-switch mandate (1.1.8-dev remediation, 2026-06-03)

Strict refinement of §11.4.43's RED step. Every RED test MUST be authored to REPRODUCE the defect on the CURRENT, pre-fix artifact (the actual broken build/deployment), capturing positive evidence per §11.4.5 / §11.4.69 / §11.4.107 that the defect is genuinely present — never a synthetic failure the fix is then written to agree with. The SAME test source MUST carry a single polarity switch (env flag / parameter, canonical `RED_MODE` , default `1` = reproduce-and-assert-defect-present) that flipped to `0` post-fix converts the test into the GREEN regression-guard asserting the defect is ABSENT. One source, two roles: the bug-catcher IS the regression-guard; no separate happy-path test is authored as the primary guard (that demonstrates only that the test agrees with the fix — the §11.4.43 PASS-bluff). RED-on-broken-artifact then GREEN-on-fixed-artifact (on a clean target per §11.4.108) MUST both be captured. Honest boundary (§11.4.6): if the RED run

does NOT fail on the broken artifact, that is a finding (close per §11.4.7 with negative evidence, or fix the test) — a RED test that passes on the known-broken artifact is a blind test. Composes §11.4.1 / §11.4.2 / §11.4.5 / §11.4.69 / §11.4.107 / §11.4.4 / §11.4.7 / §11.4.43 / §11.4.50 / §11.4.108 / §11.4.114. Propagation gate `CM-COVENANT-114-115-PROPAGATION` (literal `11.4.115`) + recommended gate `CM-RED-POLARITY-SWITCH-PRESENT` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.115. Non-compliance is a release blocker. No escape hatch — no `--green-only-test`, `--skip-red-reproduction`, `--separate-happy-path-suffices`, `--synthetic-red-OK` flag.

§11.4.116 — Real-time conductor↔autonomous-test-framework sync channel mandate (1.1.8-dev remediation, 2026-06-03)

Any autonomous, long-running test/QA/validation framework an external orchestrator (conductor agent / operator) depends on for real-time decisions MUST expose a real-time sync channel: (1) a structured append-only event stream (JSONL or equivalent, one event per line, never rewritten) emitting at minimum session-start / phase-transition / per-test-or-challenge-start / captured-evidence-path / external-call (LLM / vision / sink-probe) / error / per-item-verdict events; (2) an atomically-rewritten status snapshot (single small file written write-temp-then-rename so a reader never sees a torn write) carrying current session/phase/item + counters + last verdict. Verdicts use the closed vocabulary PASS / FAIL / SKIP / OPERATOR-BLOCKED (§11.4.45). The conductor tails it live so it stays in real-time sync, can §11.4.4-interrupt on a fresh defect, and never idles blindly (§11.4.94 / §11.4.97). Anti-bluff: a verdict event MUST carry the evidence path that backs it (§11.4.69) — a PASS event with no evidence path is a channel-layer PASS-bluff; a snapshot reporting PASS while the stream shows no evidence event for that item is a contradiction → treat as FAIL; an item with no start-event cannot have a verdict-event. When the framework is an owned project-agnostic submodule (§11.4.28), the channel stays project-neutral — the consumer registers its data (endpoints, package ids, sink hosts) at runtime via the public API, never hardcoded. Composes §11.4.4 / §11.4.5 / §11.4.69 / §11.4.27 / §11.4.28 / §11.4.45 / §11.4.52 / §11.4.89 / §11.4.94 / §11.4.97. Propagation gate `CM-COVENANT-114-116-PROPAGATION` (literal

11.4.116) + recommended gate CM-AUTONOMOUS-FRAMEWORK-SYNC-CHANNEL + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md §11.4.116. Non-compliance is a release blocker. No escape hatch — no --no-sync-channel , --end-of-session-report-suffices , --verdict-without-evidence-path , --torn-status-write-OK flag.

§11.4.117 — Computer-vision / OCR pixel-oracle fallback for non-introspectable UIs mandate (1.1.8-dev remediation, 2026-06-03)

Any test that needs to drive a UI control OR assert on-screen content MUST NOT assume the accessibility/semantic/DOM hierarchy is the source of truth for what the user sees. When the hierarchy is blank/partial/known-unreliable for the app under test (TV-Compose, leanback, canvas/ SurfaceView /GL, games, custom-rendered UIs), the test MUST fall back to a PIXEL ORACLE: (1) DRIVE input by computer-vision template-match (locate a control by its rendered appearance → tap its screen coordinates), not by a hierarchy node id; (2) ASSERT content by ROI OCR (read the rendered text the user reads — caption strip, title, error overlay) with a per-word confidence floor + region-of-interest per §11.4.107(12), not a hierarchy text attribute that may be absent/stale. The tool MUST both drive input AND read pixels — a hierarchy-only tool is NOT a content oracle. This makes §11.4.52's "near-empty hierarchy → INFEASIBLE" constructive: pixel-drive, not only SKIP. Anti-bluff (§11.4.107(10)): the CV/OCR analyzer is self-validated — golden-good fixture PASSES, golden-bad fixture FAILs, wired into meta-test; thresholds calibrated on the project's own frames, not hardcoded (§11.4.6). Honest boundary: when BOTH hierarchy is blank AND pixel oracle is infeasible (secure surface black-captures per §11.4.112, geo-unreachable), SKIP-with-reason per §11.4.3 + tracked operator-attended migration item per §11.4.52 — never fake PASS. Composes §11.4.5 / §11.4.6 / §11.4.48 / §11.4.49 / §11.4.52 / §11.4.107 / §11.4.112. Propagation gate CM-COVENANT-114-117-PROPAGATION (literal 11.4.117) + recommended gate CM-CV-OCR-PIXEL-ORACLE-FALLBACK + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md §11.4.117. Non-compliance is a release blocker. No escape hatch — no

`--hierarchy-is-content-oracle` , `--skip-pixel-fallback` , `--unvalidated-ocr-OK` , `--hardcoded-ocr-threshold-OK` flag.

§11.4.118 — Discovery-pressure to confirm known-issue-set completeness mandate (1.1.8-dev remediation, 2026-06-03)

A remediation/release cycle MUST NOT treat “every reported defect is fixed” as “the build is good” — the reported set is biased by what the operator happened to test (“we only see what we test”). After/alongside fixing the reported set, the cycle MUST run a discovery + stress pass across ALL target devices/environments that deliberately exercises subsystems, journeys, and edge cases BEYOND the reported defects — to CONFIRM the reported set is the COMPLETE critical set and surface unreported defects before the end user does. The pass MUST produce PROVABLE coverage: an enumerated list of the subsystems/user-journeys/stress scenarios actually exercised, each with its outcome (no-new-issue, or a new tracker entry per §11.4.15 / §11.4.16). “We found no other issues” is a §11.4 bluff unless accompanied by “here is the enumerated set we exercised” — absence of evidence of looking is not evidence of absence. New findings trigger §11.4.4 interrupt + the §11.4.114/§11.4.115 isolation → RED → fix loop. Honest boundary (§11.4.6): discovery reduces the unknown-unknown surface but does not prove zero remaining defects — the earned claim is “reported set + enumerated discovery set addressed,” with un-exercised subsystems stated as honest coverage gaps (§11.4.3), never silently implied clean. Composes §11.4.4 / §11.4.5 / §11.4.69 / §11.4.25 / §11.4.40 / §11.4.42 / §11.4.52 / §11.4.85 / §11.4.114 / §11.4.115 / §11.4.119. Propagation gate `CM-COVENANT-114-118-PROPAGATION` (literal `11.4.118`) + recommended gate `CM-DISCOVERY-COVERAGE-ENUMERATED` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.118. Non-compliance is a release blocker. No escape hatch — no `--reported-set-suffices` , `--skip-discovery-pass` , `--no-issues-without-coverage-list` , `--assume-complete` flag.

§11.4.119 — Single-resource-owner partitioning for parallel hardware testing mandate (1.1.8-dev remediation, 2026-06-03)

Strict refinement of §11.4.58 / §11.4.103 for hardware contention. When multiple parallel work/test/discovery streams exercise SHARED hardware or any exclusive-access resource (a media-playback path, a single HDMI/audio sink, an exclusive device handle, a serial/JTAG line, a GPU under exclusive capture), exactly ONE stream MUST own each such resource at a time. The exclusive owner drives it (playback, input injection, capture); every other concurrent stream targeting the same resource MUST be READ-ONLY (passive probes — `dumpsys` / `/proc` / `/sys` reads / sink-side network probes / log tails). Parallelism is partitioned by resource: distinct devices/sinks/handles run fully concurrent (stream-per-device), but the same device's exclusive resource is single-owner. Ownership MUST be enforced by an advisory lock/token (§11.4.58 L3 + the hardware analogue of §11.4.84 quiescence), event-driven (claim when the resource frees, release the moment work completes so the next queued stream claims it per §11.4.103 auto-backfill). Why: concurrent drivers of one exclusive resource produce CROSS-CONTAMINATED evidence (sink reports the wrong stream's audio, foreground belongs to whichever `am start` landed last, input events interleave) — a PASS under contention is a §11.4 evidence-integrity bluff. Honest boundary (§11.4.6): a “read-only” probe stream MUST issue NO state-changing command against a device it does not own; when a resource genuinely cannot be partitioned, streams serialize on it (single-owner over time), not run concurrently and hope. Composes §11.4.5 / §11.4.69 / §11.4.13 / §11.4.50 / §11.4.58 / §11.4.82 / §11.4.84 / §11.4.103 / §11.4.118. Propagation gate `CM-COVENANT-114-119-PROPAGATION` (literal `11.4.119`) + recommended gate `CM-SINGLE-RESOURCE-OWNER-PARTITION` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.119. Non-compliance is a release blocker. No escape hatch — no `--allow-concurrent-resource-drivers`, `--skip-ownership-lock`, `--read-only-may-mutate`, `--contended-evidence-OK` flag.

§11.4.120 — Fix-breaks-its-own-gate reconciliation mandate (1.1.8-dev remediation, 2026-06-03)

When a correct fix causes a pre-existing gate/test to FAIL because that gate asserted the OLD (now-removed-or-changed) behaviour, the gate FAIL is the CORRECT signal the fix landed — it MUST NOT be suppressed by either forbidden response: (1) FAKE-PASSING the gate (editing it to `always pass`, weakening its assertion to a tautology, or deleting it — a §11.4 gate-layer bluff + a §11.4.84 mutation-residue risk); (2) REVERTING the correct fix to satisfy the stale gate (re-introducing the defect). The required response is RECONCILIATION: rewrite the gate to assert the NEW mechanism the fix introduced, backed by captured evidence of the new correct behaviour, AND update its paired §1.1 mutation so the mutation breaks the NEW invariant. Discriminator vs bluffing: after reconciliation the gate + mutation still form a valid §1.1 pair (mutate the new invariant → gate FAILs); a bluffed gate's mutation no longer makes it FAIL (the assertion became a tautology). The reconciliation MUST be a visible, evidence-cited change, never a silent assertion-weakening. Honest boundary (§11.4.6): a post-fix gate FAIL is NOT automatically “stale gate, reconcile” — investigate per §11.4.102 first; the FAIL may be the gate correctly catching a REGRESSION the fix introduced, in which case the FIX is wrong, not the gate. Reconcile ONLY when investigation PROVES the gate asserted old-correct-now-removed behaviour AND the new behaviour is the intended, evidence-confirmed mechanism. Composes §11.4.1 / §11.4.4 / §11.4.6 / §11.4.84 / §11.4.102 / §11.4.108 / §1.1. Propagation gate `CM-COVENANT-114-120-PROPAGATION` (literal `11.4.120`) + recommended gate `CM-GATE-RECONCILED-NOT-FAKE-PASSED` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.120. Non-compliance is a release blocker. No escape hatch — no `--fake-pass-stale-gate`, `--revert-fix-for-gate`, `--weaken-assertion-to-pass`, `--delete-failing-gate` flag.

§11.4.121 — No-commit-while-build-writes-tracked-artifacts mandate (1.1.8-dev remediation, 2026-06-03)

A commit (especially `git add -A` / any broad stage) MUST NOT run while a build/packaging/generation step is actively writing artifacts INTO tracked (version-controlled) directories — doing so races the writer and stages a PARTIAL or stale

artifact (a §11.4.108 SOURCE → ARTIFACT integrity failure landed in version control). The commit MUST be deferred until the build step that writes tracked artifacts has COMPLETED, so the tree is quiescent at the artifact layer AND the committed artifacts are the FRESH, whole outputs (no stale pre-rebuild artifact, no half-written file). Before committing tracked build outputs, verify the writing step finished (process exit / completion marker / per-artifact mtime ≥ build-start) — a build still in flight writing tracked dirs is a HOLD on the commit, not a race to win. Build-output analogue of §11.4.84 (no commit while a mutation gate is in flight): both close the “commit captures transient non-final tree state” class at two write-sources. Where build outputs land OUTSIDE version control (gitignored `out/` / `dist/`), the race does not apply. Honest boundary (§11.4.6): “the build probably finished” is not “the build finished” — verify with a completion signal; committing source changes that DON’T touch the build’s tracked-artifact directories is fine mid-build. The PROJECT-SPECIFIC instance (which tracked directory + which build steps write it) is recorded in the consuming project’s own governance per §11.4.35. Composes §11.4.6 / §11.4.30 / §11.4.58 / §11.4.84 / §11.4.88 / §11.4.96 / §11.4.103 / §11.4.108. Propagation gate `CM-COVENANT-114-121-PROPAGATION` (literal `11.4.121`) + recommended gate `CM-NO-COMMIT-DURING-ARTIFACT-WRITE` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.121. Non-compliance is a release blocker. No escape hatch — no `--commit-during-build`, `--stage-partial-artifact-OK`, `--assume-build-finished`, `--skip-build-completion-check` flag.

§11.4.122 — No-silent-removal-of-existing-components-without-operator-confirmation mandate (User mandate, 2026-06-03)

Forensic anchor — verbatim user mandate (2026-06-03):

“Never ever remove any application, system component or service from already existing codebase / System without interactively asked question to us! THIS IS MANDATORY RULE / CONSTRAINT!”

Forensic case study (FACT). During the 1.1.8-dev burn-down, two shipped capabilities — F2 (an Apple-TV-class application) and F4 (a Huawei HMS / Mobile-

Services component) — were removed from the existing System WITHOUT first asking the operator; the operator reversed both. A removal the operator has to discover and reverse after the fact is a defect of the same severity class as a §11.4 PASS-bluff: the System silently lost a user-facing capability the operator never agreed to drop.

No application, system component, service, package, feature, driver, module, library, prebuilt asset — any already-existing end-user capability of the existing codebase / shipped System — may be removed (deleted, dropped from the package set, disabled-into-non-shipping, un-bundled, de-listed, or otherwise made unavailable to the end user) WITHOUT FIRST interactively asking the operator and receiving an EXPLICIT keep-or-remove decision. The question MUST be posed through the platform's interactive clarification mechanism per §11.4.66 (AskUserQuestion on Claude Code) — NEVER a free-text “should I remove X?” buried in narrative, NEVER a silent removal justified post-hoc, NEVER an autonomous removal decision. A silent removal is a **release blocker** regardless of how well-intentioned the rationale (deduplication, “it was broken anyway”, geo-restricted, incompatible, superseded) — the operator decides, the agent asks.

What counts as a removal (non-exhaustive): deleting an app/APK/binary from the build's package set (PRODUCT_PACKAGES / device.mk / equivalent), removing a service from the init/boot/service-registry set, dropping a kernel module / driver / config from the shipping configuration, un-bundling a prebuilt asset, deleting a submodule or its shipped output, removing a feature flag that gated a live capability, or any edit whose NET EFFECT is “an end-user capability that shipped before no longer ships.” Adding, replacing-with-operator-approved-equivalent, or fixing a capability is NOT a removal. When uncertain whether an edit constitutes a removal, treat it AS a removal and ask (per §11.4.6 no-guessing + §11.4.101 — removal of an existing user-facing capability is high-blast-radius and MUST be operator-confirmed, never autonomously decided). The tracked DROP path: ask → operator approves → mark the item Obsolete (→ Fixed.md) with Obsolete-Details reason feature-removed + an operator-approval citation (§11.4.90) → then remove; the removal never precedes the operator's yes.

Classification: universal (§11.4.17) — a platform-neutral discipline reusable by ANY project that ships a set of user-facing capabilities; the consuming project supplies its concrete capability-manifest paths per §11.4.35. Composes §11.4.66 / §11.4.101 / §11.4.90 / §11.4.112 / §11.4.6 / §11.4.40 / §11.4.42. Propagation gate CM-COVENANT-114-122-PROPAGATION (literal 11.4.122) + recommended gate

CM-NO-SILENT-COMPONENT-REMOVAL + paired §1.1 meta-test mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.122. Non-compliance is a release blocker. No escape hatch — no `--remove-without-asking`, `--silent-removal`, `--autonomous-removal-OK`, `--dedup-removal-exempt`, `--it-was-broken-anyway` flag.

§11.4.123 — Rock-solid-proof-or-deep-research mandate (User mandate, 2026-06-03)

Forensic anchor — verbatim user mandate (2026-06-03):

“Every single reported issue MUST BE fully and 100% validated with rock solid proofs! Nothing can be considered fixed or completed without hard evidence! No false results or bluff(s) of any kind is allowed! If we are not sure on how to achieve full testing, validation and verification of something we MUST ALWAYS perform deep web research for all possible data (articles, documentation, guides, and other resources) and opensourced codebases which we can use to solve our problems and perform testing with validation and verification which produces rock-solid evidence(s) and leaves no space for false results or any kind of bluff!”

Forensic case study (FACT). In the 1.1.8-dev remediation the validation method for two feature classes was, at first, genuinely unclear: relocating a `FLAG_SECURE` secure surface to a secondary display (pixel capture returns black) and asserting on-screen content in non-introspectable streaming-app UIs (blank accessibility hierarchy). Rather than declaring them “untestable” or accepting a metadata-only PASS, the cycle performed deep web research (`docs/research/testing_frameworks_20260603/`) that yielded the CV/OCR/liveness/sink-probe oracle stack (now §11.4.107 + §11.4.112 + §11.4.117) — making rock-solid evidence possible where it had appeared impossible. “Unclear how to validate” is a research trigger, NEVER a bluff licence.

Every single reported issue, every fix, and every claimed completion MUST be fully and 100% validated with rock-solid CAPTURED proof per §11.4.5 / §11.4.69 / §11.4.107 before it may be marked fixed / implemented / completed (§11.4.33 closure vocabulary). Nothing may be considered fixed or complete without hard

captured evidence — metadata-only / configuration-only / absence-of-error / grep-without-runtime PASS are all forbidden (§11.4 / §11.4.1); no false results, no bluff of any kind, at any layer.

The research-or-don't-bluff rule (the operative addition): when the agent is UNSURE how to fully test / validate / verify something — when no obvious evidence-producing method exists OR the candidate method would yield only metadata/config/absence-of-error evidence — it MUST ALWAYS first perform deep web research per §11.4.8 + §11.4.99 (official docs, articles, guides, vendor references, standards, issue trackers, reusable open-source codebases) to DISCOVER or BUILD a validation method that produces rock-solid evidence and leaves no space for a false result. Declaring something “untestable” / “not automatable” / accepting a metadata-only PASS WITHOUT first exhausting this deep-research path is itself a §11.4.123 violation — same severity class as a PASS-bluff. The research output (cited source URLs + the evidence-producing method, OR the literal “NO external solution found — original work” per §11.4.8) is the captured proof the path was exhausted. Only after that research genuinely fails may the item be classified `PENDING_FORENSICS`: / `operator-blocked` (§11.4.21) / `structurally-impossible` `won't-fix` (§11.4.112) — with the cited research as the evidence the classification is earned, never a convenience.

Classification: universal (§11.4.17) — a platform-neutral discipline reusable by ANY project; the consuming project supplies its concrete capture mechanisms + research corpora per §11.4.35. Composes §11.4.5 / §11.4.6 / §11.4.8 / §11.4.52 / §11.4.69 / §11.4.99 / §11.4.107 / §11.4.118 / §11.4.21 / §11.4.112. Propagation gate `CM-COVENANT-114-123-PROPAGATION` (literal `11.4.123`) + recommended gate `CM-ROCK-SOLID-PROOF-OR-RESEARCH` + paired §1.1 meta-test mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.123. Non-compliance is a release blocker. No escape hatch — no `--metadata-pass-suffices`, `--skip-proof`, `--untestable-without-research`, `--config-only-closure-OK`, `--bluff-when-unsure` flag.

§11.4.124 — Dead/unwired-code investigate-before-remove mandate (User mandate, 2026-06-04)

Forensic anchor — verbatim user mandate (2026-06-04):

“Before removing any seemingly-dead (zero-importer / unwired) codebase, we MUST investigate via git history where/how it was originally used and how it became dead. Removal is permitted ONLY when we have captured PROOF it is genuinely no longer needed — and that removal MUST be its own separate commit with a proper descriptive message. If there is no such proof, the code MUST be investigated for where/how it should be wired in properly, and any missing or unwired tests MUST be added. We MUST ALWAYS be extra careful with any codebase removal.”

“Zero importers / never called / unwired $\Rightarrow$ dead $\Rightarrow$ delete” is a GUESS (§11.4.6), never a finding — a “no references” result proves only *current* non-reference, not genuinely-unneeded. Before removing ANY seemingly-dead element (zero-importer / never-called / unwired function / method / type / file / module / package / asset / config / build target) the agent MUST FIRST investigate via git history (`git log --follow` , `git log -S / -G` pickaxe across all history, blame on the deleted call-site) and capture as FACT: (1) WHERE/HOW it was originally wired in, (2) WHEN/HOW it became dead — call-site deleted deliberately / by mistake (regression) / never-completed / refactored-unreachable, (3) whether “no references” is real OR a hidden reference the static tool cannot see (reflection / dynamic dispatch / build-tags / codegen / DI / plugin registry / FFI / config-driven wiring). The investigation output (cited commits + determination) is the captured evidence. **Removal is conditional:** permitted ONLY with captured PROOF the element is genuinely no longer needed; that removal MUST be its OWN SEPARATE COMMIT (independently reviewable + revertible, composes §11.4.84 quiescence + §11.4.92 multi-pass) with a descriptive message citing the git-history evidence — plus §11.4.122 operator-confirmation when the element is an end-user capability; the §11.4.90 tracked path marks it `Obsolete ( $\rightarrow$  Fixed.md)` . **No proof $\Rightarrow$ do NOT delete:** investigate WHERE/HOW to wire it in properly (restore a mistakenly-deleted call-site per §11.4.114; finish never-completed wiring) AND add any missing / unwired tests (§11.4.27 / §11.4.43 / §11.4.115 — the missing test is part of why it drifted into apparent-deadness). **Extra-caution default:** when uncertain whether removal-proof is sufficient, default to NOT removing (investigate + wire + test) per §11.4.6 + §11.4.101 + §11.4.122; “probably dead” is never sufficient — the bar is captured proof. Classification: universal (§11.4.17) — the consuming project supplies its static-analysis / importer-graph tooling + hidden-reference mechanisms per §11.4.35. Composes §11.4.6 / §11.4.8 / §11.4.84 / §11.4.90 / §11.4.92 /

§11.4.101 / §11.4.114 / §11.4.122 / §11.4.27 / §11.4.43 / §11.4.115. Propagation gate `CM-COVENANT-114-124-PROPAGATION` (literal `11.4.124`) + recommended gate `CM-DEAD-CODE-INVESTIGATE-BEFORE-REMOVE` (a net-deletion commit must be removal-only + cite the git-history investigation OR be part of a tracked Obsolete item) + paired §1.1 meta-test mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.124. Non-compliance is a release blocker. No escape hatch — no

`--zero-importers-means-dead`, `--delete-unwired-on-sight`, `--skip-git-history-investigation`, `--remove-without-proof`, `--bundle-removal-with-other-work` flag.

§11.4.125 — Code-review-agent gate before pre-build + main build (mandatory multi-layer review) (User mandate, 2026-06-04)

Forensic anchor — verbatim user mandate (2026-06-04):

“After all fixes/changes/implementations are done, BEFORE running pre-build tests and the main build, dispatch code-review agent(s) that analyze all work done + all existing data/facts + the existing codebase + current git history to determine quality, safety, and whether the fixes/changes will REALLY work; they MUST validate and verify that every test covering the fixes/changes genuinely validates the work with NO chance of false results or bluff of any kind. Any finding MUST be fixed, polished, improved, and covered with additional tests before the build proceeds. Multiple strong layers of checks.”

After all fixes / changes / implementations in a batch are done, and BEFORE running the pre-build test sweep AND the main (artifact) build (for ANY project), the agent MUST dispatch one or more dedicated code-review agent(s) (subagent-driven by default per §11.4.70/§11.4.20) performing a multi-layer review that: (1) analyzes ALL work done in the batch (every fix/change + its source diff + stated intent); (2) analyzes ALL existing data + facts (captured evidence per §11.4.5/§11.4.69/§11.4.107, tracker entries, prior findings, the §11.4.108 runtime-signature registry); (3) analyzes the existing codebase (blast radius per §11.4.92, cross-feature interaction, contract integrity of every dependency); (4) analyzes current git history (what each change touched, how it composes with concurrent/recent work,

whether it reproduces a known-broken pattern per §11.4.114/§11.4.124); (5) determines quality + safety + will-it-REALLY-work (robust + not error-prone — no solve-A-create-B; no host/data/security regression; genuinely delivers the end-user-visible behaviour per §11.4/§107); (6) validates + verifies the tests covering the work — every covering test genuinely exercises the work-under-test and catches its negation, with ZERO chance of a false result or bluff (a test that PASSES on broken-for-the-user work, a metadata-only/config-only/absence-of-error/grep-without-runtime assertion, or a gate whose paired §1.1 mutation does not make it FAIL is a finding). Any finding (defect / error-prone change / safety risk / will-not-really-work / bluff-or-false-result-capable test / missing-coverage gap) MUST be fixed, polished, improved, and covered with additional tests (four-layer per §11.4.4(b), TDD-RED-first per §11.4.43/§11.4.115) BEFORE the pre-build sweep + main build proceed; the review iterates (re-review after each remediation) until no blocking findings remain. The review is itself anti-bluff (its conclusions are captured evidence per §11.4.5/§11.4.69; a rubber-stamp review of a defective batch = PASS-bluff). It is one of MULTIPLE STRONG LAYERS — complementing, never replacing, the §1 pre-build sweep, §11.4.92 multi-pass (author-side self-review; §11.4.125 adds the structurally-separated reviewer seam per §11.4.70), §11.4.108 four-layer fix-verification, §11.4.110 build-readiness verdict, and the post-build / runtime-on-clean-target / user-visible layers. Composes §11.4 / §11.4.1 / §11.4.4 / §11.4.6 / §11.4.40 / §11.4.43 / §11.4.50 / §11.4.70 / §11.4.20 / §11.4.92 / §11.4.102 / §11.4.107 / §11.4.108 / §11.4.110. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-125-PROPAGATION` (literal `11.4.125`) + recommended gate `CM-CODE-REVIEW-GATE-BEFORE-BUILD` (build starts only with a fresh code-review-completed marker for the current batch, produced after the last fix + before the pre-build sweep + main build) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.125. Non-compliance is a release blocker. No escape hatch — no `--skip-code-review`, `--build-without-review`, `--no-review-gate`, `--review-optional`, `--trust-the-author` flag.

§11.4.126 — Default autonomous-loop working mode from first prompt (User mandate, 2026-06-04)

Forensic anchor — verbatim user mandate (2026-06-04):

“Make sure that you continue work in endless fully autonomous loop, do not stop until new fully validated and verified version (tag) is created and published (all submodules and main repo) or IN A CASE OF some other main stream work until it is fully completed with all side work streams and nothing else is left in our working queue! THIS MUST BE ALWAYS the default working mode without us asking you! We tend to achieve ABSOLUTE EFFICIENCY, with this and all other projects which will incorporate this MANDATORY RULE / CONSTRAINT!!! This way of (your) working will be ALWAYS applied / followed / executed / fully respected, as soon as we assign / send first request (prompt) in the session! This stops only if we explicitly say so or nothing is left to be done in current working scope (release that will come / upcoming version)!!! Any mimicking (imitation) of this behavior / rules / mandatory constraints, false results or any kind of bluff(s) is ABSOLUTELY FORBIDDEN!!!”

The endless fully-autonomous loop is the **DEFAULT working mode**, engaged automatically the moment the operator sends the FIRST request / prompt of a session — the operator MUST NOT have to ask for it, request it, restate it, or re-enable it per session. §11.4.87 framed the endless-loop covenant as an explicit-instruction opt-in (“continue in endless loop fully autonomously” or a semantically-equivalent phrasing); §11.4.126 is the **capstone** that promotes the same covenant to always-on: from the first prompt onward, every agent operates in the §11.4.87 loop discipline as the standing default, with §11.4.94 zero-idle, §11.4.97 maximum-idle-use, §11.4.101 autonomous-decision-over-blocking, and §11.4.103 continuous-parallel-stream all engaged by default — no per-session activation handshake. The continuation contract: the loop continues until ONE of two terminal conditions holds — (A) **Release scope** — a new, fully-validated-and-verified version (tag) is created AND published across all owned submodules AND the main repo to all configured remotes (per §2.1 multi-upstream push + §11.4.40 full-suite-retest-before-tag + §11.4.113 absolute-no-force-push merge-onto-latest-main); OR (B) **Non-release main-stream scope** — the main-stream goal is fully completed AND every side work stream is done AND the working queue holds nothing left for the current scope. Until (A) or (B) holds, the agent MUST keep working (claim the next priority item, dispatch the next parallel stream, progress every non-blocked item per §11.4.42 / §11.4.72 / §11.4.94 / §11.4.103). The loop STOPS ONLY on: (1) the operator explicitly saying so (STOP / pause / end); (2) nothing left to do in the

current working scope — the upcoming release / current main-stream goal — with the queue genuinely empty per the (A)/(B) terminal conditions; (3) a §12 host-session-safety demand (the loop yields to host safety unconditionally). Idle-while-blocked parks one work unit, it does not stop the loop — the agent keeps progressing every non-blocked item in parallel per §11.4.101 + §11.4.94 + §11.4.97. Goal — ABSOLUTE EFFICIENCY (no operator-side restart overhead, no idle gaps, no stop-and-wait round-trips); applies to this project AND every project that incorporates this Constitution. Anti-bluff: mimicking / imitating this loop behaviour, narrating continuation without performing it, fabricating progress, or emitting false / bluff results of ANY kind is ABSOLUTELY FORBIDDEN — this composes the entire §11.4 anti-bluff covenant family (§11.4 / §11.4.1 / §11.4.2 / §11.4.5 / §11.4.6 / §11.4.50 / §11.4.69 / §11.4.107); the agent MUST genuinely perform the continuous work and capture positive evidence for every closure, and a report claiming the loop ran while no real work / no captured evidence was produced is a §11.4 PASS-bluff at the operating-mode layer. Classification: universal (§11.4.17). Composes with §11.4.87 (the endless-loop covenant — §11.4.126 promotes it from opt-in to always-on default) / §11.4.94 / §11.4.97 / §11.4.101 / §11.4.103 / §11.4.66 / §11.4.6 / §11.4.40 / §11.4.42 / §11.4.72 / §11.4.113 / §2.1 / §12. Propagation gate CM-COVENANT-114-126-PROPAGATION (literal 11.4.126 across the consumer fleet) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md §11.4.126. Non-compliance is a release blocker. No escape hatch — no --ask-before-continuing , --single-turn-only , --not-default-loop , --mimic-OK flag.

§11.4.127 — Session-handoff resumption-prompt mandate (User mandate, 2026-06-06)

Forensic anchor — verbatim user mandate (2026-06-06): “make sure that in situations like this now when new session is needed you ALWAYS prepera such sentence - which will be valid for particular moment and the phase of the project and enough for work to continue.”

When the agent determines a fresh session is needed (context-window limits, performance degradation) OR the operator asks whether a new session is needed / requests a handoff, the agent MUST ALWAYS prepare + proactively provide a ready-to-paste **resumption prompt valid for that EXACT moment and project phase** — self-contained enough that pasting it into a fresh session resumes work

with ZERO loss. Two variants on demand: a SHORT first-sentence (“Read `<handoff docs>` , then continue `<terminal goal>` ...”) AND a FULL detailed block. The prompt MUST: (1) point to the live handoff doc(s) — `.remember/remember.md` if present + `docs/CONTINUATION.md` per §12.10 — read FIRST + `git fetch --all` ; (2) state current PHASE + immediate NEXT action + terminal goal; (3) embed exact live-state anchors (build IDs / artifact MD5, device/target serials, commit HEAD, in-flight PIDs + log paths, captured-evidence paths); (4) restate binding constraints (anti-bluff §11.4, no-force-push §11.4.113, exact version/naming, hardware/target gotchas); (5) be MOMENT-VALID, NEVER a generic template. Handoff doc(s) MUST be current BEFORE the prompt is given (§12.10). A missing / stale / generic prompt is a §11.4.127 violation. Composes §12.10 / §11.4.6 / §11.4.66 / §11.4.87 / §11.4.103 / §11.4.126. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-127-PROPAGATION` (literal `11.4.127`) + paired §1.1 meta-test mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.127. Non-compliance is a release blocker. No escape hatch — no `--skip-handoff-prompt` , `--generic-prompt-OK` , `--no-resumption-sentence` , `--handoff-without-state` flag.

§11.4.128 — Always-on device-recording mandate (User mandate, 2026-06-06)

Forensic anchor — direct user mandate (2026-06-06): we MUST ALWAYS live-record all available data from all devices we use for testing (or known to be under manual testing), EXTRA carefully so it never harms the device / its performance / causes side effects; raw recordings are NOT processed without need (token-conscious) and are ALWAYS git-ignored + code-intelligence-excluded; only curated evidence is committed, and only at release prep.

For EVERY test/debug device the project uses + every device under known manual testing, across EVERY reachable transport (USB / wireless ADB / SSH / serial / network introspection API), the project MUST ALWAYS live-record all analysable data: activities, all logs, performance metrics (CPU/memory/I/O/thermal/load), every sink-side report per §11.4.13, and any other live-changeable parameter. (1) **Extra-careful, side-effect-free** — non-invasive read-only probes only, bounded sampling, bounded write-volume, an observer-effect budget; a recorder that perturbs the device-under-test is a §11.4.128 violation, NOT evidence. (2) **Background + parallel + subagent-driven** per §11.4.103 + §11.4.70 — never blocks the main

stream. (3) **Token-conscious — record-now, analyse-later** — raw data NOT processed without need; the only standing analyse-trigger is release-tag prep (§11.4.40 / §11.4.42) OR explicit operator ask. (4) **Raw is git-ignored (with a §11.4.77 regen-mechanism declaration) AND code-intelligence-excluded (§11.4.78/§11.4.79)** — only CURATED evidence is committed, and only at release prep under `docs/qa/<run-id>/` (§11.4.83). (5) **Deterministic layout** `<recording-root>/YYYY-MM-DD/<combined main+submodules state hash>/<DEVICE>_<SERIAL>/recording_NNN/<files>` . (6) **Anti-bluff** — a recorder claimed running but with no growing corpus is a §11.4 bluff; every curated finding traces to a real raw-corpus path; recorder health is itself captured evidence per §11.4.5/§11.4.69.

Composes §11.4.2 / §11.4.5 / §11.4.13 / §11.4.69 / §11.4.40 / §11.4.42 / §11.4.70 / §11.4.77 / §11.4.78 / §11.4.79 / §11.4.83 / §11.4.103 / §11.4.119. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-128-PROPAGATION` (literal `11.4.128`) + recommended gate `CM-DEVICE-RECORDING-ALWAYS-ON` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.128. Non-compliance is a release blocker. No escape hatch — no `--skip-recording` , `--record-without-layout` , `--commit-raw-corpus` , `--index-raw-corpus` , `--analyse-corpus-always` , `--invasive-probe-OK` flag.

§11.4.129 — Huge-blocker release protocol (User mandate, 2026-06-06)

Forensic anchor — direct user mandate (2026-06-06): when a huge blocker is discovered during release validation we MUST stop all testing, fix ALL discovered issues, process all recorded data from the last session, land rock-solid fixes, author NEW validation+verification tests of ALL supported test types, rebuild, reflash, and RESTART the full validation+verification of every fix/change from the last release tag to now — on both devices in parallel, recorded, with real physical captured proofs and no bluff.

On discovery of a HUGE BLOCKER (release-blocking-severity defect: core user-facing capability broken, regression invalidating the in-flight cycle, or blast radius reaching the batch's other fixes) during release validation, execute in order with NO spot-check shortcut: (1) **STOP all testing** on every device (the §11.4.4 test-interrupt STOP at release granularity — continuing past a huge blocker is the §11.4 PASS-

bluff). (2) **Fix ALL discovered issues** — not just the blocker; root-cause each per §11.4.102 + isolate regressions against the last known-good tag per §11.4.114. (3) **Process all recorded data from the last session** — analyse the §11.4.128 raw-corpus slice (this IS the §11.4.128(3) release-prep analyse-trigger). (4) **Land rock-solid fixes** per §11.4.123 + §11.4.43/§11.4.115 + §11.4.9. (5) **Author NEW validation+verification tests of ALL supported test types** per §11.4.27 + §11.4.85, each anti-bluff + paired §1.1 mutation. (6) **Rebuild (full, not module-only) + reflash to a CLEAN target** per §11.4.108. (7) **RESTART the full validation+verification from the last release tag to now** per §11.4.40 — RESTART, never resume — on both/all owned devices IN PARALLEL per §11.4.103/§11.4.119, every run RECORDED per §11.4.128, real physical captured proofs per §11.4.5/§11.4.69/§11.4.107, no bluff. This anchor BINDS the existing release anchors for the huge-blocker case (adds STOP → fix-all → process-recordings → new-tests-all-types → rebuild → reflash → full-restart + the restart-not-resume rule), citing them rather than duplicating.

Composes §11.4.4 / §11.4.40 / §11.4.42 / §11.4.9 / §11.4.27 / §11.4.85 / §11.4.102 / §11.4.108 / §11.4.114 / §11.4.115 / §11.4.123 / §11.4.128 / §11.4.103 / §11.4.119.

Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-129-PROPAGATION` (literal `11.4.129`) + recommended gate `CM-HUGE-BLOCKER-FULL-RESTART` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.129. Non-compliance is a release blocker. No escape hatch — no `--resume-after-blocker`, `--spot-validate-after-fix`, `--skip-recording-analysis`, `--skip-new-tests`, `--module-only-after-blocker`, `--single-device-restart` flag.

§11.4.130 — Post-remediation validate-the-fix-FIRST-after-redeploy (User mandate, 2026-06-06)

Forensic anchor — direct user mandate (2026-06-06): when a blocker discovered during release validation is fixed and a new artifact (rebuild / new flashing image / redeploy) is produced + the target reflashed, we MUST first re-test the SPECIFIC last-failing features + validate the just-incorporated fixes BEFORE the broader / full validation.

When a blocker / critical failure found during release validation is FIXED and a new artifact is produced + the target reflashed / redistributed / updated, the agent MUST:

(1) **re-test the SPECIFIC last-failing features FIRST** (targeted guard tests for exactly the defects this fix addressed) BEFORE any broader / full-suite validation;

(2) **validate the just-incorporated fixes with real captured evidence** — the §11.4.115 RED test flips GREEN at `RED_MODE=0` on the new artifact AND the §11.4.108 runtime-signature verifies on the CLEAN target the redeploy produced (metadata-only / config-only / absence-of-error / grep-without-runtime PASS forbidden per §11.4 / §11.4.1; proof per §11.4.5/§11.4.69/§11.4.107/§11.4.123); (3) **only after the targeted fix is CONFIRMED working** proceed to the §11.4.40 full retest from the last tag to now. Rationale: a first fix attempt may not work / may be incomplete / may regress again under the new artifact — confirming the targeted fix FIRST catches a fix-did-not-take case immediately instead of hours later at the END of a full cycle (then restarting per §11.4.129); cheap-confirmation-first is §11.4.82 applied to the post-blocker reflash. This is the §11.4.46 recent-work-validation gate specialised for the post-blocker-reflash case + the targeted-confirmation phase that GATES §11.4.129's step-7 full-restart. Honest boundary (§11.4.6): “the fix probably took” ≠ “the fix took” — the RED → GREEN flip + runtime-signature on the new artifact is the proof; a still-FAILing targeted re-test re-enters the §11.4.114/§11.4.115 isolate → RED → fix loop, never proceeds to the full cycle on a still-broken fix. Composes §11.4.4 / §11.4.40 / §11.4.46 / §11.4.108 / §11.4.114 / §11.4.115 / §11.4.123 / §11.4.129 / §11.4.82. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-130-PROPAGATION` (literal `11.4.130`) + recommended gate `CM-FIX-FIRST-AFTER-REDEPLOY` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.130. Non-compliance is a release blocker. No escape hatch — no `--skip-targeted-retest`, `--full-cycle-first`, `--assume-fix-took`, `--validate-fix-at-end`, `--skip-red-green-flip-on-new-artifact` flag.

§11.4.131 — Standing session-resumption file mandate (User mandate, 2026-06-07)

Forensic anchor — verbatim user mandate (2026-06-07): “Make this markdown a standard file which will be written EVERY TIME when we need fresh session out of the box! It MUST BE always up to date and in sync so whenever new session is created all we have to do is just point to it!”

Every project MUST maintain a SINGLE canonical, always-current **session-resumption file** at a fixed, project-declared standard path (declared once per §11.4.35, never moved without a §11.4.66 operator decision). This file is the OUT-

OF-THE-BOX entry point for any fresh session: creating a new session requires ONLY pointing the new agent at this one file. §11.4.131 promotes §11.4.127 (PREPARE a resumption prompt on demand) into a STANDING, version-controlled ARTIFACT — ALWAYS present, ALWAYS in sync. (A) **Existence + fixed path** — exists at the declared path at all times, encoded as a literal path in the project-layer instantiation (§11.4.35), never silently moved. (B) **Always written + always synced** — (re)written whenever a fresh session is needed OR the live state materially changes (new HEAD, build/artifact id, phase, device/target state, in-flight job, blocking decision) — the §12.10 trigger set; a stale resumption file is a §11.4.131 violation of the same severity class as a §12.10 stale-CONTINUATION violation. (C) **Content (composes §11.4.127)** — both SHORT + FULL variants; points to `.remember/remember.md` + `docs/CONTINUATION.md` read FIRST + `git fetch`; embeds exact live-state anchors (HEAD, build/artifact ids + checksums, device serials, in-flight PIDs + log paths, captured-evidence paths); states PHASE + immediate NEXT + terminal goal; restates binding constraints (anti-bluff §11.4, no-force-push §11.4.113, exact version/naming, hardware gotchas); MOMENT-VALID, never a generic template (§11.4.6). (D) **Export + freshness** — §11.4.65 scope (synchronized `.html` / `.pdf` siblings) + §11.4.44 revision header. (E) **Out-of-the-box resumption** — a fresh session, given ONLY this file's path, fully resumes with zero additional context. Composes §12.10 / §11.4.127 / §11.4.65 / §11.4.44 / §11.4.6 / §11.4.66 / §11.4.126. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-131-PROPAGATION` (literal `11.4.131`) + recommended gate `CM-SESSION-RESUMPTION-FILE-PRESENT` + paired §1.1 meta-test mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.131. Non-compliance is a release blocker. No escape hatch — no `--skip-resumption-file`, `--ephemeral-prompt-only`, `--stale-resumption-OK`, `--generic-template-OK` flag.

§11.4.132 — Risk-ordered validation priority mandate (User mandate, 2026-06-07)

Forensic anchor — verbatim user mandate (2026-06-07): “We MUST ALWAYS first test and validate features, functionalities and fixes/changes that have been worked most recently, the ones which were most problematic, which have the most chance to crash or break again, the ones which have been re-opened the most times! Then, after we validate and verify all this with real (physical) proofs and hard

evidence, with no false results and bluffs of any kind, we continue with all other existing tests in the test suites! This IS MANDATORY.”

Tests / validations / verifications MUST run in **RISK-DESCENDING order** — the highest-risk set FIRST, and ONLY AFTER that set is fully GREEN with real (physical) captured evidence does the remainder of the suite run. Risk ranking is computed from a CLOSED set of factors, highest-risk first: (a) **most-recently-worked** features / fixes / changes; (b) **historically most-problematic** (longest defect history, most prior fixes/failures); (c) **highest crash/break/regress likelihood** (greatest blast radius / complexity / dependency surface); (d) **most-reopened** per §11.4.55 reopens-count (a high reopen count is the strongest empirical fragility signal). Each item in the highest-risk set MUST pass with real (physical) captured evidence per §11.4.5/§11.4.69/§11.4.107 — no metadata-only / config-only / absence-of-error / grep-without-runtime PASS (§11.4/§11.4.1), no false results, no bluff (§11.4.6). ONLY AFTER the entire highest-risk set is GREEN with captured proof does the rest of the suite run; running the suite in arbitrary order, or running lower-risk tests before the highest-risk set is GREEN, is a §11.4.132 violation. §11.4.132 REFINES/STRENGTHENS §11.4.130 (generalises “validate the just-fixed items first” to the full risk-ordered set) + §11.4.46 (adds explicit risk-ordering within the recent/high-risk set) + §11.4.42 (applies the implementation-layer priority discipline to VALIDATION ordering). Classification: universal (§11.4.17) — the consuming project supplies its recency / problematic-history / reopen-count sources (e.g. §11.4.93 workable-items DB `reopens_count` + `last_modified`) per §11.4.35. Composes §11.4.4/.5/.6/.7/.40/.42/.46/.50/.55/.69/.107/.130. Propagation gate `CM-COVENANT-114-132-PROPAGATION` (literal `11.4.132`) + recommended gate `CM-RISK-ORDERED-VALIDATION-PRIORITY` + paired §1.1 meta-test mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.132. Non-compliance is a release blocker. No escape hatch — no `--skip-risk-ordering`, `--any-order-OK`, `--suite-order-fixed` flag.

§11.4.133 — Target-System + hardware safety mandate (User mandate, 2026-06-08)

Forensic anchor — verbatim user mandate (2026-06-08): “Make sure that all changes we do to the System are ALWAYS safe for the System itself and for the hardware the system runs on! This is MANDATORY.”

Every change to the TARGET system (firmware, kernel, init/boot scripts, drivers, sysfs/devfreq/voltage/clock/thermal/regulator register writes, partition/bootloader/U-Boot, HAL, framework, prebuilts, device config) MUST ALWAYS be safe for BOTH (a) the target System itself — MUST NOT brick, boot-loop, corrupt data, or render the device unrecoverable — AND (b) the hardware it runs on — MUST NOT exceed safe electrical/thermal/voltage/clock limits or damage panels/storage/radios/regulators. Concrete obligations: (1) reversible-first — verify irreversible high-blast-radius changes (bootloader/U-Boot MD5, partition layout) against known-good values + capture a pre-op backup (§9.2) BEFORE applying; (2) NO unverified hardware-control writes — never write an unverified value to a voltage/clock/regulator/thermal-throttle/current-limit sysfs node or register that could exceed datasheet limits, the safe range established as FACT (§11.4.6), never guessed; (3) thermal/perf changes (forcing a performance governor, pinning the top OPP, disabling thermal management) MUST respect the device's cooling design, validated by captured thermal evidence; (4) flashing MUST use the sanctioned tool + a freshly-built integrity-verified image — never an ad-hoc partition write or stale/unverified artifact; (5) unprovable-safety ⇒ blocked — a change whose target/hardware safety cannot be established from captured evidence is treated as UNSAFE and blocked (§11.4.6 + §11.4.101 reversible-first + §11.4.123 rock-solid-proof). DISTINCT from §12 host-session safety: §12 protects the DEVELOPER's HOST + session; §11.4.133 protects the TARGET device + its hardware — both apply, neither weakens the other. Classification: universal (§11.4.17) — the consuming project supplies its concrete hardware-control surfaces, datasheet-safe ranges, known-good bootloader/image hashes, and sanctioned flashing tool per §11.4.35. Composes §12 / §11.4.6 / §11.4.101 / §11.4.108 / §11.4.123. Propagation gate `CM-COVENANT-114-133-PROPAGATION` (literal `11.4.133`) + recommended gate `CM-TARGET-HARDWARE-SAFETY` + paired §1.1 meta-test mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.133. Non-compliance is a release blocker. No escape hatch — no `--unsafe-hardware-write`, `--skip-system-safety`, `--brick-risk-accepted` flag.

§11.4.134 — Code-review iterate-until-GO + rock-solid-evidence mandate (User mandate, 2026-06-08)

Forensic anchor — verbatim user mandate (2026-06-08): “For any fixes/changes given back to us for re-work by the code-review process, once we fix/improve everything per the code-review's requests, we MUST RE-RUN code-review AGAIN

until we get a GO from it with NO new issues reported or warnings of any kind! All results produced by this whole process MUST ALWAYS give us rock-solid PHYSICAL evidence that the fixed/improved codebase really works now as expected, with no false results and no bluff(s) of any kind.”

When the §11.4.125 code-review returns ANY finding — BLOCKING, nit, or warning — and the author fixes/improves the batch per that review, the code review MUST BE RE-RUN, and MUST KEEP being re-run after each remediation round, until it returns a clean GO with ZERO new issues AND ZERO warnings of any kind. A single pass that “addressed the findings” is NOT sufficient: the corrected batch MUST pass a FRESH adversarial review (a re-review can surface NEW findings introduced by the very fixes that closed the prior ones — the §11.4.1 fix-A-creates-B failure mode). The loop terminates ONLY on a clean GO (no new findings, no warnings); a residual warning is itself a finding that re-arms the loop. Every round’s verdict AND every fix’s validation MUST carry rock-solid PHYSICAL captured evidence per §11.4.5 / §11.4.69 / §11.4.107 (captured audio / video / sysfs / dumphsys / sink-side / runtime-signature) proving the fixed/improved codebase REALLY works as expected — never metadata-only / configuration-only / absence-of-error / grep-without-runtime; no false results, no bluff at any round; a reported GO unbacked by captured physical evidence is itself a §11.4 PASS-bluff at the review-loop layer. §11.4.134 REFINES / STRENGTHENS §11.4.125 (iterate “until no blocking findings remain”): it makes the loop EXPLICIT (re-run after every remediation round, not once), raises termination to ZERO findings AND ZERO warnings (not merely zero-blocking), and BINDS rock-solid physical evidence to every round. Classification: universal (§11.4.17). Composes §11.4.125 / §11.4.1 / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.69 / §11.4.107 / §11.4.50 / §11.4.108 / §11.4.123. Propagation gate `CM-COVENANT-114-134-PROPAGATION` (literal `11.4.134`) + recommended gate `CM-CODE-REVIEW-ITERATE-UNTIL-GO` + paired §1.1 meta-test mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.134. Non-compliance is a release blocker. No escape hatch — no `--skip-rereview`, `--single-review-pass`, `--warnings-ok`, `--evidence-optional` flag.

§11.4.135 — Standing regression-guard suite + every-fixed-defect-gets-a-permanent-regression-test (User mandate, 2026-06-08). Every project MUST maintain a STANDING regression-guard suite that runs on EVERY build+deploy and BLOCKS the release tag on any failure. Every closed defect (stable ticket id, e.g. ATM-NNN) MUST, in the SAME commit as its fix (extending the §11.4.43

DOCUMENT step), register a permanent §11.4.115 RED-on-broken-artifact regression test into the suite — `RED_MODE=1` capturing the historical defect on a pre-fix artifact (the proof the guard is real), `RED_MODE=0` the standing GREEN guard asserting the defect is ABSENT. A closure without a registered guard is a §11.4.123 violation. The suite runs FIRST in the post-deploy cycle (highest-risk set per §11.4.132) and is a §11.4.40 release-gate blocker. Forensic anchor (FACT): the wrong-subtitle-on-2nd-display defect was “fixed” via a source-side

`CONTROL_MENU_LABEL_DENYLIST` that NO test mirrored or re-ran, so the NEXT chrome class recurred silently while the GREEN suite passed. Industry-standard bug-driven testing (Google content-driven testing; AOSP CTS/Tradefed) made mechanical + enforced. Composes §11.4.4 / §11.4.40 / §11.4.43 / §11.4.46 / §11.4.50 / §11.4.107 / §11.4.108 / §11.4.115 / §11.4.118 / §11.4.123 / §11.4.124 / §11.4.130 / §11.4.132. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-135-PROPAGATION` (literal `11.4.135`) + recommended gates `CM-REGRESSION-GUARD-REGISTERED` / `CM-REGRESSION-GUARD-SUITE-WIRED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.135. Non-compliance is a release blocker. No escape hatch — no `--skip-regression-guard`, `--no-guard-on-close`, `--guard-optional` flag.

§11.4.136 — Real-content end-to-end playback-test mandate (User mandate, 2026-06-08). Refines/strengthens §11.4.107. Any test asserting media playback works MUST drive REAL content (catalog stream or offline reference clip) through the user’s path (§11.4.48 UI-driven → §11.4.117 CV/OCR fallback) and assert it genuinely PLAYS via the §11.4.107 liveness battery PLUS a decoder-health census — a numeric drop-buffer budget, no buffer-timestamp re-order/discard, no codec-reject (cite Android/Media3 ExoPlayer OEM pre-OTA playback-test mandate: “too many dropped buffers” >25, “unexpected presentation timestamp”, “test timed out”). Metadata-only / launch-only / registration-only / single-frame / config-only PASS is forbidden (§11.4 / §11.4.1). A golden/reference clip corpus (BBC ExoPlayer testing samples) is the offline ground-truth. Composes §11.4.5 / §11.4.48 / §11.4.50 / §11.4.107 / §11.4.117 / §11.4.123 / §11.4.13 / §11.4.69. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-136-PROPAGATION` (literal `11.4.136`) + recommended gate `CM-REAL-CONTENT-PLAYBACK-TEST` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.136. Non-compliance is a release blocker. No escape hatch — no `--launch-proves-playback`, `--skip-decoder-health`, `--metadata-playback-pass-suffices` flag.

§11.4.137 — Subtitle/caption content-correctness oracle + secure-display-proxy-honesty mandate (User mandate, 2026-06-08). Refines §11.4.117 + §11.4.107 + §11.4.112. Forensic anchor (FACT): tests tasked to “physically verify the 2nd-display subtitle” PASSEd GREEN while subtitles did NOT show / showed WRONG — the streaming player surface is FLAG_SECURE so `screencap -d <secondary>` returns BLACK (autonomous PIXEL verification structurally impossible per §11.4.112), so the test fell back to the accessibility-scraped/`persist.atmosphere.subdebug` proxy, and the proxy accepted a chrome/menu LABEL (Аудио и субтитры) as a valid subtitle because the prose floor accepted any multibyte prose and NO menu-label denylist + NO position/cadence check existed. The mandate: a subtitle-correctness test MUST classify the cue’s *content class* — a present cue is NOT a correct cue. CHROME (FAIL) if a known control/menu label (closed multilingual deny-list MIRRORED from source, case-folded incl. non-ASCII), time/numeric chrome, not prose, OUTSIDE the lower safe-title band (CEA-708 9-anchor grid), OR STATIC across the window (real subtitle changes → ≥2 distinct prose cues, a metamorphic relation). DIALOGUE (PASS) only when prose + not-denied + not-chrome + position-ok + cadence ≥2 OR fuzzy-matches the SOURCE-extracted cue via normalized edit distance (§11.4.123 host ground truth). The oracle MUST be self-validated golden-good/golden-bad (§11.4.107(10)) and the deny-list MUST be verified present in the SHIPPED artifact (§11.4.108) — a source-green denylist with no test mirror + no artifact check is the exact recurrence pattern forbidden here. Secure-display honesty (§11.4.112): where FLAG_SECURE makes pixel verification impossible, the rock-solid autonomous proof is the player’s caption telemetry + source-track presence + content-class oracle — NEVER a faked pixel “physical” pass; human-eye pixel confirmation is `operator_attended` (§11.4.52) with a tracked migration item. App-agnostic (keys off content class). Composes §11.4.3 / §11.4.5 / §11.4.6 / §11.4.107 / §11.4.108 / §11.4.112 / §11.4.115 / §11.4.117 / §11.4.123 / §11.4.13 / §11.4.69. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-137-PROPAGATION` (literal 11.4.137) + recommended gate `CM-SUBTITLE-CONTENT-CORRECTNESS-ORACLE` + paired §1.1 mutation (strip the denylist/position/cadence check → golden-bad Аудио и субтитры PASSES → gate FAILs). **Canonical authority:** constitution submodule `Constitution.md` §11.4.137. Non-compliance is a release blocker. No escape hatch — no `--present-cue-is-correct` , `--skip-chrome-oracle` , `--length-heuristic-suffices` , `--pixel-pass-on-secure-display` , `--skip-position-check` , `--skip-cadence-check` flag.

§11.4.138 — Operator-escape => mandatory bluff-audit + permanent guard

(User mandate, 2026-06-08). When the operator (or any out-of-band channel) finds a defect that the GREEN test suite passed, this is by definition a §11.4 PASS-bluff — it MUST trigger, before the fix is closed: (1) a §11.4.102 systematic-debugging pass to FACT-root-cause; (2) a bluff-audit identifying the EXACT assertion that should have caught it but didn't, cited to `file:line` (canonical example: `lib/subtitle_content_validation.sh:sub_is_prose()` returning TRUE for `Аудио и субтитры`); (3) a permanent §11.4.135 regression guard registered in the SAME commit as the fix, with its §11.4.115 RED capturing the operator-found defect; (4) the bluff-audit committed under `docs/research/<scope>/<defect>_bluff_audit/`. Closing an operator-found defect WITHOUT the bluff-audit + permanent guard is itself a §11.4 violation (the bluff that let it through is still live and the defect will recur). Composes §11.4 / §11.4.1 / §11.4.102 / §11.4.108 / §11.4.115 / §11.4.118 / §11.4.123 / §11.4.135. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-138-PROPAGATION` (literal `11.4.138`) + recommended gate `CM-OPERATOR-ESCAPE-BLUFF-AUDIT` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.138. Non-compliance is a release blocker. No escape hatch — no `--close-without-bluff-audit`, `--operator-find-is-just-a-bug`, `--skip-permanent-guard` flag.

§11.4.139 — Fresh-process clean-artifact runtime-signature mandate (User mandate, 2026-06-08). Refines §11.4.108. Before any post-deploy validation — ESPECIALLY a non-pixel proxy verification (the subdebug/accessibility-cue channel used for FLAG_SECURE displays) — the harness MUST assert running-artifact == built-artifact: the deploy yielded a CLEAN target (mutable-overlay/userdata wiped) OR a pre-validation check proves no stale overlay shadows the deployed code (e.g. every guarded package — incl. the Presenter that emits the subtitle cue — resolves to the system partition, no per-user override). A stale shadow of the cue-emitting component (e.g. a Presenter APK predating the denylist) makes the proxy report on code that was never deployed — any PASS is a §11.4 PASS-bluff. Each fix declares ONE machine-checkable runtime signature verified on the clean target (the §11.4.108 registry IS the definition of done); for the subtitle class the signature is “the shipped Presenter APK contains the denylist literal (case-insensitive) AND the subdebug channel emits `candidate REJECTED reason=chrome-label` for a menu label.” Composes §11.4.46 / §11.4.108 / §11.4.130 / §11.4.135 / §11.4.137. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-139-PROPAGATION` (literal `11.4.139`) + recommended gate `CM-CLEAN-ARTIFACT-RUNTIME-SIGNATURE` + paired §1.1 mutation. **Canonical authority:** constitution

submodule Constitution.md §11.4.139. Non-compliance is a release blocker. No escape hatch — no `--validate-against-running-state`, `--skip-clean-precondition`, `--shadow-OK` flag.

§11.4.140 — Universal action-prefix system (`ACTION_NAME ::`) (User mandate, 2026-06-09; GRAMMAR_ADDENDUM 2026-06-09). When a user prompt's FIRST non-blank line starts with a recognised action prefix, you MUST: (1) look the action token up in the action registry `constitution/actions/registry.yaml` (or `$HELIX_ACTION_REGISTRY`); (2) if it is a registered action, REPLACE the prefix with that action's `expansion` text and apply its `rules`; (3) execute the remainder of the prompt under the expanded instruction. **Four EQUIVALENT forms** — same action, same expansion, same execution: (1) `ACTION_NAME :: <rest> (bare :: )`, (2) `PREFIX::ACTION_NAME :: <rest> (namespaced :: )`, (3) `/ACTION_NAME <rest> (bare slash)`, (4) `/PREFIX::ACTION_NAME <rest> (namespaced slash)`. Thus `BACKGROUND :: x ≡ DEFAULT::BACKGROUND :: x ≡ /BACKGROUND x ≡ /DEFAULT::BACKGROUND x`. `PREFIX` is an action NAMESPACE; the reserved default namespace is **DEFAULT**, and an action runs WITH or WITHOUT the prefix. Grammar (all hold): anchored at the FIRST non-blank line only (mid-prose tokens never match); the action token AND the namespace are UPPERCASE-only `[A-Z][A-Z0-9_]*` (lowercase never matches); the namespace separator `::` inside the token carries NO surrounding spaces (`PREFIX::ACTION_NAME`), DISTINCT from the action-body separator `" :: "` (one ASCII space on each side of `::` — avoids C++ `Foo::Bar`, YAML `key: value`, URLs) in forms 1/2 and the slash-body separator (one space) in forms 3/4; stacked prefixes (`A :: B :: rest`) apply outer-to-inner, left-to-right (expand `A`, re-scan, expand `B`, then the residual is the task); a leading `\` escapes the prefix for BOTH the `::` and the slash form (`\BACKGROUND :: x`, `\ /BACKGROUND x` — treat literally, strip the backslash, NO expansion) so action names can be discussed. **Conflict rule (slash form):** `/ACTION_NAME` (form 3) is honored as the action ONLY when `ACTION_NAME` (case-folded) does not collide with a built-in/host slash command (registry `slash_bare: auto + slash_conflicts: [...]`); form 4 (`/PREFIX::ACTION_NAME`) is ALWAYS unambiguous and always honored. An unknown token that matches the grammar shape (any of the 4 forms) but is NOT registered is NEVER silently expanded or silently dropped — ask which registered action was meant (§11.4.66 / §11.4.105) or treat it as a literal

prompt, NEVER invent an expansion (§11.4.6); any prompt not satisfying the grammar is an ordinary prompt and the system is a no-op. The registered action **BACKGROUND** expands to: *“The following prompt that we will provide MUST BE executed in background in parallel with all main work streams using the subagents-driven development approach! All work done MUST PRODUCE rock solid evidence covered with hard physical proof(s) that all done is working as expected and as specified without any false results and without any bluff!”* (composes §11.4.20 / §11.4.70 subagent-driven, §11.4.58 / §11.4.103 parallel streams, §11.4.89 background execution, §11.4.5 / §11.4.69 / §11.4.107 captured physical evidence, §11.4 anti-bluff). The system is UNIVERSAL (every CLI agent reads this block via its context carrier per §11.4.35), extensible (new action = new registry row), decoupled + reusable (§11.4.28), and loads out-of-the-box. Classification: universal (§11.4.17). **Canonical authority:** constitution submodule Constitution.md §11.4.140. Non-compliance is a release blocker. No escape hatch — no `--skip-action-prefix`, `--ignore-prefix`, `--no-registry`, `--invent-expansion-OK`, `--single-layer-only` flag.

§11.4.141 — Token-efficiency mandate (research-derived + operator mandate, 2026-06-09). Every project worked on by AI coding agents MUST cut token spend (input AND output) toward **30–40% of current (a 60–70% reduction)** WITHOUT degrading quality/performance/safety or breaking any existing mechanism, via a composable, safety-ranked measure set: (1) **prompt-cache the static governance prefix** — the always-loaded governance forms a byte-stable cache-breakpointed prefix with no volatile bytes ahead of it; cache reads cost $\sim 0.1\times$ base input (the dominant cost driver — measured $\sim 170K$ tokens of governance re-sent every turn, externally corroborated by Claude Code issue #24147); caching is transparent so it removes no rule, weakens no gate, changes no verdict — only billing (PRIMARY, biggest + safest lever); (2) **subagent model-tiering + output-to-file** — mechanical non-judgment work (search/grep/status/doc-export/read-only probes) to a Haiku-class model, the strong model RESERVED for all reasoning/verdicts/fix-design (§11.4.102)/code-review (§11.4.125)/demotion (§11.4.7), large output persisted to a file not an inline 350–520K-token transcript; the cheap model never emits a PASS so §11.4.50 + anti-bluff are untouched; (3) **thin always-loaded INDEX + on-demand detail** — concise index (one line per fix/anchor, EACH carrying the literal 11.4.N token so propagation gates pass) with the canonical full text kept gate-scanned in `constitution/Constitution.md` and reachable in one hop — a de-

duplication realising §11.4.35, never a deletion; (4) **CodeGraph/retrieval-first over full-file loading** (§11.4.78/§11.4.79); (5) **output-token reduction** — terse status + `effort:"low"` on the mechanical allowlist only; (6) **tool-call batching + no re-reads**; (7) **compaction/context-editing for long sessions. Mandatory measured proof:** a token-accounting harness measures tokens-per-development-cycle BEFORE vs AFTER on a frozen deterministic workload from the authoritative `usage` object (input/cache_read/cache_creation/output split; NEVER `tiktoken`, NEVER the client-side cost estimate), reproduced N times (§11.4.50), $\text{pass} = \text{AFTER} \leq 40\% \text{ of BEFORE}$ OR the measured best-safe reduction with a cited cold-cache reason; the AFTER run MUST show ZERO regression on the pre-build sweep + meta-test mutation sweep + propagation gates + a strong-model reasoning probe + a cache-warm proof (`cache_read_input_tokens > 0`) — cost reduction with quality regression is a §11.4 FAIL. The headline number is the *measured* reduction, never the design estimate (§11.4.6/§11.4.123). No measure may break/degrade any existing mechanism, and the rule is structured so none can. Composes §11.4.5/.6/.20/.40/.50/.58/.69/.70/.78/.79/.80/.103/.106/.123/.125/.128/§12.6/§1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-141-PROPAGATION` (literal `11.4.141`) + recommended gate `CM-TOKEN-EFFICIENCY` + paired §1.1 mutation (inject a pre-breakpoint volatile token → cache collapses → measured reduction falls below bar → gate FAILs). **Canonical authority:** constitution submodule `Constitution.md` §11.4.141. **Project instantiation (§11.4.35):** [the consuming project fills in: confirm the Claude Code governance-prefix cache is warm + free of pre-breakpoint volatiles; tier the §11.4.96-SAFE mechanical subagents (search/grep/status/doc-export) to Haiku with output-to-file under `qa-results/`; replace the consumer `CLAUDE.md` 112-row Applied-Fixes inline table + verbatim anchor restatements with a literal-anchor index pointing at `constitution/Constitution.md`; keep CodeGraph/lumen-first navigation; run the harness BEFORE/AFTER to prove the measured reduction.] Non-compliance is a release blocker. No escape hatch — no `--skip-token-efficiency`, `--no-cache-governance`, `--assert-reduction-without-measuring`, `--tier-down-reasoning`, `--inline-all-governance`, `--tiktoken-estimate-OK` flag.

§11.4.142 — Universal code-review mandate — every change reviewed, always, no exception (User mandate, 2026-06-09). Verbatim operator mandate: “ALL changes we do MUST pass through the code review step!!! ALWAYS!!!” EVERY change made to ANY repository this Constitution governs — without exception — MUST pass through an INDEPENDENT code-review step BEFORE it

is accepted, committed, or built. NO change class is exempt: source code, fixes, tests, gates, meta-test mutations, documentation, doc-tooling, build/CI scripts, configuration, governance files (Constitution / CLAUDE / AGENTS / QWEN), conductor main-stream edits, sub-agent output, refactors, single-line edits — if a diff exists, it gets an independent review. This is the ABSOLUTE form of §11.4.125 (code-review agent gate after a batch, before pre-build sweep + main build): §11.4.142 strips every implicit scoping §11.4.125's "after all fixes/changes/ implementations are done" phrasing could leave open — no "just a doc edit", no "just a one-liner", no "the author already self-reviewed (§11.4.92)", no "trivial change" carve-out. **Independence is load-bearing** — the reviewer MUST be structurally separated from the author (a dedicated code-review agent, subagent-driven by default per §11.4.70 / §11.4.20, or a distinct human), NEVER the author re-reading their own work; §11.4.92's multi-pass self-evaluation is the AUTHOR-side discipline and PRECEDES (never satisfies) §11.4.142. **The review is itself anti-bluff** (§11.4 / §11.4.1) — a rubber-stamp "looks good" is not a review; it MUST genuinely analyse correctness, safety (no host §12 / data §9 / target-hardware §11.4.133 regression), will-it-really-work (no solve-A-create-B), end-user behaviour (§11.4 / §107), and test-genuineness (§1.1), its conclusions captured evidence per §11.4.5 / §11.4.69, and **it iterates to a clean GO per §11.4.134** — any finding (BLOCKING / nit / warning) re-arms the loop, acceptance only on ZERO new findings + ZERO warnings with rock-solid physical evidence. Honest boundary (§11.4.6): a passing review does NOT replace §11.4.108 four-layer runtime-signature verification nor the §11.4.40 full-suite retest — it is one of MULTIPLE STRONG LAYERS, and the FIRST one every change crosses. Composes §11.4.1 / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.20 / §11.4.40 / §11.4.69 / §11.4.70 / §11.4.92 / §11.4.108 / §11.4.110 / §11.4.125 / §11.4.134 / §107 / §1.1. Classification: universal (§11.4.17) — the consuming project supplies its reviewer-dispatch mechanism + change-acceptance seam (commit wrapper / merge queue / PR gate) per §11.4.35. Propagation gate `CM-COVENANT-114-142-PROPAGATION` (literal `11.4.142`) + recommended gate `CM-EVERY-CHANGE-REVIEWED` (every accepted change carries a fresh independent-review-completed marker for its diff, produced by a reviewer distinct from the author, before acceptance/commit/build) + paired §1.1 mutation (strip the literal → propagation gate FAILs; accept a diff with no independent-review marker → `CM-EVERY-CHANGE-REVIEWED` FAILs). **Canonical authority:** constitution submodule `Constitution.md` §11.4.142. Non-compliance is a release blocker. No escape hatch — no `--skip-review`, `--no-review`, `--trivial-change-exempt`, `--doc-edit-exempt`, `--self-review-suffices`, `--review-after-commit` flag.

§11.4.143 — Real-user-journey mandate for video-streaming-app full-automation tests (User mandate, 2026-06-10). Verbatim operator mandate: “All video streaming apps ... require to choose some title and to press proper UI button to start or resume playing! Proper UI interaction to play exact show with proper content and subtitles is MANDATORY! Without it we just eventually play on 2nd display sample, and that’s it mostly! THIS MUST BE ADDED as MANDATORY RULE regarding testing of any video streaming app in general with full automation tests! Universal in root constitution + a consuming project’s extensions.” Any full-automation test that asserts a video player / streaming application plays content MUST drive the REAL end-user journey through the app’s OWN UI — launch → BROWSE the actual catalog → choose a SPECIFIC title → press the real Play/Resume button → confirm THAT chosen content is genuinely playing, with its correct subtitles, on the intended routing target. A test that bypasses the journey with a sample / demo / built-in-loop clip, a deep-link / `am start -a VIEW` / intent shortcut, a synthetic or pre-staged stream, or any path that does NOT exercise the app’s own browse-select-play UI is a §11.4 PASS-bluff at the user-journey layer: it validates ROUTING (that *something* reaches the display) while leaving the user-visible behaviour the operator cares about — “the show I picked actually plays” — unproven (the operator’s “we just eventually play on 2nd display sample, and that’s it mostly” gap). The mandate (ALL hold): (1) **Real journey, not a shortcut** — launch → catalog browse → specific-title selection → real Play/Resume press through the app’s own UI (§11.4.48 UI-driven), NEVER a deep-link / intent / sample / loop-clip shortcut; (2) **Chosen-content confirmation** — the PASS proves the SPECIFIC selected title is playing (not merely that pixels move on the target) via the §11.4.107 liveness battery on the §11.4.136 real-content path + the §11.4.137 subtitle content-correctness oracle for the chosen title’s captions; (3) **Non-introspectable UIs use the pixel oracle** — when the app’s accessibility hierarchy is blank/unreliable (TV-Compose / leanback / canvas / GL), DRIVE input + ASSERT content via the §11.4.117 CV/OCR pixel oracle, never a hierarchy-only tool; (4) **Login via the credential single-source** (§11.4.10), never hardcoded, never logged; (5) **Honest SKIP, never a faked PASS** — where the autonomous journey is genuinely infeasible (hard human-only login / CAPTCHA, geo-block per §11.4.3, secure-surface pixel-blanking per §11.4.112) the test is `operator_attended SKIP`-with-reason per §11.4.52 + §11.4.3 with a tracked migration item — NEVER a metadata-only / sample-played / routing-only PASS. Honest boundary (§11.4.6): “the routing fired so the title is playing” is a guess — only the chosen-content liveness + subtitle oracle on the real journey proves it. Composes §11.4.48 /

§11.4.107 / §11.4.117 / §11.4.136 / §11.4.137 / §11.4.52 / §11.4.3 / §107 / §1.1.

Classification: universal (§11.4.17) — the consuming project supplies its concrete app roster, login-credential source, routing target, and UI-driving / pixel-oracle harness per §11.4.35. Propagation gate `CM-COVENANT-114-143-PROPAGATION` (literal `11.4.143`) + recommended gate `CM-VIDEO-REAL-JOURNEY-TEST` (every video-streaming-app playback test drives the real browse-select-play journey + confirms the chosen content + subtitles, or SKIPS-with-reason) + paired §1.1 mutation (replace a real-journey test's browse-select-play path with a deep-link / sample shortcut → gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule

`Constitution.md` §11.4.143. Non-compliance is a release blocker. No escape hatch — no `--sample-playback-ok`, `--skip-real-journey`, `--deep-link-suffices`, `--routing-only-pass`, `--no-title-selection` flag.

§11.4.144 — Tracked/recorded-device availability-following mandate (User mandate, 2026-06-10). Direct operator mandate: every device the project is tracking / following / recording (test / debug / manual-testing device, across every reachable transport — USB / wireless ADB / SSH / serial / network introspection API) MUST be availability-FOLLOWED — its connection state continuously monitored, any drop handled, never silently abandoned and never presented as a continuous recording. The §11.4.128 always-on recorder KNOWS a tracked device is absent (its per-device loop guards on the reachable state) but, lacking a following discipline, merely spins idle — no data captured, no offline event logged, no resume, no escalation — so the recording corpus presents a continuous timeline with a silent hole, a §11.4 PASS-bluff at the recording-integrity layer (it claims continuous capture while no data exists for the gap). On a tracked device leaving its reachable state the system MUST, automatically: (a) **DETECT** the drop and **log an honest offline event** into the recording corpus (§11.4.6 — a silent gap presented as continuous capture is a fabricated-continuity bluff; §11.4.128 — a silent recording gap defeats always-on recording); (b) **WAIT** for the device to return using the project's ALREADY-DEFINED reconnection timings — never invented numbers (§11.4.6), the SAME grace / reconnect / poll budgets the project's recovery path already uses; (c) **RE-ATTACH** — resume recording / tracking the moment the device returns and log an honest online / resume event; (d) **ESCALATE** to the project's sanctioned device-recovery path (per §11.4.69 feature class `device_recovery`) if the device does not return within the defined timeout — through the sanctioned, authorization-gated recovery entry point ONLY, never bypassing its gate and never performing a destructive recovery (e.g. a power-cycle)

autonomously without that authorization (§11.4.21 / §11.4.101 — high-blast-radius recovery is gated; while blocked the system keeps following the device, logging the blocked-escalation honestly). A tracked-device drop that produces a silent corpus hole, a never-resumed recording, or a never-escalated permanent absence is the bluff this anchor forbids. Composes §11.4.128 (always-on recording — §11.4.144 closes its drop-handling gap) / §11.4.69 (`device_recovery` sink-side positive evidence) / §11.4.6 (honest offline / online events, reused-not-invented timings) / §11.4.14 (watchdog children reaped on stop) / §11.4.21 + §11.4.101 (gated, non-autonomous escalation). Classification: universal (§11.4.17) — the consuming project supplies its concrete tracking transport, device set, already-defined reconnection timings, and sanctioned recovery entry point per §11.4.35.

Propagation gate `CM-COVENANT-114-144-PROPAGATION` (literal `11.4.144`) + recommended gate `CM-DEVICE-AVAILABILITY-FOLLOWED` + paired §1.1 meta-test mutation (strip the watchdog wiring / let an absence go unlogged → gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.144. Non-compliance is a release blocker. No escape hatch — no `--skip-availability-following` , `--silent-recording-gap-OK` , `--no-reconnect-wait` , `--invent-reconnect-timing` , `--skip-recovery-escalation` , `--autonomous-power-cycle-OK` flag.

§11.4.145 — Independent multi-angle impact-research per change (User mandate, 2026-06-10). For EVERY fix / change / new feature, INDEPENDENT impact-research agents (subagent-driven §11.4.70/§11.4.20, structurally separate from the author — §11.4.92 self-eval PRECEDES, never satisfies — adversarial “refute-the-change” stance to defeat the documented LLM confirmation-bias failure mode) MUST research the change AND its connected/dependent features across a CLOSED SET OF EIGHT ANGLES — (1) correctness/logic, (2) regression (what existing working feature could break — via call-graph impact enumerating every direct + transitive caller and contract-dependent feature, each mapped to its test), (3) latent/dangerous-code — the recents class (runtime-only failure, interface/ABI/contract mismatch, concurrency/data-race, resource lifecycle), (4) security (taint/injection/secret/unsafe-API), (5) performance (latency/memory/thermal under load vs baseline), (6) host/data/target-hardware safety per §12/§9/§11.4.133, (7) cross-feature interaction (shared state/timing/hardware contention), (8) business-logic conformance (matches the spec AND the connected features’ contracts, not merely compiles). Forensic anchor (FACT, project-internal): the recents / 3-button-nav path shipped a latent AIDL-interface-contract mismatch that PASSED code review yet

failed at RUNTIME ONLY (ANR on a recents-reaping gesture) because no review angle interrogated the interface contract / concurrency-lifecycle / silently-broken connected feature — the §11.4.108 SOURCE → ARTIFACT → RUNTIME → USER-VISIBLE gap caught only after the fact; §11.4.145 shifts that discovery LEFT. Each angle MUST name the TOOL(S) used + obtain the required DEPTH (the diff, the full changed unit, its declared contract, the spec section, every connected feature — NEVER the diff lines alone) + emit a captured-evidence conclusion per §11.4.5/ §11.4.69 (“looks fine” forbidden, §11.4.1); a genuinely-N/A angle is recorded `NOT_APPLICABLE: <reason>` per §11.4.6, never silently skipped. Output = a per-change impact-research REPORT (one section per angle: tool + evidence path + verdict) + a single GO/NO-GO that BLOCKS acceptance/commit/build on ANY unmitigated risk in ANY angle; the change is fixed/mitigated + affected angles re-researched, iterating to a CLEAN GO (zero unmitigated risk + zero warning) per §11.4.134 before the §11.4.125 review gate. Honest boundary (§11.4.6): a GO proves cross-angle internal-consistency + bounded blast-radius — it does NOT replace §11.4.108 runtime-signature verification nor §11.4.40 full-suite retest; it is one of MULTIPLE STRONG LAYERS, the FIRST research pass every change crosses. Composes/strengthens §11.4.92/.125/.108/.110/.134/.78/.79/.107/.85/.133/ §12/§9/.99/.6/§1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-145-PROPAGATION` (literal `11.4.145`) + recommended gate `CM-IMPACT-RESEARCH-PER-CHANGE` + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; accept a change with a report missing an angle or an unmitigated NO-GO → `CM-IMPACT-RESEARCH-PER-CHANGE` FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.145. Non-compliance is a release blocker. No escape hatch — no `--skip-impact-research`, `--single-angle-suffices`, `--author-researches-own-change`, `--diff-skim-OK`, `--no-go-overridable`, `--trivial-change-no-research` flag.

§11.4.146 — Reproduce-first test + same-test-confirms-fix + mandatory extend-to-all-cases workflow (User mandate, 2026-06-10). Every reported problem MUST be handled by a NAMED three-step test workflow — §11.4.146 does NOT re-author its component disciplines, it NAMES + BINDS them into the operator’s exact per-defect sequence and adds ONLY the two emphases the components leave implicit. **(STEP 1 — REPRODUCE-FIRST + INVESTIGATE)** before any fix, author the §11.4.43 RED test as a §11.4.115 RED-baseline-on-the-broken-artifact (reproduce the defect on the CURRENT pre-fix artifact, capture defect-present physical evidence per §11.4.5/§11.4.69/§11.4.107); NEW

EMPHASIS (D1) that same RED test is ALSO a deliberate investigation instrument per §11.4.102(A) — it MUST gather ADDITIONAL forensic data characterising the defect (triggers, boundaries, input/topology scope, adjacent failure modes) that feeds BOTH the fix design AND the STEP-3 extend scope; a RED test used only as a binary present/absent gate with no characterisation satisfies §11.4.115 but NOT §11.4.146 STEP 1. **(STEP 2 — SAME-TEST-CONFIRMS-FIX)** after the fix the SAME test source (the §11.4.115 polarity switch flipped `RED_MODE=1→0`) confirms the defect ABSENT — RED-on-broken then GREEN-on-fixed, both captured, on a clean target per §11.4.108/§11.4.139, validated-first-after-redeploy per §11.4.130, deterministic per §11.4.50; no separate happy-path test substitutes (the §11.4.43/§11.4.115 PASS-bluff). **(STEP 3 — EXTEND-TO-ALL-CASES, mandatory per-fix)** NEW EMPHASIS (D2) immediately after STEP 2 the test MUST be FANNED OUT across the full case-space of the SAME functionality — all flows, valid + invalid + boundary (§11.4.85 empty/max/off-by-one) + concurrent (§11.4.85 contention) + failure-injection (§11.4.85 chaos) + topology variants (§11.4.3) — confirming no issue of any kind, with PROVABLE enumerated coverage per §11.4.118 (a listed case-set with per-case outcome, never “no other issues found”); a REQUIRED step, NOT deferred to a release-cycle discovery sweep and NOT reduced to a single guard; each user-visible case carries rock-solid physical evidence per §11.4.123 + is registered into the §11.4.135 standing regression-guard suite; a newly-discovered fan-out defect triggers §11.4.4 test-interrupt + re-enters STEP 1. (ANTI-BLUFF, all steps) every PASS is rock-solid CAPTURED physical evidence per §11.4.123 (§11.4.5/§11.4.69/§11.4.107) — metadata-only / config-only / absence-of-error / grep-without-runtime PASS forbidden (§11.4/§11.4.1); unclear validation method ⇒ deep-research-before-declaring-untestable per §11.4.123/§11.4.8/§11.4.99; an operator-found defect the green suite missed triggers §11.4.138. Honest boundary (§11.4.6): STEP 3 reduces the unknown-unknown surface but does not prove zero remaining defects (§11.4.118) — un-exercised cases stated as honest gaps, never silently implied clean. Composes

§11.4.43/.115/.102/.130/.108/.139/.50/.85/.118/.135/.3/.123/.5/.69/.107/.138/.4/§107/ §1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-146-PROPAGATION` (literal `11.4.146`) + recommended gate `CM-REPRODUCE-FIRST-THEN-EXTEND` (every closed defect’s fix carries a §11.4.115 reproduce-first polarity test with captured defect-characterisation [STEP 1] + its `RED_MODE=0` GREEN confirmation [STEP 2] + an enumerated per-functionality extend case-set registered into the §11.4.135 suite [STEP 3]; a closure with the reproduce → confirm pair but NO enumerated extend case-set FAILs) + paired §1.1 meta-test mutation (strip the

literal → propagation gate FAILs; close a defect with only the reproduce → confirm pair and no extend-case-set → `CM-REPRODUCE-FIRST-THEN-EXTEND` FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.146. Non-compliance is a release blocker. No escape hatch — no `--skip-reproduce-first`, `--fix-without-red`, `--skip-extend-to-all-cases`, `--reproduce-confirm-suffices`, `--defer-extend-to-release-sweep`, `--single-guard-suffices`, `--no-defect-characterisation` flag.

§11.4.147 — Crashed-agent respawn-until-complete + no-work-loss registry mandate (User mandate, 2026-06-10). Verbatim operator intent: “any agent that crashed because of something MUST BE respawned and finish its work at some point! We MUST NOT lose any work, forget about or have it corrupted!” Forensic case study (FACT): dispatched subagents died on TRANSIENT causes (`API Error: Server is temporarily limiting requests rate-limit killed 5 at once`, `API Error: socket connection closed unexpectedly killed 2, one left` PARTIAL edits — two source files written, the dependent SystemUI sliders + doc + test NOT done), each re-dispatched by pure conductor vigilance with NO mechanical guarantee — the moment conductor context is lost / compacted / the conductor itself crashes, an in-flight-but-dead work unit is silently dropped (lost / forgotten / corrupted). Every dispatched agent/subagent MUST be tracked through its full lifecycle so a crash NEVER loses, forgets, or corrupts its work; a crashed agent is NOT a completed agent (§11.4.6 — “it probably finished before dying” is a forbidden guess; abnormal termination is positive evidence the unit is OPEN). The mandate (ALL hold): **(a) durable agent REGISTRY** — every dispatched agent has an append-only machine-readable registry entry (stable id, task + §11.4.58 file-scope, declared output path(s) + §11.4.5/§11.4.69 evidence dir, dispatch timestamp, status from the CLOSED SET `{dispatched | in-flight | crashed | respawned | complete}`), reusing the §11.4.116 sync substrate (JSONL event stream + atomic status snapshot; “an entry with no dispatch-event cannot show `complete` ”); the registry is the SINGLE SOURCE OF TRUTH for “is this work owed?” and an unregistered dispatch is itself a violation; **(b) mechanical CRASH-DETECTION + RESPAWN-until-complete** — any abnormal termination (transient rate-limit/socket-close, any non-completion exit, or a terminal error after the runtime’s own retries are exhausted) flips the entry to `crashed` + keeps the unit OPEN, and the unit is respawned (fresh agent claims the same id + scope + preserved state) and re-respawned until an agent reaches `complete` ; respawn is the safe/reversible/bounded decision taken autonomously per §11.4.101 (never

blocks the loop for a human), transient causes use the project's ALREADY-DEFINED backoff budgets (never invented, §11.4.6), a deterministically-reproducible non-transient terminal error is investigated per §11.4.102 before further respawn (never a blind retry loop); **(c) PARTIAL-STATE preserve → §11.4.84-check → resume-or-clean-restart** — the crashed agent's uncommitted edits + output doc are PRESERVED (never silently discarded → lost, never blindly committed → corruption), the respawn runs the §11.4.84 quiescence check on the preserved tree (every modified file accounted-for vs scope, no mutation/ // always pass / `_mutated_*` residue, no half-written/torn artifact), then EITHER RESUMES idempotently when it passes OR CLEAN-RESTARTS from a known-good base (§9.2 pre-op backup, reversible §11.4.101) when inconsistent — nothing lost, nothing corrupted; per-agent `git worktree` isolation (§11.4.58 L4 / §11.4.84) keeps the partial tree from contaminating other streams; **(d) “crash ≠ done” COMPLETION criterion** — a unit is DONE only when an agent reaches `complete` with its required captured evidence/output landed (§11.4.5/§11.4.69 + the §11.4.116 verdict-carries-evidence-path rule); the endless-loop done-condition (§11.4.87/§11.4.94/§11.4.97/§11.4.126) MUST NOT read satisfied while any entry is `dispatched / in-flight / crashed / respawned -not-yet- complete`, the zero-idle survey (§11.4.94) treats every non- `complete` entry as an OPEN item to reclaim, and a registry showing `complete` without landed evidence is a §11.4 PASS-bluff at the agent-lifecycle layer. Honest boundary (§11.4.6): the registry + respawn guarantee work is not lost/corrupted, NOT that it is correct — the respawned output still crosses §11.4.108 + §11.4.125/§11.4.142 review + §11.4.40 retest; §11.4.147 is the durability layer beneath those, the agent-side analogue of §11.4.144 (same detect → wait/backoff → re-attach/respawn → escalate shape). Composes §11.4.6/.58/.84/.87/.94/.97/.101/.116/.102/.108/.125/.142/.40/.128/.144/§9.2/§1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-147-PROPAGATION` (literal `11.4.147`) + recommended gate `CM-CRASHED-AGENT-RESPAWN-TRACKED` + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; mark a `crashed` entry as the loop's done-condition `complete` without landed evidence, OR drop a dispatched agent without a registry entry → `CM-CRASHED-AGENT-RESPAWN-TRACKED` FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.147. Non-compliance is a release blocker. No escape hatch — no `--skip-agent-registry`, `--crash-equals-done`, `--no-respawn`, `--discard-partial-state`, `--blind-commit-partial`, `--forget-dead-agent`, `--loop-done-with-crashed-entries` flag.

§11.4.148 — Workable-item integrity (status+type+id) + comprehensive structured description + bidirectional external-tracker sync + BLOCKED unblock-choices mandate (User mandate, 2026-06-10). BINDS + STRENGTHENS the workable-item discipline (§11.4.15 status / §11.4.16 type / §11.4.54 id / §11.4.21 Operator-blocked / §11.4.91 description-clarity / §11.4.93 + §11.4.95 SQLite SSoT / §11.4.106 docs_chain) into ONE integrity contract spanning DB ↔ docs ↔ external tracker, adding the operator's three emphases: **(D1) no item without a valid status + valid type + stable id — on ALL three surfaces** (an item missing any of the three in the DB, the rendered docs, OR the external tracker FAILs the validator — release-blocker; the id is the cross-surface binding key); **(D2) comprehensive structured description per item** — WHAT it is (§11.4.91 ≥6-word/ ≥40-char clear meaning) + HOW it manifests + HOW to reproduce (§11.4.115/.146) + ACCEPTANCE CRITERIA (§11.4.123/.69 captured-evidence verdict), in the §11.4.93 DB description column AND the docs AND the tracker, never a stub/ §11.4.91-anti-pattern fragment; **(D3) BLOCKED items carry WHY + enumerated unblock CHOICES** — tightens §11.4.21: every operator-blocked item (incl. Blocked / BLOCKED documented alias normalised to canonical operator-blocked , never a silent fork §11.4.6) MUST enumerate the closed list of decisions/ actions that would unblock it ([A]...·[B]...·[C]... , mirroring §11.4.66's 2–4-option shape); a BLOCKED item with no enumerated choices FAILs; **(D4) regular never-missed bidirectional DB↔docs↔tracker sync** — the §11.4.93/.95 git-tracked SQLite DB is the SSoT, docs + tracker DERIVED; full sync (db-to-md / md-to-db byte-identical round-trip §11.4.93 + tracker push) runs regularly + on every change, §11.4.86 drift-proof fingerprint (sha256 of the sorted item keyset, NOT mtime) gates freshness, §11.4.106 docs_chain-bound so drift is mechanically caught not vigilance-dependent; **(D5) generic external-tracker push** carries statuses (collapsed onto the tracker's native set per §11.4.33/.112 when fixed, precise value preserved in a header, never lost), types, assignee (§11.4.104 participant handle, UNSET defaults from a project env var, never hardcoded/logged §11.4.10), and sub-tasks, IDEMPOTENT (dry-run-then-real, match-by-stable-key [<ID>] prefix / id custom-field, present⇒UPDATE/absent⇒CREATE, rate-limited, credential-redacted, sink-side created=N updated=M failed=0 proof §11.4.69), the MACHINERY project-agnostic §11.4.28 (consumer registers its tracker/list/field-map at runtime). Anti-bluff §11.4: every sync + validator pass carries captured evidence §11.4.5/.69; honest boundary §11.4.6 — guarantees well-formed items + agreeing surfaces, NOT that the underlying work is correct (still crosses §11.4.108/.40/.123). Composes

§11.4.15/.16/.21/.33/.34/.54/.66/.86/.91/.93/.95/.104/.106/.112/.123/.10/.28/.69/.6/

§1.1. Classification: universal (§11.4.17) — consumer supplies its DB path, id prefix, tracker service + list/board id + field map + default-assignee env var, docs_chain context per §11.4.35. Propagation gate CM-COVENANT-114-148-PROPAGATION (literal 11.4.148) + recommended gates CM-ITEM-INTEGRITY-STATUS-TYPE-ID / CM-ITEM-COMPREHENSIVE-DESCRIPTION / CM-BLOCKED-UNBLOCK-CHOICES / CM-TRACKER-SYNC-IDEMPOTENT + paired §1.1 mutation (gate-code = separate work item). **Canonical authority:** constitution submodule Constitution.md §11.4.148. Non-compliance is a release blocker. No escape hatch — no --item-without-status , --item-without-type , --item-without-id , --stub-description-OK , --blocked-without-choices , --skip-tracker-sync , --one-way-sync-OK , --non-idempotent-tracker-push , --hardcode-assignee flag.

§11.4.149 — Per-workable-item testing-diary mandate (User mandate, 2026-06-10). Every workable item MUST carry an append-only TESTING DIARY — a chronological record of every test run against it — distinct from §11.4.93 item_history (lifecycle STATE transitions) + §11.4.55 Reopens (reopen cycles): the diary records TEST EXECUTIONS (which may or may not change status). The mandate (ALL hold): **(a)** a test_diary table additive to the §11.4.93/.95 SQLite SSoT, one row per run soft-keyed to the item id, capturing ISO-8601-UTC date_time + tested_by (closed set User|Operator|AI-agent|HelixQA) + result (§11.4.45 PASS|FAIL|SKIP + detail) + in-depth observations (long markdown, facts or explicit UNCONFIRMED: / PENDING_FORENSICS: §11.4.6, the background a future fix needs) + action_taken + status_changed +from/to (did this run change status + WHY) + §11.4.69 evidence_path + feature_class , with a SCHEMA CONSTRAINT making a PASS row WITHOUT a non-empty evidence path impossible (a PASS-bluff rejected by the schema itself); **(b)** BOTH an in-depth diary doc + a derived at-a-glance summary VIEW (total/pass/fail/skip + last-verdict + last-run + status-changes + distinct testers/feature-classes, DERIVED never duplicated §11.4.93) feeding §11.4.132 risk-ordering; **(c)** four-format per-item Diary / Diary_Summary exports §11.4.65 + §11.4.86-drift-proof-fingerprinted (sha256 of the sorted diary keyset) + §11.4.106 docs_chain-bound so a diary row not exported/pushed FAILs a gate (never-missed); **(d)** external-tracker SUB-TASK model — the item task's description stays the item (§11.4.148 D2, NOT polluted with diary text), each run a CHILD sub-task (type task) with a {TODO|In-progress|Completed} lifecycle (collapsed per §11.4.33/.112), observations + action + an Evidence: line (path not raw artefact §11.4.10/.13) + a diary-entry

idempotency key, reusing the §11.4.148 D5 idempotent rate-limited credential-redacted sink-side-proven push, mapper project-agnostic §11.4.28; **(e)** MINIMAL-LLM deterministic bash/Go tooling (zero LLM in the data path — the `observations` prose is authored by whoever ran the test; tooling only stores/renders/pushes/validates), 100% test-covered §11.4.27 (unit + integration-against-the-real-tracker with an honest §11.4.3 SKIP-when-token-absent never a faked PASS + export + HelixQA Challenge + paired §1.1 mutation) so a diary PASS-bluff is mechanically impossible. Honest boundary §11.4.6 — the diary guarantees a complete auditable per-item test-history, NOT that any single run's verdict is correct (rests on its own §11.4.69/.107/.123 evidence). Composes §11.4.6/.27/.45/.50/.55/.65/.69/.86/.93/.95/.106/.107/.123/.132/.148/.10/.13/.28/.33/.112/ §1.1. Classification: universal (§11.4.17) — consumer supplies its DB path, diary doc layout, export formats, tracker sub-task field map, docs_chain context per §11.4.35. Propagation gate `CM-COVENANT-114-149-PROPAGATION` (literal `11.4.149`) + recommended gates `CM-TEST-DIARY-SYNC` / `CM-DIARY-PASS-REQUIRES-EVIDENCE` + paired §1.1 mutation (gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.149. Non-compliance is a release blocker. No escape hatch — no `--skip-testing-diary`, `--diary-without-evidence`, `--no-diary-summary`, `--diary-without-tracker-subtask`, `--llm-in-diary-data-path`, `--diary-export-optional` flag.

§11.4.150 — Mandatory deep multi-angle web research per change/issue, before declaring fixed or structural (User mandate, 2026-06-11). Verbatim operator mandate: “For every single issue we fix or improvement we make — besides tight systematic-debugging, fixing, code review by independent agents, comprehensive tests — we MUST ALWAYS do deep web research from various angles! No matter how big/small/simple/complex, dig deep the internet for articles, technical documentation, APIs, open-source code — EVERYTHING that can help make the best possible solution OR confirm we don't have a more serious problem we're unaware of! We MUST do everything possible to STOP the constant issue-reopening and finally start closing items as fixed+working! Do this ALWAYS in parallel with the main work stream!” For EVERY fix / improvement / change / closure — no matter how big, small, simple, or complex — the agent MUST, IN ADDITION to tight systematic-debugging (§11.4.102), the fix, independent-agent code review (§11.4.125 / §11.4.142), and comprehensive multi-layer tests (§11.4.4(b) / §11.4.40), perform a DOCUMENTED **deep multi-angle web research pass** digging the internet from VARIOUS angles — articles, official + vendor technical documentation,

API references, standards, issue trackers, maintainer guidance, reusable open-source code — to BOTH (i) discover the best possible solution AND (ii) confirm there is NOT a more serious underlying problem the team is unaware of. Forensic case study (FACT, 2026-06-11): a subtitle-on-secondary-display goal verdicted Won't-fix: structurally-impossible (§11.4.112 secure-surface pixel-blanking) was OVERTURNED by a deep multi-angle research pass that surfaced the real OCR-of-the-PRIMARY-display path — a “we can’t” became a shipped capability; the canonical anti-pattern of a structural verdict reached for want of research. The mandate (ALL hold): **(A) No closure-as-fixed/structural WITHOUT a documented deep-research pass** — no item may be marked Fixed / Implemented / Completed (§11.4.33) OR classified structurally-impossible won't-fix (§11.4.112) until a deep multi-angle pass is documented; §11.4.150 makes §11.4.8's pass UNCONDITIONAL (every item, however trivial, at the closure AND structural-verdict gates). **(B) Multiple angles, not a single lookup** — ≥ 2 genuinely-distinct angles (official-docs / standards / known-bug-trackers / alternative-approach / failure-mode / security / performance / platform-constraint / open-source-precedent) so a single-source confirmation-bias miss (§11.4.145) cannot pass; a one-link drive-by is NOT deep multi-angle. **(C) Confirm-no-bigger-problem, not just find-a-fix** — explicitly seek evidence the fix does not mask a deeper defect AND the closure/verdict is genuinely safe; “we found nothing worse” requires the enumerated search (§11.4.118). **(D) Latest-source + cited** — LATEST authoritative versions per §11.4.99 (never training-data/memory), each cited by URL + access date in the research artefact AND the closure commit footer (Deep-research <date>: <urls> OR the literal NO external solution found – original work per §11.4.8). **(E) Reopen-breaking is the PURPOSE** — STOP the constant Fixed↔Reopened churn (§11.4.34 / §11.4.55); a reopen whose root cause a deep pass would have surfaced is a §11.4.150 miss; composes §11.4.7 (demotion-evidence) + §11.4.112 (structural verdict now REQUIRES the cited-authorities pass). **(F) ALWAYS in parallel with the main stream** — background subagent-driven (§11.4.70 / §11.4.20 / §11.4.103 / §11.4.89) concurrent with main fix/build/test work, NEVER serialising it or stalling the loop (§11.4.94 / §11.4.97 / §11.4.101 / §11.4.126). **(G) Apply ASAP to every workable item** — retroactively + going forward across the §11.4.93 / §11.4.95 SSoT; items closed without a documented pass are re-audited in the §11.4.40 / §11.4.42 release-gate sweep. Honest boundary (§11.4.6): the pass reduces the unknown-unknown surface + breaks the most common reopen causes — it does NOT prove zero remaining defects (§11.4.118) and does NOT replace §11.4.108

runtime-signature verification, §11.4.125 / §11.4.142 review, or §11.4.40 retest; it is the research layer every fix/closure/structural-verdict additionally crosses, and “we probably don’t have a bigger problem” without the enumerated multi-angle search is a guess (§11.4.6), never a finding. Composes §11.4.8 / §11.4.99 / §11.4.123 / §11.4.118 / §11.4.145 / §11.4.125 / §11.4.142 / §11.4.7 / §11.4.112 / §11.4.34 / §11.4.55 / §11.4.70 / §11.4.20 / §11.4.89 / §11.4.103 / §11.4.40 / §11.4.42 / §11.4.93 / §11.4.95 / §11.4.6 / §1.1. Classification: universal (§11.4.17) — the consuming project supplies its concrete research corpora, angle set, item tracker, and closure-commit-footer convention per §11.4.35. Propagation gate `CM-COVENANT-114-150-PROPAGATION` (literal `11.4.150`) + recommended gate `CM-DEEP-RESEARCH-PER-ISSUE` (every closed/structural-verdicted item carries a documented multi-angle deep-research artefact + cited-source closure footer, run in parallel, before the closure/structural verdict is accepted) + paired §1.1 mutation (strip the literal → propagation gate FAILs; close an item or reach a structural verdict with no documented multi-angle research artefact / cited footer → `CM-DEEP-RESEARCH-PER-ISSUE` FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.150. Non-compliance is a release blocker. No escape hatch — no `--skip-deep-research`, `--single-source-suffices`, `--trivial-change-no-research`, `--close-without-research`, `--structural-without-research`, `--serialise-research`, `--research-from-memory-OK` flag.

§11.4.151 — Project-prefixed release-tag/version-naming mandate (User mandate, 2026-06-12). Verbatim operator mandate: “Every release tag and version name we create — on the main repository and on every Submodule we own — MUST be prefixed with the project’s release prefix, e.g. `myproject-1.0.0-dev-0.0.1`. The prefix MUST come from `HELIX_RELEASE_PREFIX` in our `.env` if it is set, otherwise from the lowercased project root directory name. The SAME prefix MUST be used across the main repo and all owned Submodules in one release so a release is greppable across every repository.” Every release tag AND every version name created on the main repository AND on every owned-by-us submodule (§11.4.28) MUST be prefixed with the project’s release prefix, form `<PREFIX>-<version>` (canonical example: `myproject-1.0.0-dev-0.0.1`), so a release is identifiable + greppable across every repository it spans (one `git tag -l '<PREFIX>-*'` enumerates the whole release surface). **Prefix resolution order (closed-set, deterministic — §11.4.6 no-guessing):** (1) `HELIX_RELEASE_PREFIX` from the project’s `.env` — authoritative when set; `.env` is git-ignored per §11.4.30 and the variable is documented in the tracked

`.env.example` (a §11.4.77 re-obtain mechanism, never committed); (2) fallback = the lowercased snake_case form of the project root directory name (no spaces) per §11.4.29 — used whenever the env var is unset/empty, so a prefix is ALWAYS resolvable from the checkout with zero operator input. **The prefix MUST be IDENTICAL across the main repo and all owned submodules within a single release** — a release that tags the main repo `<PREFIX>-1.2.0` while tagging an owned submodule with a different/unprefixed value is a §11.4.151 violation (the cross-repo grep no longer enumerates the release). Version codes increment monotonically within the prefix (`<PREFIX>-...-0.0.1` → `<PREFIX>-...-0.0.2` → ...), never reset, never skipped. Honest boundary (§11.4.6): the prefix guarantees a release is identifiable + uniform across every repository, NOT that its contents are correct — the tag is still created only after the §11.4.40 full-suite retest GREEN and reaches every upstream via the §11.4.113 merge-onto-latest-main path (NEVER a force-push), fanned out per §2.1. Composes §2.1 / §11.4.29 / §11.4.30 / §11.4.40 / §11.4.113 / §11.4.126 (the release-scope terminal condition is a published, prefixed tag). Classification: universal (§11.4.17) — the consuming project supplies its concrete prefix value + the `HELIX_RELEASE_PREFIX` env var per §11.4.35. Propagation gate `CM-COVENANT-114-151-PROPAGATION` (literal `11.4.151`) + recommended gate `CM-RELEASE-PREFIX-NAMING` (every release tag/version on the main repo + every owned submodule carries the resolved `<PREFIX>-` prefix, identical across the release) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; create an unprefixed or differing-prefix release tag → `CM-RELEASE-PREFIX-NAMING` FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.151. Non-compliance is a release blocker. No escape hatch — no `--no-release-prefix`, `--unprefixed-tag`, `--prefix-optional`, `--differing-submodule-prefix` flag.

§11.4.152 — Crashlytics-recorded-data continuous monitoring + systematic-debug + regression-test-coverage mandate (User mandate, 2026-06-13).

Verbatim operator intent: “For every project that has Firebase Crashlytics enabled / wired, we MUST continuously monitor ALL of the Crashlytics-recorded data — crashes, ANRs, performance traces, and non-fatals — systematically debug each, fix and improve, and cover everything with validation and verification tests. This MUST be checked regularly, with no false results and no bluff of any kind!” Every project that has Firebase Crashlytics enabled/wired (SDK linked into a shipping artifact, crash+non-fatal+ANR reporting active) MUST treat the Crashlytics console as a first-class captured-evidence channel from real end-user devices and continuously process every datum it records — an open Crashlytics issue with a

green test suite is a §11.4 PASS-bluff at the field-telemetry layer (a real user hitting a broken feature while the suite reports green). STRENGTHENS+COMPLETES §11.4.47: §11.4.47 owns the periodic REVIEW + dedup + Issue-creation pass that SURFACES Crashlytics/Analytics/Performance findings; §11.4.152 owns what happens to each surfaced item AFTER — the systematic-debug → fix/improve → regression-test-coverage lifecycle that drives it to a proven regression-immune closure (§11.4.47 finds it; §11.4.152 fixes it + proves it stays fixed; both mandatory, neither substitutes). The mandate (ALL hold): (1) **continuous monitoring of ALL four surfaces** — fatal crashes, ANRs, performance traces/regressions, AND non-fatals (skipping any one is a PASS-bluff; non-fatals are the silent class — every catch/fallback/recovered-error path is a real degraded UX and MUST be triaged, not ignored because the app did not crash); (2) **systematic-debugging of each (reproduce-before-fix)** per §11.4.102 Iron Law + §11.4.115 RED-baseline-on-the-broken-artifact (the recorded stacktrace/ANR-trace/non-fatal-context IS the §11.4.5/.69 captured defect-present evidence) BEFORE the fix — a fix without a prior reproducing test is a §11.4.43/.123 violation; unreproducible ⇒ deep-research-before-declaring-untestable §11.4.123/.150; (3) **a fix/improvement for every confirmed issue** against its proven root cause (§11.4.9/.43/.108), “acknowledged in console” is not a fix, muting a recurring issue without a tracked rationale is a §11.4.6/.90 violation; (4) **validation+verification regression-test coverage per closed issue** — in the SAME commit as the fix register a permanent §11.4.135 regression guard (§11.4.115 polarity test: `RED_MODE=1` captures the recorded defect on the pre-fix artifact, `RED_MODE=0` standing GREEN guard asserting ABSENT) exercising the same user-reachable path the stacktrace identifies, with rock-solid captured evidence §11.4.5/.69/.107/.123 + a closure log citing the console issue id/URL + root-cause + fix commit + the validation/verification test paths; a Crashlytics issue marked resolved WITHOUT a falsifiable regression test is FORBIDDEN (the silent-recurrence vector, §11.4.138 operator-escape class applied to field telemetry); (5) **regular cadence** reusing the §11.4.47 five-trigger set (pre-build/pre-flash/pre-distribute/pre-tag blocking; daily + post-deployment-burn-in non-blocking) — a one-time sweep never repeated is a violation; (6) **no false results, no bluff** (§11.4/.1/.6 — “no new issues” requires the enumerated monitored-surfaces+window §11.4.118; “fixed” requires the RED → GREEN flip §11.4.115/.130; a guard whose §1.1 mutation does not FAIL it is a bluff gate). Honest boundary §11.4.6: processing every recorded issue breaks the silent-recurrence vector — it does NOT prove zero remaining field defects nor replace §11.4.108/.40; an issue is “closed” only when its guard is GREEN on a clean artifact, never when the console

mark is flipped. Classification: universal (§11.4.17) — the consuming project supplies its Crashlytics handle, console-access credential (§11.4.10, never logged), severity table, and per-issue closure-log path per §11.4.35; the reference project-level instantiation is a consuming project's §6.O (Crashlytics-Resolved Issue Coverage — per-issue validation test + Challenge test + `.lava-ci-evidence/crashlytics-resolved/<date>-<slug>.md` closure log) + §6.AC (Comprehensive Non-Fatal Telemetry — every catch/fallback records a non-fatal with triage context). Composes

§11.4/.1/.5/.6/.9/.34/.40/.43/.47/.69/.90/.102/.107/.108/.115/.118/.123/.130/.135/.138/.150/

§1.1. Propagation gate `CM-COVENANT-114-152-PROPAGATION` (literal `11.4.152`) + recommended gate `CM-CRASHLYTICS-ISSUE-FULLY-COVERED` (every closed Crashlytics issue carries a systematic-debug root-cause record + a registered §11.4.135 regression guard + a closure-log entry citing the console issue id/URL + the validation/verification test paths; a console-resolved issue with no falsifiable regression guard FAILs) + paired §1.1 meta-test mutation (gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md`

§11.4.152. Non-compliance is a release blocker. No escape hatch — no `--skip-crashlytics-monitoring`, `--monitor-crashes-only`, `--skip-non-fatals`, `--resolve-without-regression-test`, `--console-mark-is-fixed`, `--monitor-once`, `--mute-without-rationale` flag.

§11.4.153 — Comprehensive per-feature Status + Status_Summary document set with mandatory video-recording confirmation (User mandate, 2026-06-15).

Every project MUST maintain, under `docs/features/`, a comprehensive feature Status document set (`Status.md` + its §11.4.56 `Status_Summary.md` companion) enumerating EVERY system component, EVERY client app/binary/surface (TUI / CLI / Web / desktop / mobile / API / gRPC / library / submodule / infrastructure), and EVERY feature — including features ported from any incorporated CLI-agent / submodule catalogue (§11.4.74) — with NO single feature left out. (1) **Total categorized coverage** — per-feature table organized Component → Category, reconciled against the actual codebase (a code-present feature missing from the table, or a row with no code, is a §11.4.6/§11.4.118 bluff). (2) **Per-feature fields** — Component / Feature / Category / Implementation / Wiring (genuinely end-user-reachable, not merely compiled, §11.4.108) / Real-use / Tests-coverage (four-layer §11.4.4(b)) / Validation (PASS / FAIL / SKIP / PENDING_FORENSICS / OPERATOR-BLOCKED per §11.4.45) / **Video-recording confirmation** (path to the real-use video or honest gap marker). (3) **Mandatory per-feature real-use video** — every user-visible “confirmed” claim backed by a recorded real-use video of a

genuine end-user scenario (real prompts → real LLM/service responses → real results), NEVER a frozen/stale frame (§11.4.107), NEVER faked/mock/demo-loop/bluff response or LLM error passed off as success (§11.4.2/§11.4.5), stored at the project-declared recording path (§11.4.35); a confirmed row with no real video or a bluff video = §11.4 PASS-bluff; autonomous-infeasible ⇒ honest §11.4.3/§11.4.52 SKIP + tracked migration item, NEVER a faked confirmation. (4) **Video-analysis remediation loop** — every video analysed; any defect it surfaces triggers §11.4.102 systematic-debug → fix → §11.4.146 retest → re-record → clean GO per §11.4.134 before “confirmed” (a video exposing a broken feature is a §11.4.4 test-interrupt). (5) **Always-in-sync** — §11.4.45-class roster/corpus-backed, §11.4.106 docs_chain-bound + §11.4.86 drift-proof fingerprint (sha256 of sorted feature-key roster AND sorted video-artefact roster, NOT mtime), re-syncs out-of-the-box; stale = violation. (6) **Four-format export** — HTML + PDF + **DOCX** (this doc class ADDS DOCX to the §11.4.65 HTML+PDF set; other classes unchanged), in sync per §11.4.60. (7) Follows §11.4.44/.45/.56/.57/.59/.60. (8) **MP4 format REQUIRED**. All video confirmations MUST be in `.mp4` format (H.264). Window-specific capture ONLY (§11.4.159(A)). Vision validation REQUIRED (§11.4.159(D)). `.cast` files are supplementary only. Honest boundary §11.4.6 — guarantees a complete video-confirmed always-synced ledger, NOT per-feature correctness (rests on each row's §11.4.69/.107/.123 evidence) and NOT a §11.4.40 retest substitute. Classification: universal (§11.4.17). Composes §11.4.2/.5/.44/.45/.52/.56/.57/.59/.60/.65/.86/.102/.106/.107/.108/.118/.123/.134/.146/ §1.1. Propagation gate `CM-COVENANT-114-153-PROPAGATION` (literal `11.4.153`) + recommended gates `CM-FEATURE-STATUS-COMPLETE` + `CM-FEATURE-STATUS-VIDEO-CONFIRMED` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.153. Non-compliance is a release blocker. No escape hatch — no `--skip-feature-status`, `--feature-without-video`, `--frozen-video-OK`, `--bluff-response-video-OK`, `--skip-video-analysis`, `--feature-ledger-incomplete-OK`, `--no-docx-export`, `--allow-feature-ledger-drift` flag.

§11.4.154 — Window-scoped capture + fresh-corpus rotation for feature/QA recordings (User mandate, 2026-06-15). Verbatim: “recording you perform does record the window containing apps and services, not the whole desktop or monitor screen!” + “All old recording files MUST BE removed when new one starts!” Refines §11.4.2/.5/.107/.153 recording discipline with two capture-hygiene invariants. **(A) Window-scoped, NOT whole-screen** — every feature/QA video MUST capture ONLY the window/surface of the app/service under test (GUI window / CLI-TUI

terminal pane / web tab-viewport / device-emulator-simulator frame), NEVER the whole desktop/monitor or unrelated windows; whole-desktop capture leaks operator-private content (§11.4.10/.83), dilutes the §11.4.107 liveness/freeze oracle, and breaks the §11.4.137 OCR/ROI content oracle. Target the window/region by stable identity (window id/title, device serial, browser context, tmux target) per §11.4.111 — never a fixed full-screen device index. Platform genuinely cannot capture below whole-screen ⇒ honest §11.4.3 SKIP + tracked migration item, never a whole-screen pass-off. **(B) Fresh-corpus rotation** — when a new recording run for a scope begins, the agent's OWN prior in-scope stale recordings at the raw recording path MUST be removed FIRST so the live corpus reflects the current run (§11.4.107 not-stale + §11.4.86 roster-freshness). Honest boundary (§11.4.6 + §9.2): “remove old” = the agent's own prior recordings for the SAME scope/project ONLY — NEVER another project's/scope's/operator-authored files; uncertain ⇒ surface, don't delete (§11.4.122); committed docs/qa/<run-id>/ evidence (§11.4.83) is the durable record, NOT rotated. Classification: universal (§11.4.17). Composes §11.4.2/.5/.10/.83/.86/.107/.111/.122/.128/.137/.153/§9.2/.6. Propagation gate CM-COVENANT-114-154-PROPAGATION (literal 11.4.154) + recommended gate CM-WINDOW-SCOPED-FRESH-CORPUS-RECORDING + paired §1.1 mutation.

Canonical authority: constitution submodule Constitution.md §11.4.154. Non-compliance is a release blocker. No escape hatch — no

--whole-screen-capture-OK , --skip-window-scope , --keep-stale-recordings , --no-corpus-rotation , --full-desktop-recording flag. **(C) MP4 auto-conversion REQUIRED.** Any .cast file produced MUST be auto-converted to .mp4 via agg + ffmpeg immediately after capture. The .mp4 is the primary evidence; .cast is supplementary only.

§11.4.155 — Project-name-prefixed feature/QA recording filenames (User mandate, 2026-06-15). Verbatim: “All recorded videos MUST START with prefix: the PROJECT NAME (ALWAYS USE THE PROJECT NAME). Project name MUST be obtained according to the constitution's own project-name resolution.” Every recorded video the project produces — every feature/QA real-use recording (§11.4.153), every window-scoped capture (§11.4.154), every always-on device recording (§11.4.128), every raw or curated artefact at the project-declared recording path (§11.4.35) + the committed docs/qa/<run-id>/ trail (§11.4.83) — MUST have a filename that STARTS WITH the PROJECT-NAME prefix, ALWAYS; an unprefixed recording is a §11.4.155 violation (a multi-project/scope corpus on one host per §11.4.128/.103 becomes un-greppable + un-attributable — the §11.4.151 identify-and-grep failure on the recording-corpus axis). **Prefix resolution**

(closed-set, deterministic — §11.4.6, IDENTICAL to §11.4.151): (1)

`HELIX_RELEASE_PREFIX` from `.env` (authoritative, git-ignored §11.4.30, documented in tracked `.env.example` §11.4.77) else (2) lowercased snake_case project-root dir name §11.4.29 — ALWAYS resolvable, zero operator input. SAME prefix for EVERY recording in a checkout so `ls '<PREFIX>--- '*` enumerates the corpus; canonical form `<PREFIX>---<feature-or-scope>---<run-id>.<ext>` (`---` delimits the prefix unambiguously); MUST equal the §11.4.151-resolved release-tag prefix (divergence is itself a §11.4.155 violation — one project, one name). Honest boundary (§11.4.6): the prefix guarantees attribution + greppability, NOT content validity (still §11.4.107/.137/.153) and does NOT relax §11.4.154's window-scope/rotation (rotation removes the agent's OWN `<PREFIX>--- *` only; foreign/operator files surfaced not deleted §11.4.122/§9.2). Classification: universal (§11.4.17). Composes §11.4.151/.128/.153/.154/.111/.83/.6/.29/.30/.35/.77/.86/§1.1. Propagation gate `CM-COVENANT-114-155-PROPAGATION` (literal `11.4.155`) + recommended gate `CM-RECORDING-PROJECT-NAME-PREFIX` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.155. Non-compliance is a release blocker. No escape hatch — no `--no-recording-prefix` , `--recording-without-project-name` , `--unprefixed-recording` , `--prefix-optional-for-recording` , `--differing-recording-prefix` flag.

§11.4.156 — All CI/CD automation (GitHub Actions / GitLab pipelines / equivalents) MUST be disabled (User mandate, 2026-06-15). Verbatim operator mandate: “Any GitHub actions or GitLab pipelines MUST BE disabled! Add this critical mandatory rule / mandatory constraint into the root constitution Submodule, commit and fetch all its changes to all upstreams and make sure we respect and follow this rule (we do apply it) ASAP!!!” Every repository this Constitution governs — main repo, this constitution submodule, every owned + nested submodule we author and push — MUST ship with ALL server-side CI/CD automation DISABLED: no push to any owned upstream may trigger a GitHub Actions run, GitLab pipeline, or equivalent (Jenkins/CircleCI/Travis/Drone/Woodpecker/Bitbucket/Azure, any `on: push / schedule / workflow_dispatch`). GENERALISES + makes ABSOLUTE the §11.4.75 Layer-5 posture (remote CI DISABLED, workflow preserved at a `...disabled-local-only` non-`.yaml` name a provider ignores) across ALL governed repos; enforcement migrates to the LOCAL §11.4.75 git-hook ritual + §11.4.40 pre-tag sweep, never a remote runner. ALL hold: **(A)** zero active `.github/workflows/*.yaml|yml` / `.gitlab-ci.yml` / `.gitlab/**` / equivalent at the ROOT of any governed repo/submodule (the only place a provider executes);

(B) “disabled” = a push triggers ZERO runs — delete OR rename to a non-trigger name (§11.4.75 `.disabled / .disabled-local-only`); a live- on: + `if:false` workflow still queues runs, NOT compliant; (C) scope = repos we author+push — vendored/third-party nested configs below the root (AOSP `external/**`, `prebuilts/**`, vendored submodules) are INERT (a provider never runs a non-root config), OUT of scope, MUST NOT be mass-edited (§11.4.29 vendor-exempt); test = “does a push to OUR upstream trigger a run?” yes⇒disable, inert⇒document+leave (§11.4.6 verify-not-assume); (D) no new CI may be added (release blocker); (E) pre-push verify `git ls-files | grep -E '^\.github/workflows/.*\ya?ml$|^\.gitlab-ci\.yaml$'` empty for authored repos, §11.4.109-class PreToolUse guard + gate enforce mechanically. Honest boundary (§11.4.6): file-level disabling stops FILE-triggered runs, NOT provider-side server settings (org-default required workflows, branch-protection required checks, provider scheduled exports) — the operator turns those off; the agent documents what it cannot reach, never claims unachieved completeness. Composes §11.4.75/.29/.6/.40/.42/.109/.113/§2.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-156-PROPAGATION` (literal `11.4.156`) + recommended gate `CM-NO-ACTIVE-CI` + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; add a root `.github/workflows/x.yml` to an authored repo → `CM-NO-ACTIVE-CI` FAILs). **Canonical authority:** constitution submodule `Constitution.md` §11.4.156. Non-compliance is a release blocker. No escape hatch — no `--allow-ci`, `--enable-workflow`, `--keep-pipeline`, `--remote-ci-OK`, `--ci-exempt` flag.

§11.4.157 — GEMINI.md maintained in lockstep with CLAUDE.md /

AGENTS.md / QWEN.md (User mandate, 2026-06-15). Verbatim operator

mandate: “Make sure with CLAUDE.md, AGENTS.md, QWEN.md we maintain GEMINI.md too! Add this mandatory fact / rule to root constitution Submodule we are inheriting / extending - CONSTITUTION.md, CLAUDE.md, QWEN.md, AGENTS.md, GEMINI.md and other related relevant files!” Forensic FACT (2026-06-15): when §11.4.156/§11.4.157 were authored, Constitution/CLAUDE/AGENTS/QWEN carried the family through §11.4.155 but GEMINI.md had silently drifted to §11.4.141 — 14 mandates (§11.4.142–155) never propagated — a §11.4 propagation-bluff (a Gemini-CLI agent reads a stale Constitution). GEMINI.md is a FIRST-CLASS governance context carrier EQUAL to CLAUDE.md/AGENTS.md/QWEN.md, never optional/best-effort. ALL hold: (A) five-carrier lockstep — no governance change is complete until GEMINI.md carries it alongside the other three mirrors; GEMINI.md is added to the §11.4.26 propagation + cross-reference set

explicitly; **(B)** no silent drift — GEMINI.md lagging the other mirrors' highest rule is a §11.4.157 violation (§11.4.65-class), back-fill required; **(C)** equal status — GEMINI.md restates the SAME literal 11.4.N anchors the propagation gates require (§11.4.35), fleet count INCLUDES GEMINI.md; **(D)** consumer projects' own CLAUDE/AGENTS/QWEN/GEMINI bind too (§11.4.35). Honest boundary (§11.4.6): the §11.4.142–155 GEMINI.md back-fill is a tracked release-blocking remediation; claiming GEMINI.md “in sync” while the back-fill is incomplete is itself a §11.4.157 violation. Composes §11.4.26/.35/.17/.44/.65/.140/.156/§1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-157-PROPAGATION (literal 11.4.157, GEMINI.md INCLUDED) + recommended gate CM-GEMINI-MD-LOCKSTEP + paired §1.1 meta-test mutation. **Canonical authority:** constitution submodule Constitution.md §11.4.157. Non-compliance is a release blocker. No escape hatch — no --skip-gemini-md, --gemini-optional, --gemini-lag-OK, --four-carrier-suffices flag.

§11.4.158 — Intensive all-feature/flow/edge-case video-recording + read-the-screen content-verification mandate (User mandate, 2026-06-16). Every project MUST be covered by intensive automated testing that exercises + RECORDS every feature/flow/use-case/edge-case (valid/invalid/boundary/concurrent/failure-injection §11.4.85), each recording showing the feature GENUINELY WORKING with REAL results (real prompts → real responses → real outputs §11.4.153) and NO false/simulated/stale/frozen result (§11.4.2/.5/.107) — a recording showing an error/bluff/non-working feature is a FINDING (→ §11.4.153(4)/§11.4.4 fix → retest → re-record), never a confirmation. The testing System MUST ACTUALLY READ every shown log line / message / UI label / dialog / toast / status text and VERIFY it is a genuine working result (OCR/ROI §11.4.117/.137 + confidence floor for pixel surfaces; direct capture for terminal/log; §11.4.107(10) self-validated golden-good/golden-bad analyzer) — “a video was produced” is NOT evidence, “the System read the screen + confirmed a genuine result” is. HelixQA (§11.4.27) MUST drive this exercise → record → read → score pass, PASS only on a read-confirmed genuine result + captured artefact path (§11.4.69). Default recording save path = \$HOME/Downloads (host user's home Downloads, resolved at runtime never hardcoded) unless a project declares an override per §11.4.35; §11.4.155 project-prefix + §11.4.154 window-scope/rotation + §11.4.128 git-ignored raw corpus + §11.4.83 curated docs/qa evidence apply. **Vision analysis MANDATORY for every recording** — after every recording, the agent MUST read the terminal output / video content and verify the feature actually works. A recording without a vision-confirmed verdict is a §11.4.158 violation. Composes

§11.4.2/.5/.25/.27/.52/.69/.83/.85/.107/.108/.117/.118/.128/.137/.138/.153/.154/.155/
§1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-158-
PROPAGATION (literal 11.4.158) + recommended gates CM-INTENSIVE-
RECORDING-COVERAGE + CM-RECORDING-CONTENT-READ-VERIFIED + paired §1.1
mutation. **Canonical authority:** constitution submodule Constitution.md
§11.4.158. Non-compliance is a release blocker. No escape hatch — no --skip-
recording-coverage , --video-without-content-read , --happy-path-
recording-suffices , --recording-path-anywhere , --unread-recording-OK ,
--skip-helixqa-read flag.

§11.4.159 — Mandatory window-specific video recording + vision validation mandate (User mandate, 2026-06-20). Every feature test, validation, verification, challenge, and QA session that produces video evidence MUST comply with ALL of the following: **(A) Window-specific recording ONLY** — every video MUST record ONLY the target application window (Terminal pane, TUI app, browser tab, emulator frame), NEVER the whole desktop/monitor screen; use macOS screencapture -l<window_id> , Linux xdotool + ffmpeg , or equivalent; whole-desktop capture leaks operator-private content (§11.4.10), dilutes the §11.4.107 liveness oracle, and breaks §11.4.137 OCR/ROI; platform genuinely cannot capture below whole-screen => honest §11.4.3 SKIP + tracked migration item. **(B) MP4 format REQUIRED** — .mp4 (H.264, movflags +faststart , pix_fmt yuv420p); .cast files are supplementary only; auto-convert via agg + ffmpeg per §11.4.154(C). **(C) Project-name prefix REQUIRED** — filename MUST start with project name in snake_case from HELIX_RELEASE_PREFIX in .env (§11.4.151) or lowercased project root dir name (§11.4.29); format <project_name>-<feature>-<timestamp>.mp4 ; unprefix = violation. **(D) Mandatory vision validation** — after EVERY recording, the agent MUST read the terminal output / video content and verify: (i) feature ACTUALLY WORKS, (ii) LLM responses are REAL, (iii) all tests show PASS, (iv) no “TODO implement” / “simulate” / “for now” patterns, (v) output demonstrates end-user working feature; MUST produce a verdict (PASS/FAIL) with evidence path (§11.4.69). **(E) Terminal window cleanup** — after each recording, dismiss/close ONLY the Terminal window used for that recording (window-specific close via osascript / xdotool); MUST NOT close windows belonging to other processes. **(F) Real results ONLY** — every recording MUST show REAL working features; errors/empty output/simulated responses trigger fix → retest → re-record. **(G) Re-runnable evidence** — the command shown MUST be re-runnable to produce the same results. **(H) Fresh-corpus rotation** — remove agent’s own prior in-scope stale recordings FIRST (§11.4.154). **(I) Content verification MANDATORY** — not

duration-based — the value of a recording is NOT its duration but its CONTENT; a recording MUST demonstrate the ACTUAL feature being used with REAL results; before accepting ANY recording, the agent MUST verify: expected output patterns ARE present (e.g., test PASS lines, API response data, feature-specific output), the feature ACTUALLY WORKS as demonstrated (not just “something ran”), LLM responses are REAL content (not simulated, not placeholder, not empty), every claim of “working” is backed by visible evidence; a 5-second recording showing a feature working correctly is MORE valuable than a 60-second recording of empty terminal; duration is NOT a proxy for quality. **(J) Expected-content specification REQUIRED** — before recording, the agent MUST specify what content SHOULD appear (expected patterns, expected test results, expected API responses); after recording, the agent MUST verify these patterns ARE present; if expected content is MISSING, the recording is REJECTED regardless of duration. **(K) Content-verification recording workflow** — mandated workflow: (1) SPECIFY expected content patterns, (2) RECORD the feature execution, (3) EXTRACT all text from the recording, (4) VERIFY expected patterns are present, (5) CHECK for simulated/placeholder content, (6) ACCEPT only if ALL patterns found AND zero bluffs detected, (7) REJECT and re-record if ANY pattern missing or bluff detected. **(L) Root cause analysis REQUIRED for rejected recordings** — when a recording is rejected (missing expected content, bluff detected, or empty capture), the agent MUST investigate WHY before re-recording per §11.4.102; determine the root cause (timing issue, wrong command, tool failure, etc.) and fix it; simply re-recording without understanding WHY the first attempt failed is a §11.4.159 violation. **(M) Real-time monitoring RECOMMENDED** — for complex features, use real-time monitoring that analyzes output DURING recording (not after); this catches issues immediately and allows corrective action before the recording completes.

Classification: universal (§11.4.17). Composes

§11.4.2/.3/.5/.10/.29/.69/.83/.107/.111/.128/.137/.151/.153/.154/.155/.158/§1.1.

Propagation gate CM-COVENANT-114-159-PROPAGATION (literal 11.4.159) +

recommended gate CM-WINDOW-VIDEO-VALIDATED + paired §1.1 mutation.

Canonical authority: constitution submodule Constitution.md §11.4.159. Non-compliance is a release blocker. No escape hatch — no --whole-screen-ok, --cast-only, --skip-vision-validation, --no-cleanup, --simulated-recording-ok, --unprefixed-recording flag.

§11.4.160 — Vision-verified recording + HelixQA bridge mandate (User mandate, 2026-06-21). Compact summary: every video recording for feature/QA evidence MUST be processed through a vision/OCR pipeline that reads on-screen

content and confirms expected results BEFORE acceptance; the recording system MUST provide a bridge feeding captured frames to HelixQA's test infrastructure (or equivalent) for automated read-the-screen verification against SPECIFY-phase expected patterns (§11.4.159(J)); the bridge MUST capture frames at ≤5s intervals, run OCR/vision analysis with a self-validated golden-good/golden-bad analyzer (§11.4.107(10)), compare extracted text against specified patterns, produce a per-frame PASS/FAIL verdict with an evidence path to the frame, and surface failures immediately so the recording can be re-done per §11.4.159(L). The capture interval, OCR confidence floor, and pattern-matching thresholds are project-configured per §11.4.35, calibrated on the project's own fixtures (§11.4.6). Honest boundary (§11.4.6): vision verification confirms the feature produced expected on-screen output — it does NOT replace §11.4.5 captured-evidence quality analysis (audio/video metrics) nor §11.4.108 runtime-signature verification; FLAG_SECURE/DRM-protected/sink-blanked surfaces document the gap per §11.4.112 and use the §11.4.117 proxy oracle instead. Classification: universal (§11.4.17). Composes §11.4.5/.27/.69/.107(10)/.112/.117/.153/.158/.159(J)/§1.1. Propagation gate CM-COVENANT-114-160-PROPAGATION + recommended gate CM-VISION-VERIFIED-RECORDING-BRIDGE + paired §1.1 mutation.

§11.4.161 — Rootless container runtime mandate (User mandate, 2026-06-21).

Compact summary: every project MUST use Podman in rootless mode (or equivalent rootless container runtime) for ALL containerized workloads — Docker in rootful mode, sudo, or any escalation to root for container management is STRICTLY FORBIDDEN unless the target platform has no rootless option AND that constraint is documented per §11.4.112; the vasic-digital/containers Submodule (§11.4.76) MUST be used as the sole container orchestration layer — no ad-hoc docker/podman commands outside the Submodule's pkg/boot / pkg/compose / pkg/health primitives; if a missing capability forces a raw command, the containers Submodule MUST be extended upstream per §11.4.74 rather than worked around; all container-related integration tests MUST boot infra on-demand via the Submodule (the on-demand-infra invariant). Honest boundary (§11.4.6): rootless execution eliminates the container-to-root privilege-escalation vector — it does NOT replace §11.4.10 credentials-handling (credentials in containers still require leak audits) nor §12.3 container hygiene (memory limits/OOM policies/restart backoff still apply). Classification: universal (§11.4.17). Composes §11.4.76/.74/.10/.112/§12.3/.6. Propagation gate CM-COVENANT-114-161-PROPAGATION + recommended gate CM-ROOTLESS-CONTAINER-RUNTIME + paired §1.1 mutation.

§11.4.162 — OpenDesign UI design system mandate (User mandate, 2026-06-21). Compact summary: every project producing user-facing interfaces (web, desktop, mobile, TUI) MUST use OpenDesign (<https://github.com/nexu-io/open-design>) as the mandatory UI design-and-refinement system — NOT ad-hoc CSS or one-off design tools; install as a project dependency and use its design tokens/themes system for: (a) all color palette definitions supporting BOTH light and dark themes from project brand assets (§11.4.35), (b) typography scale and spacing, (c) component-level design tokens; if a desired UI pattern is not supported, extend OpenDesign upstream per §11.4.74 (extend-don't-reimplement); every UI component MUST ship light + dark theme variants; elements MUST NOT overlap, fonts MUST NOT collide, labels MUST NOT overlay labels — any layout regression is a §11.4.162 violation; all UI changes MUST be covered by the project's standard test types including visual regression tests (before/after screenshots with per-pixel or perceptual-diff PASS/FAIL). Honest boundary (§11.4.6): OpenDesign governs design tokens and themes — it does NOT replace functional testing (§11.4.27), WCAG accessibility assertions (§11.4.107), nor the §11.4.48/§11.4.49 UI-driven and dual-approach testing methodology. Classification: universal (§11.4.17). Composes §11.4.74/.25/.27/.4(b)/.107/.48/.49/.35/.69/§1.1. Propagation gate CM-COVENANT-114-162-PROPAGATION + recommended gate CM-OPENDESIGN-UI-SYSTEM + paired §1.1 mutation.

§11.4.163 — Universal Media Validation & Verification Mandate (User mandate, 2026-06-21). Compact summary: every recorded artifact (video MP4, audio WAV, screenshots PNG, asciinema cast, text output) MUST pass through a MEDIA VALIDATION pipeline before acceptance — OCR for video/screenshots with confidence floor (§11.4.117/.107(12)), transcription for audio, text parsing for asciicast; compare extracted content against SPECIFY-phase expected patterns (§11.4.159(J)); self-validated analyzer with golden-good/golden-bad fixture pair (§11.4.107(10)) — golden-bad MUST FAIL or the pipeline is a bluff gate; produce structured verdict (PASS/FAIL + evidence path + matched/unmatched patterns + pinpoint data on FAIL — which frame/line/timestamp, expected vs actual); triggerable as post-recording AND real-time step (§11.4.159(M)); paired §1.1 mutation ensures golden-bad fixture produces FAIL. Honest boundary: confirms artifact content matches intended patterns — does NOT replace §11.4.5 quality analysis, §11.4.107 liveness, nor §11.4.108 runtime-signature. Classification: universal (§11.4.17). Composes §11.4.107(10)/.112/.117/.159(J)/.159(K)/.159(M)/.83/.5/.69/.102/.134/§1.1.

Propagation gate `CM-COVENANT-114-163-PROPAGATION` + recommended gate `CM-MEDIA-VALIDATION-PIPELINE` + paired §1.1 mutation.

§11.4.164 — Universal Constitution Auto-Propagation & Hook System (User

mandate, 2026-06-21). Compact summary: every fetch+pull of the constitution submodule MUST trigger `constitution/scripts/post_update_hook.sh` (inherited by reference §11.4.28, NEVER copied) that: (a) detects which files changed (`Constitution.md/CLAUDE/AGENTS/QWEN/GEMINI` + `scripts/ hooks/ skills/ mcp/ plugins/`) via `git diff --name-only`; (b) registers new/modified skills with the agent's skill system via consumer's `scripts/register_skills.sh`; (c) registers new/modified MCP servers via consumer's `scripts/register_mcp.sh`; (d) installs new/modified hooks into `.git/hooks/`; (e) makes scripts executable + validates syntax (`sh -n / bash -n` per §11.4.67); (f) emits a summary to stdout + `.constitution/state/last_update.log`; (g) on failure logs exact file:line and continues (§11.4.6); (h) tested by §1.1 mutation (add skill → hook detects it).

Consumer MUST invoke hook after every constitutional pull; failure to invoke is non-compliance. Honest boundary: installs components — does NOT guarantee consumer runtime acceptance nor replace §11.4.32 validation sweep. Classification: universal (§11.4.17). Composes §11.4.28/.32/.35/.67/.77/.102(B)/.122/.31/§1.1.

Propagation gate `CM-COVENANT-114-164-PROPAGATION` + recommended gate `CM-CONSTITUTION-AUTO-PROPAGATION` + paired §1.1 mutation.

§11.4.165 — Universal Independent Verification Agent Mandate (User

mandate, 2026-06-21). Compact summary: every code change OR recorded media artifact MUST pass an INDEPENDENT verifier structurally separate from the author (§11.4.70/.20, NEVER author self-review — §11.4.92 PRECEDES, never satisfies), producing structured findings with evidence paths and iterating to GO per §11.4.134. (a) CODE: §11.4.142 review + build+test diff scope + run paired §1.1 mutations + verify §11.4.108 runtime-signature registry; (b) MEDIA: run §11.4.163 pipeline + confirm genuine content (no mock/stub/placeholder §11.4.159(I)) + confirm format (MP4 H.264/WAV/PNG/cast+mp4 §11.4.159(B)) + cross-check evidence path; (c) DOCS: validate HTML+PDF+DOCX exports current §11.4.65 + revision header incremented §11.4.44 + §11.4.86 fingerprint match; (d) CONFIG: validate YAML/JSON/TOML syntax + schema cross-reference + §11.4.10 leak check; (e–f) structured findings, iterate to zero-finding GO; (g) verifier self-validated by §1.1 mutation (knowingly broken change MUST be caught). Honest boundary: confirms source/artifact integrity — does NOT replace §11.4.108 runtime-signature nor §11.4.40 full-suite retest. Classification: universal (§11.4.17). Composes §11.4.142/.163/.44/.65/.86/.108/.125/.134/.10/.40/.92/.70/.20/.102/.159(I)/.159(B)/

§1.1. Propagation gate `CM-COVENANT-114-165-PROPAGATION` + recommended gate `CM-INDEPENDENT-VERIFICATION-AGENT` + paired §1.1 mutation.

§11.4.166 — REPEALED (operator decision, 2026-06-22). The Universal Semgrep static-analysis mandate is repealed — Semgrep is NO LONGER mandatory. Static analysis remains encouraged, not mandated. Scaffolding, `submodules/semgrep`, `MCP/pre-commit/PATH` wiring, `docs_chain` `semgrep_status` context, and the `CM-COVENANT-114-166-PROPAGATION` / `CM-SEMGREP-WIRED` gates removed. Anchor 11.4.166 retired. See constitution `Constitution.md` §11.4.166.

§11.4.167 — Big-work-item feature work-stream lifecycle (User mandate, 2026-06-23). Compact summary: every BIG work item (new feature OR drastic/ large fix) MUST be developed as its own isolated **feature work-stream** — its OWN full project copy in a sibling directory, its OWN `feature/<slug>` branch, its OWN per-feature flashable builds + feature release tags, kept SEPARATE from the trunk until the operator manually approves after testing, with the trunk regularly merged INTO every feature stream. (A) one big item = one feature work-stream in a sibling project copy (`<project>_<slug>`, §11.4.29 slug); small changes stay on the trunk. (B) disk-feasible duplication MANDATORY, naive deep copies FORBIDDEN — a CoW/reflink clone (`cp -a --reflink=auto` / FS snapshot) on btrfs/XFS/ZFS/APFS, else a shared-object-store mechanism (`git worktree` + externalized build output); per-stream `out/` excluded from the clone + one shared ccache; ~0-disk-per-fresh-stream verified by captured evidence (§11.4.69) before relied on (§11.4.6 no-guessing). (C) own branch + own tags greppable per §11.4.151 (`<base>-feat-<slug>[-<iter>]`), NEVER merged to trunk until §11.4.40 full retest GREEN on a clean target → §11.4.41 merge-first → §11.4.113 ff-only (no-force) merge. (D) trunk merged INTO every live stream FREQUENTLY (per trunk tag / daily, `git merge` never rebase a shared/tagged branch); a stale stream is flagged. (E) feature branch + mirrored feature tag cascades to EVERY touched submodule the moment a submodule is modified (§11.4.113 + tag-cascade); untouched submodules stay trunk-pinned. (F) single-builder build QUEUE (FIFO + advisory lock, §11.4.58 L1 / §12.7 / §12.8) + finite-device test queue (§11.4.119 per-device exclusive ownership); idle streams' regenerable build output GC'd (§11.4.77). (G) every change crosses the full quality gauntlet (§11.4.142/.125/.134 review + §11.4.145 impact research + §11.4 / .5 / .69 / .107 anti-bluff testing) — a feature stream is NOT a quality carve-out. (H) resumable, data-safe lifecycle — a stream registry (§11.4.116 / §11.4.147) tracks branch/base-tag/out-state/last-trunk-merge/merge-

approval; atomic `.partial` → rename create + §11.4.84 quiescence resume; retire only after merged/approved + §9.2 backup; bounded by §12 / §12.6. Honest boundary (§11.4.6): guarantees ISOLATION + disk-feasibility + greppable identity + controlled trunk↔feature merge — NOT that any stream's work is correct (still crosses §11.4.108 + §11.4.40). Classification: universal (§11.4.17) — consumer supplies its CoW mechanism, sibling-dir + tag naming, build/device-queue counts, big-item threshold per §11.4.35. Propagation gate `CM-COVENANT-114-167-PROPAGATION` (literal `11.4.167`) + recommended gates `CM-FEATURE-WORKSTREAM-COW-CLONE` / `CM-FEATURE-WORKSTREAM-NO-MERGE-UNTIL-APPROVED` / `CM-FEATURE-WORKSTREAM-TRUNK-SYNC-CADENCE` / `CM-FEATURE-WORKSTREAM-SUBMODULE-CASCADE` + paired §1.1 mutations. **Canonical authority:** constitution submodule `Constitution.md` §11.4.167. Non-compliance is a release blocker. No escape hatch — no `--full-clone-per-feature`, `--deep-copy-stream`, `--skip-cow-clone`, `--merge-feature-without-approval`, `--no-trunk-sync`, `--skip-submodule-cascade`, `--unbounded-parallel-builds`, `--shared-device-no-lock`, `--retire-unmerged-stream`, `--feature-stream-quality-carveout` flag.

§11.4.168 — Exported-document independent content + textual + full-visual validation mandate (User mandate, 2026-06-23). Compact summary: every generated/exported document (HTML / PDF / DOCX / any format in the §11.4.65 export scope) MUST pass INDEPENDENT validation by a review agent structurally separate from the generator (§11.4.70/§11.4.142, NEVER the author self-checking), on BOTH the source AND every exported artifact, ALWAYS, across THREE layers: (1) CONTENT — the export faithfully carries the source's intent + data, nothing dropped/truncated/garbled; (2) TEXTUAL — human-readable, NO raw markup / diagram-source (Mermaid `gantt` / `graph` / `flowchart` / `sequenceDiagram` etc.) / unrendered code-fence leaking as body text (the exact 2026-06-23 raw-gantt-in-PDF failure); (3) FULL VISUAL — embedded diagrams render as IMAGES not source, layout intact, no overlap/cut-off/garble/collapse, verified by RENDERING the export (`pdftotext` catches raw source as text + `pdfimages` confirms rendered images + `pdftoppm` → OCR confirms human-readable visual content) with captured evidence per §11.4.5/§11.4.69/§11.4.107. Anti-bluff: a rubber-stamp is not validation; the analyzer is self-validated golden-good/golden-bad per §11.4.107(10) (a golden-bad export seeded with raw `gantt` source MUST FAIL); iterate to a clean GO per §11.4.134. Forensic FACT (2026-06-23): a “green” Mermaid fix shipped raw `gantt` source as readable text into user-facing PDFs because validation grepped the HTML for `class="mermaid"` and NEVER checked the exported PDF — operator

caught it (§11.4.138). A generated doc shipping without this content+textual+visual validation, or containing raw diagram source / unreadable garble, is a §11.4 PASS-bluff at the documentation layer. Honest boundary (§11.4.6): confirms the exported artifact a reader opens is faithful/readable/visually-rendered — does NOT prove the source content is correct (§11.4.142/§11.4.135) nor replace the §11.4.65 mtime-parity freshness gate (this is the READABILITY/FIDELITY layer the mtime gate cannot see); FLAG_SECURE/un-rasterisable surfaces document the gap per §11.4.112 + §11.4.117 proxy oracle, never a faked visual PASS. Classification: universal (§11.4.17). Composes

§11.4.65/.73/.107/.117/.135/.138/.142/.159/.163/.165/§1.1. Propagation gate CM-COVENANT-114-168-PROPAGATION (literal 11.4.168) + recommended gate CM-EXPORTED-DOC-VISUALLY-VALIDATED + paired §1.1 mutation (a PDF/HTML/DOCX containing raw diagram source as body text → gate FAILs). **Canonical authority:** constitution submodule Constitution.md §11.4.168. Non-compliance is a release blocker. No escape hatch — no --skip-visual-validation , --raw-source-in-pdf-ok , --textual-check-suffices , --self-validate-export , --diagram-source-as-text-ok flag.